

Trabajo Final - Cronos Notes

Universidad de la Cuenca del Plata - Sede Formosa

Facultad de Ingeniería, Tecnología y Arquitectura

Lic. En Sistemas de Información

Ingeniería de Software II - Prof. Dr. Javier Ruiz Díaz

Programación en Ambientes Web - Prof. Ing. Silvia Rosana Hoferek

Junio del 2026

Alumnos:

Ayala, José Andrés – DNI: 46.065.017

Dellagnolo, Ricardo Agustín – DNI: 45.817.894

Fornari, Nahir Agustín – DNI: 46.154.364

Romea Acevedo, Clara Agustina - DNI: 46.154.215

Índice

Introducción	8
Técnicas de Elicitación	9
<i>Cuestionarios</i>	9
<i>Análisis de Sistemas en Internet</i>	14
Lofi.co	14
Lofi.cafe	16
StudyFetch	17
LifeAt	18
Contexto	21
<i>Necesidad/Problemática</i>	21
Propuesta de Desarrollo	23
<i>Alcances del Sistema</i>	24
<i>Límites del Sistema</i>	25
Objetivos	26
<i>Objetivo General</i>	26
<i>Objetivos Específicos</i>	26
Planificación	27
<i>Metodología de Desarrollo</i>	27
<i>Cronograma</i>	28
Duración Total del Proyecto	28
Incremento N°1: Semanas 1 y 2 (del 20 de abril al 3 de mayo)	28
Incremento 2: Semanas 3 y 4 (del 4 de mayo al 17 de mayo)	29
Incremento 3: Semanas 5 y 6 (del 18 de mayo al 31 de mayo)	30
Incremento 4: Semanas 7 y 8 (del 1 de junio al 14 de junio)	31
Recursos	32
<i>Recursos Humanos</i>	32
<i>Recursos Técnicos (Hardware y Software)</i>	33
<i>Desarrollo / Recursos del Equipo</i>	33
<i>Stack Tecnológico</i>	33
<i>Librerías Específicas y Utilidades</i>	36
<i>Herramientas de Construcción y Base de Datos</i>	37
<i>Recursos del Cliente</i>	38
Hardware Requerido	38
Software Requerido	38
Requerimientos	40
<i>Requerimientos Funcionales de Mejora</i>	42
<i>Requerimientos No Funcionales</i>	44
<i>Requerimientos de Usuario</i>	45
Diagrama de Contexto	47
Diagrama de Casos de Uso	48
Diagrama de Clases	55
Interfaces de Usuario	56

Metodología a Utilizar	56
Diseño de las Pantallas	56
Logo de la Aplicación	56
Página de Bienvenida	57
Registro - Modo Claro y Oscuro	57
Inicio de Sesión - Modo Claro y Oscuro	58
Página Principal/Dashboard - Modo Claro y Oscuro	59
Página de Perfiles	60
Página de Tareas	60
Página de Sesión - Modo Claro y Oscuro	61
Página de Calendario	62
Modal de Nueva Tarea	63
Página de Toma de Apuntes	64
Estructura del Proyecto	65
Carpetas del Proyecto	65
Explicación de los Directorios	66
Capa Backend: app/	66
Capa Frontend: resources/js/	66
Base de Datos	67
Codificación del Proyecto	68
RF-01: Autenticación de Usuario	68
Descripción y Objetivo	68
Tecnologías, Herramientas y Librerías	68
Archivos Involucrados en el Requerimiento	69
Flujo de Datos y Control	71
Pruebas y Validación	73
RF-02: Gestión de Perfiles	75
Descripción y Objetivo	75
Reglas de Negocio y Restricciones	75
Tecnologías, Herramientas y Librerías	75
Archivos Involucrados en el Requerimiento	76
Flujo de Datos y Control	77
Pruebas y Validación	79
RF-03: Gestión de Tareas	80
Descripción y Objetivo	80
Tecnologías, Herramientas y Librerías	80
Archivos Involucrados en el Requerimiento	81
Flujo de Datos y Control	82
Pruebas y Validación	84
RF-04: Sesión Pomodoro	86
Descripción y Objetivo	86
Tecnologías, Herramientas y Librerías	86
Archivos Involucrados en el Requerimiento	87
Flujo de Datos y Control	88

Pruebas y Validación	90
RF-05: Sistema de Rachas	92
Descripción y Objetivo	92
Tecnologías, Herramientas y Librerías	92
Archivos Involucrados en el Requerimiento	93
Flujo de Datos y Control	94
Pruebas y Validación	96
RF-06: Sistema de Estadísticas	98
Descripción y Objetivo	98
Tecnologías, Herramientas y Librerías	98
Archivos Involucrados en el Requerimiento	99
Flujo de Datos y Control	100
1. Petición de Interpretación de Racha	100
Pruebas y Validación	101
RF-M01: Alternancia a Modo Zen	103
Descripción y Objetivo	103
Tecnologías, Herramientas y Librerías	103
Archivos Involucrados en el Requerimiento	104
Flujo de Datos y Control	105
Pruebas y Validación	106
RF-M02: Ejecución de Inicio Rápido	107
Descripción y Objetivo	107
Tecnologías, Herramientas y Librerías	107
Archivos Involucrados en el Requerimiento	107
Flujo de Datos y Control	108
Pruebas y Validación	109
RF-M03: Renderizado de Calendario Visual	110
Descripción y Objetivo	110
Tecnologías, Herramientas y Librerías	110
Archivos Involucrados en el Requerimiento	111
Flujo de Datos y Control	111
Pruebas y Validación	112
RF-M04: Procesamiento de IA para Prioridad	113
Descripción y Objetivo	113
Tecnologías, Herramientas y Librerías	113
Archivos Involucrados en el Requerimiento	114
Flujo de Datos y Control	114
Pruebas y Validación	115
RF-M05: Editor de Apuntes en Tiempo Real	116
Descripción y Objetivo	116
Tecnologías, Herramientas y Librerías	116
Archivos Involucrados en el Requerimiento	117
Flujo de Datos y Control	118
Pruebas y Validación	119

<i>RF-M06: Mezclador de Sonidos</i>	120
Descripción y Objetivo	120
Tecnologías, Herramientas y Librerías	120
Archivos Involucrados en el Requerimiento	121
Flujo de Datos y Control	121
Pruebas y Validación	122
<i>RF-M07: Sesión de Invitado</i>	123
Descripción y Objetivo	123
Tecnologías, Herramientas y Librerías	123
Archivos Involucrados en el Requerimiento	124
Flujo de Datos y Control	125
Pruebas y Validación	126
<i>RF-M08: Conexión con APIs Externas</i>	127
Descripción y Objetivo	127
Tecnologías, Herramientas y Librerías	127
Archivos Involucrados en el Requerimiento	128
Flujo de Datos y Control	129
Pruebas y Validación	130
<i>RF-M09: Registro de Cancelación de Sesión</i>	131
Descripción y Objetivo	131
Tecnologías, Herramientas y Librerías	131
Archivos Involucrados en el Requerimiento	132
Flujo de Datos y Control	133
Pruebas y Validación	134
<i>RF-M10: Estimación de Esfuerzo</i>	135
Descripción y Objetivo	135
Tecnologías, Herramientas y Librerías	135
Archivos Involucrados en el Requerimiento	135
Flujo de Datos y Control	136
Pruebas y Validación	137
<i>RF-M11: Perfil de Usuario Compartido</i>	138
Descripción y Objetivo	138
Tecnologías, Herramientas y Librerías	139
Archivos Involucrados en el Requerimiento	139
Flujo de Datos y Control	141
Pruebas y Validación	142
<i>RF-M12: Cambios en el Sistema de Rachas</i>	144
Descripción y Objetivo	144
Tecnologías, Herramientas y Librerías	144
Archivos Involucrados en el Requerimiento	145
Flujo de Datos y Control	146
Pruebas y Validación	147
<i>RF-M13: Cambios en las Estadísticas</i>	148
Descripción y Objetivo	148

Tecnologías, Herramientas y Librerías	149
Archivos Involucrados en el Requerimiento	149
Flujo de Datos y Control	150
Pruebas y Validación	151
Calidad del Software	153
<i>Cuestionario de Evaluación de Calidad (20 Preguntas)</i>	154
Adecuación Funcional	154
Eficiencia de Desempeño	154
Compatibilidad	155
Usabilidad	155
Fiabilidad	155
Seguridad	156
Mantenibilidad	156
Portabilidad	157
Análisis de riesgos	158
<i>1. Desincronización y Ruptura Falsa de Rachas por Zonas Horarias (Streak Inconsistency)</i>	158
Descripción	158
Impacto	158
<i>2. Conflictos de Concurrencia y Pérdida de Datos en Perfiles Compartidos (Race Conditions)</i>	158
Descripción	158
Impacto	158
<i>3. Pérdida de Datos en Grabaciones de Audio por Caídas de Conexión o Cierres de Pestaña</i>	158
Descripción	158
Impacto	159
<i>4. Vulnerabilidad y Dependencia Extrema de APIs de Terceros (Google)</i>	159
Descripción	159
Impacto	159
<i>5. Cuello de Botella y Agotamiento de Almacenamiento en Servidor por Archivos de Audio</i>	159
Descripción	159
Impacto	159
Hoja de Información de Riesgo y Plan de Continencia	160
Conclusión	165
Anexos	166
<i>Respuestas del Cuestionario</i>	166

Introducción

Este proyecto se desarrolla como una intercátedra entre Ingeniería de Software II y Programación en Ambientes Web. El propósito central es la evolución y escalamiento de **Cronos Notes**, una plataforma web ya desarrollada por el mismo equipo de trabajo, para transformarla en una herramienta de alto rendimiento que ayude a estudiantes y profesionales a organizar su tiempo evitando el agotamiento.

El desarrollo se fundamenta en la resolución de la fragmentación tecnológica actual, donde el usuario debe alternar entre múltiples aplicaciones para gestionar tareas, cronometrar tiempos y tomar apuntes. Mediante la integración de pilares como la gestión por perfiles, gestión de tareas por prioridad, el uso de la técnica Pomodoro y un seguimiento de estadísticas para motivar al usuario, Cronos Notes busca combatir la procrastinación y reducir el agotamiento.

En esta nueva etapa, buscamos que la aplicación vaya más allá de una agenda básica. Para eso, vamos a implementar mejoras clave como la integración de Inteligencia Artificial para priorizar tareas automáticamente, un módulo avanzado de toma de apuntes (con métodos como Cornell), y la sincronización con herramientas externas como Google Calendar y Spotify. También sumaremos funciones para mejorar la experiencia de usuario, como un Modo Zen minimalista, un mezclador de sonidos de ambiente y un Modo Invitado para probar el sistema sin registrarse.

Técnicas de Elicitación

Para determinar las mejoras a implementar en el sistema existente, el equipo optó por las siguientes técnicas de elicitación de requisitos:

Cuestionarios

Se utilizó **Google Forms** para administrar un cuestionario. Este tenía como objetivo principal solicitar a los encuestados que valoraran la importancia de las mejoras propuestas para el sistema. Adicionalmente, se incluyó un espacio para que pudieran sugerir características o funcionalidades no mencionadas previamente.

Luego de una semana de que el cuestionario fue publicado, se obtuvieron un total de 50 respuestas. A continuación se detallan las preguntas que se realizaron, en conjunto con las conclusiones:

- **Pregunta N°1: ¿Cuál es tu principal motivo para usar un gestor de tareas?**
 - Preparación de exámenes académicos.
 - Gestión de proyectos personales o laborales.
 - Mejorar la concentración mediante la técnica Pomodoro.
 - Otro: _____

La gran mayoría se divide entre la **Opción 2** (54%) y la **Opción 1** (38%). Una minoría lo usaría estrictamente por la técnica Pomodoro o para organización diaria general.

Esto nos deja ver que Cronos debe equilibrar herramientas de gestión de proyecto con herramientas de estudio, para satisfacer ambos nichos.

- **Pregunta N°2: Calificar del 1 al 5 (donde 1 es ‘Nada importante’ y 5 es ‘Muy importante’) respecto a las siguientes propuestas para el usuario:**

- **Inicio Rápido:** Poder iniciar un temporizador sin configurar perfiles o tareas previas.
- **Modo Zen:** Una vista minimalista que solo muestre el tiempo y la tarea actual, ocultando distracciones.
- **Estimación de Esfuerzo:** Indicar cuántas sesiones Pomodoro crees que te llevará completar una tarea específica al iniciarla.
- **Ambiente Inmersivo:** Uso de imágenes o GIFs relajantes durante el temporizador para reducir el estrés.

De esta pregunta, pudimos sacar varias conclusiones:

- Existe un gran interés por el “**Inicio Rápido**” y el “**Modo Zen**”. Los usuarios nos dan a entender que estas funciones son las más deseadas, ya que la mayoría de respuestas se encuentran entre 4 y 5. Iniciar un cronómetro debe ser un proceso rápido y sencillo. También desean una reducción de distracciones visuales.
- La propuesta de “**Estimación de Esfuerzo**” tiene una valoración Media-Alta, donde la mayoría de respuestas van entre 3 y 5. Por lo tanto, si bien es una funcionalidad útil para la planificación, es menos crítica que las anteriormente mencionadas.
- Respecto a la opción de “**Ambiente Inmersivo**”, es la propuesta más dividida de todas, obteniendo respuestas desde 1 hasta 5. Lo que para unos es relajante, para los demás es una distracción. Debido a esto, creemos que debe ser una opción que los usuarios puedan activar y desactivar a gusto.

- **Pregunta N°3: ¿Qué tanto te motivarían los siguientes elementos para mantener una racha de estudio?**
 - **Sistema de Niveles:** Ganar experiencia (XP) y subir de nivel al completar sesiones de estudio.
 - **Recompensas Visuales:** Desbloquear nuevos temas de colores, iconos o sonidos exclusivos al alcanzar metas de horas de uso.
 - **Competencia Social:** Rankings semanales para comparar tu progreso con otros estudiantes.
- Opciones disponibles: **Nada, Poco, Neutral, Bastante, Mucho**

Al igual que la pregunta anterior, el equipo de desarrollo concluyó que:

- El factor más motivador es tener **recompensas visuales** (como medallas o íconos) a medida que uno utiliza el sistema.
- Los elementos de **Sistema de Niveles** y **Competencia Social** fueron de menor interés que el anterior, pero siguen teniendo un gran impacto en la valoración de los usuarios

Esto nos da a entender que los potenciales usuarios de Cronos prefieren una mejora personal estética antes que competir con el resto. En consecuencia, se priorizará la personalización visual antes que un ranking social.

- **Pregunta N°4: ¿Qué herramientas adicionales considerarías esenciales dentro de la misma aplicación? Selecciona, cómo máximo, tres opciones.**
 - **Toma de Apuntes:** Un módulo integrado para notas con métodos como Cornell o esquemas jerárquicos.
 - **Calendario Visual:** Una vista mensual para observar fechas límite y tareas programadas.

- **Mezclador de Sonido/Integración con Spotify:** Personalizar tu ambiente combinando lluvia, ruido blanco y música de fondo.
- **Salas de Estudio:** Espacios virtuales para compartir sesiones de cronómetro con otros usuarios.
- **Ventana Flotante:** Que el temporizador se mantenga visible en una ventana pequeña mientras navegas en otras pestañas.
- **Inteligencia Artificial:** Una IA que ayude a priorizar tareas basándose en la fecha límite y/o prioridad.

Las tres opciones más votadas fueron el **Calendario Visual**, la **Inteligencia Artificial** y la **Toma de Apuntes**. Los usuarios buscan tener varias herramientas integradas para aumentar su productividad. Por ende, se le dará mayor prioridad a estas tres herramientas antes que a las demás que obtuvieron menos votos.

- **Pregunta N°5: Calificar del 1 al 5 (donde 1 es ‘Nada importante’ y 5 es ‘Muy importante’) ¿Qué tan relevante es poder integrar Cronos con otras herramientas que ya utilizas (ej. Google Calendar, Spotify)?**

La mayoría (37 de 50 usuarios) creen que es muy importante integrar aplicaciones/herramientas que ya utilizan en su vida cotidiana, con votaciones entre 4 y 5.

En consecuencia, el equipo considera que es importante cumplir con esta demanda de los usuarios, para así también evitar que los mismos “salten” entre diversas aplicaciones para realizar sus actividades y aumentar la retención de uso.

- **Pregunta N°6: ¿Cómo preferirías que el sistema maneje la procrastinación?**

- Prefiero que registre cuántas veces cancelo sesiones antes de tiempo.
- Me gustaría recibir mensajes de motivación si detecta que estoy cancelando sesiones seguido.
- Otro: _____

Las preferencias del usuario están muy divididas entre **Mensajes motivacionales** y el **Registro de Cancelaciones**. Otros usuarios sugieren métodos más agresivos como “ruidos fuertes”.

De esta pregunta, podemos concluir que un sistema híbrido que muestre un mensaje motivador y registre la falla en la estadística sería la solución óptima.

- **Pregunta N°9: Calificar del 1 al 5 (donde 1 es ‘Nada importante’ y 5 es ‘Muy importante’) ¿Qué tan importante es para ti poder probar la aplicación en un "Modo Invitado" antes de registrarte formalmente?**

30 de 50 respuestas indican que es ‘Importante’ o ‘Muy importante’ tener un modo invitado para probar algunas funcionalidades de la página antes de registrarse. 15 encuestados consideran que es algo ‘Poco importante’, mientras que el resto lo consideran ‘Nada importante’

Por esta razón, consideramos que es importante implementar un modo invitado, para aumentar la tasa de retención inicial y minimizar la barrera de entrada al no obligar a un registro inmediato.

- **Pregunta N°10: ¿Qué otra característica o mejora te gustaría que el sistema Cronos tenga que no haya sido mencionada?**

Se obtuvieron 13 respuestas para esta pregunta, dado que no es de carácter obligatorio. Algunas de las respuestas obtenidas fueron:

- “Un calendario donde solo anotar fechas facultativas, y que me las recuerde por medio de notificaciones”.
 - “Una interfaz con historial de etapas laborales o de estudio, y de periodos de descanso”.
 - “Que proponga ideas sobre qué hacer en los descansos, como técnicas de respiración o enseñanzas de cómo manejar el tiempo”.
 - “Implementar una lista de tareas grupales para el caso de trabajo o sesiones de estudio grupales”.
- Podemos observar que existen demandas por funciones de bienestar y colaboración entre usuarios que no estaban contempladas originalmente. Sin embargo, al ser mejoras que no fueron mayoritariamente nombradas, quedarán como mejoras a posterior para una versión futura de la aplicación.

Análisis de Sistemas en Internet

Se llevó a cabo una búsqueda en internet para recolectar información acerca de las aplicaciones ya existentes en el mercado, analizando las funcionalidades que estas presentan. Algunas de las aplicaciones analizadas fueron:

Lofi.co

Aplicación web que evolucionó y ahora forma parte de una experiencia más amplia bajo el nombre de *Lofi Garden*. Consistía en un generador de atmósferas, diseñado para combatir la

soledad del trabajo remoto y la monotonía del estudio.

Entre sus principales características, se encontraban:

Entornos Visuales Inmersivos. La característica principal eran los escenarios ilustrados, muchos en un estilo *pixel art*. Se podía elegir entre habitaciones en un día lluvioso, una cafetería en una ciudad concurrida, un bosque tranquilo bajo las estrellas y demás. Del mismo modo, casi todos los escenarios permitían alternar entre **modo día/noche** con un solo clic, cambiando la iluminación y el ambiente.

Música y Mezclador de Sonido Ambiental. Los usuarios podían mezclar la música que ya ofrecía la aplicación (ritmos relajados, animados o para dormir) con sonidos de fondo realistas, gracias a barras de volumen individuales. Entre las opciones de sonido, se encontraban:

Ruido de Lluvia. Desde una llovizna ligera hasta una tormenta.

Ambiente de Cafetería. Sonido de tazas chocando y gente murmurando.

Mezclador de Ruidos. Opción de ruido marrón, blanco o rosado.

Naturaleza. Viento, grillos o el crepitar de una fogata.

Herramientas de Productividad Integradas. Si bien el objetivo principal de la aplicación era permitir al usuario que se relaje, también se buscaba que aumente la productividad del mismo. Por lo tanto, en pantalla se incluían los siguientes *widgets*:

Temporizador Pomodoro. Temporizador personalizable para gestionar ciclos de trabajo y descanso.

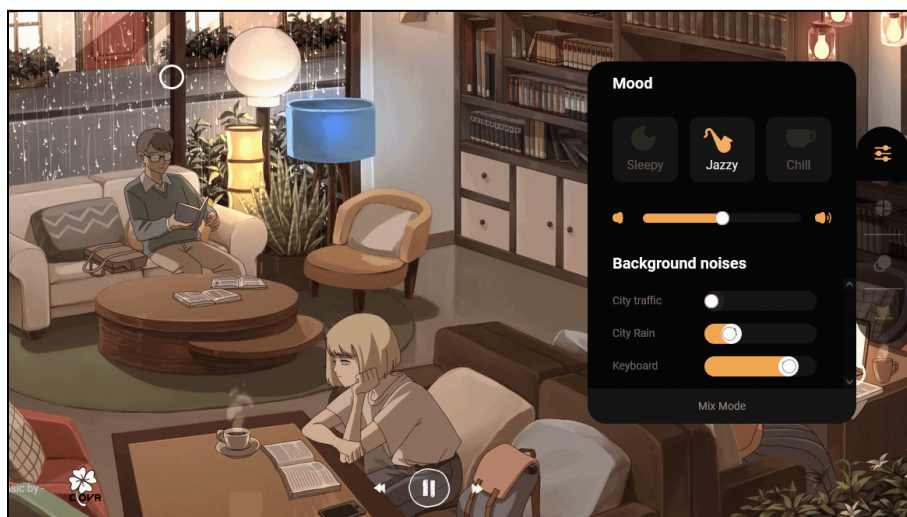
Lista de Tareas. Permite hacer una *to-do list* para no tener que salir de la pestaña.

Bloc de Notas. Para realizar apuntes rápidos.

Contador de Tiempo. Para saber cuánto tiempo se llevaba conectado en la página.

Figura 1

Interfaz de lofi.co donde se observan el mezclador y reproductor de sonido



Lofi.cafe

Una radio web completamente minimalista en términos de diseño, dado que su objetivo es reducir la carga cognitiva de los usuarios. Entre sus características principales se encuentran:

Interfaz Libre de Distracciones. La pantalla prácticamente no posee *widgets* extras más que el contenido, el cual consiste en una lista de transmisiones de YouTube con música relajante.

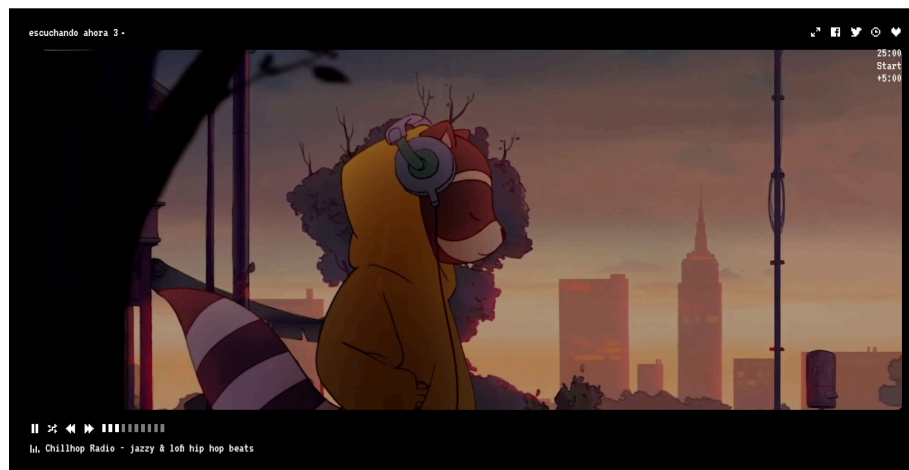
Atajos de Teclado. Se puede cambiar de estación, pausar o ajustar el volumen sin usar el mouse, manteniendo las manos en posición de trabajo

Estética Retro. Utiliza una paleta de colores y fuentes que reducen la fatiga visual.

Temporizador Pomodoro. Implementa un cronómetro sencillo de 25 minutos como mínimo, el cuál puede ir aumentando en incrementos de 5 minutos.

Figura 2

Captura de pantalla de la página lofi.cafe



StudyFetch

Esta plataforma está diseñada para el procesamiento activo de la información. Si bien su valor no reside en el ambiente, posee una transformación de datos que le permite hacer la diferencia con aplicaciones similares. Las características principales de StudyFetch son:

Spark.ai (Tutor Personalizado). A diferencia de un chatbot convencional, StudyFetch posee una inteligencia artificial que procesa los materiales adjuntados (sea PDFs, grabaciones, vídeos) y responde basándose exclusivamente en los mismos.

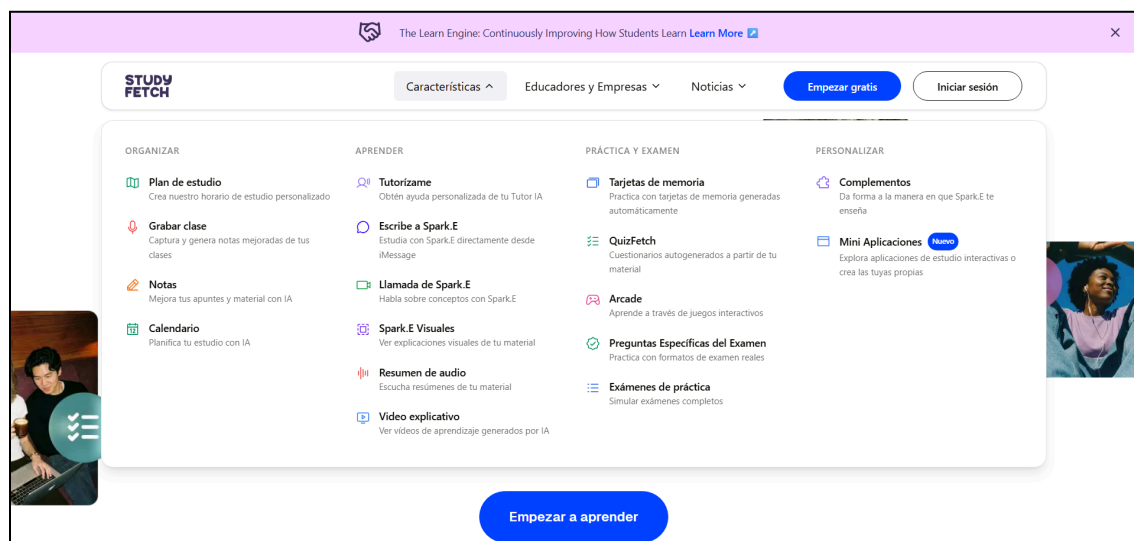
Generación Automática de Herramientas de Estudio. Permite crear *flashcards*, guías de estudio y cuestionarios de práctica a partir de un archivo subido.

Transcripción en Tiempo Real. Se pueden grabar clases y convertirlas instantáneamente en notas estructuradas.

UI estilo Dashboard. Posee una interfaz densa y cargada, dado que prioriza la gestión de múltiples cursos y materiales.

Figura 3

Página principal de StudyFetch, mostrando las funcionalidades que posee



LifeAt

En la actualidad es, probablemente, la herramienta digital más ambiciosa en lo que respecta a la personalización de un espacio de trabajo, dado que permite centralizar gran cantidad de información. Dentro de las características clave que posee esta aplicación, se destacan:

Espacio Virtual a Gran Escala. A diferencia de, por ejemplo, la ya inexistente y mencionada *lofi.co*, LifeAt cuenta con una biblioteca de más de 1000 fondos/espacios. La mayoría son fragmentos de vídeos en alta definición con sonido ambiental vinculado a la

imágen, las cuales pueden cambiarse con un clic, algo vital y rápido para los usuarios que desean ir variando entre entornos visuales.

Integración de Herramientas. El software prioriza que te mantengas en el mismo el mayor tiempo posible, por lo que brinda una gran cantidad de *widgets* y funcionalidades:

Sincronización con Google Calendar. Permite ver los eventos del día registrados sobre la aplicación mencionada.

Lista de Tareas Avanzada. Permite organizar tareas por prioridad y tacharlas mientras corre el tiempo.

Integración con Spotify y Apple Music. Tiene un reproductor que permite conectarse a las cuentas de los usuarios, permitiendo escuchar playlists propias sin cambiar de pestaña.

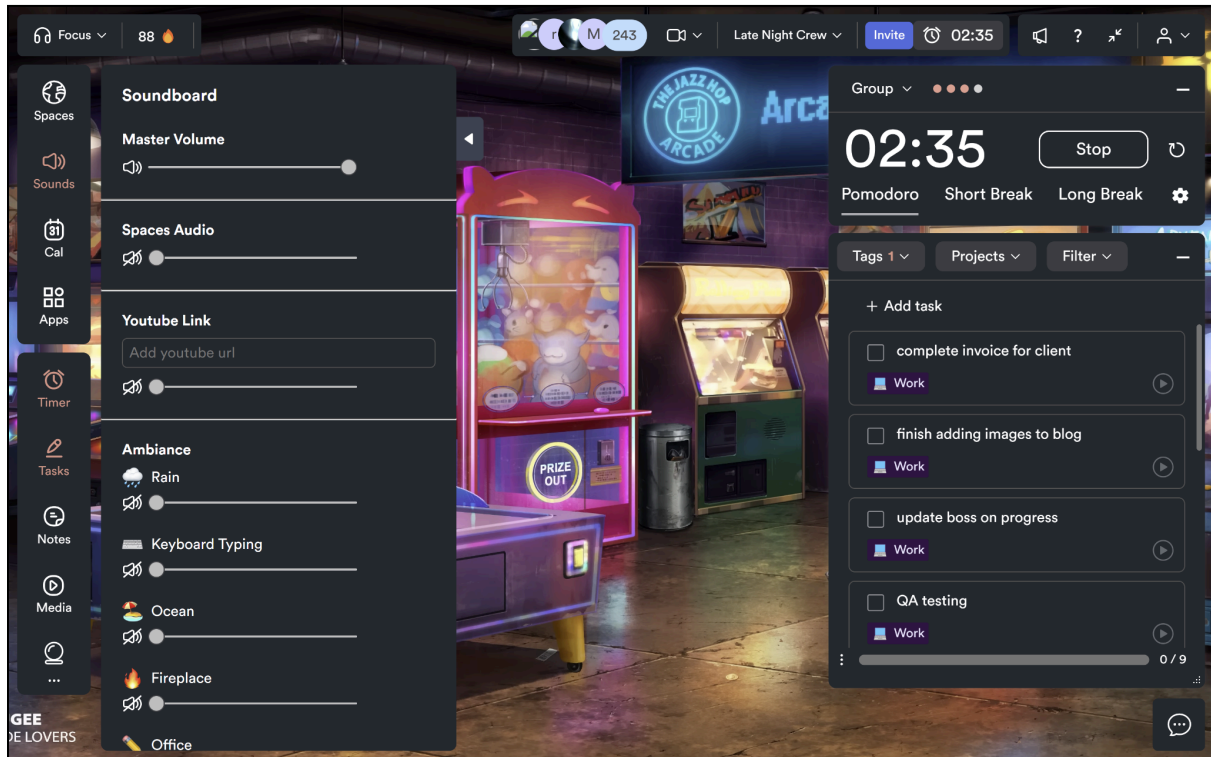
Modo Respiración. Incluye pequeñas herramientas con ejercicios de concentración y respiración guiada en momentos de estrés.

Salas de Estudio Virtuales. Es la característica diferenciadora de la aplicación. Permite unirse a salas donde otras personas están en periodos de concentración. También permite crear salas privadas, compartiendo el link con otros usuarios para trabajar “juntos” en un entorno estético compartido.

Gamificación y Retención. Implementa mecánicas de juegos para asegurar que el usuario se mantenga activo. Entre estas mecánicas están la **racha**, un contador de días consecutivos en los que se usa la app para concentrarse.; y las **monedas o premios**, recompensas por tiempo de enfoque que permiten desbloquear nuevas imágenes o videos para el espacio de concentración.

Figura 4

Escritorio virtual de LifeAt, con sus widgets personalizables.



Contexto

En los últimos años, la forma en que trabajamos y estudiamos ha migrado hacia algunas formas digitales. En esa transición, aunque ofrece flexibilidad, ha traído problemas como las distracciones constantes (notificaciones, multitareas, redes sociales, etc.). En este contexto, la capacidad de mantener la concentración se ha vuelto un recurso valioso y escaso.

Particularmente en el ámbito académico y trabajo remoto, se observa que distintos usuarios van saltando entre diferentes aplicaciones, ya sea para anotar pendientes, otra para cronometrar el tiempo, o incluso tener un calendario de las tareas o proyectos personales, esta fragmentación termina generando una carga extra, donde el usuario pierde mucho tiempo, más de lo que pierde ejecutándolo.

Necesidad/Problemática

En la actualidad, muchas personas —especialmente estudiantes, trabajadores remotos y profesionales independientes— enfrentan dificultades para gestionar su tiempo de manera eficiente, lo cual repercute negativamente en su rendimiento y bienestar. Aunque existen herramientas aisladas para registrar tareas o medir tiempos, pocas integran de forma efectiva ambas funcionalidades dentro de una misma aplicación. Esta carencia motiva la pregunta central del presente trabajo: **¿cómo desarrollar un sistema que permita al usuario organizar sus tareas personales e implementar de forma flexible la técnica Pomodoro para mejorar su productividad?**

El problema reside en la falta de soluciones personalizables y accesibles que combinen gestión de tareas, como así también el control del tiempo de trabajo con pausas programadas, adaptadas a las necesidades individuales del usuario.

Estudiar esta problemática resulta necesario debido al creciente interés por métodos que promuevan la productividad personal sin comprometer la salud mental. La técnica Pomodoro ha demostrado ser eficaz para combatir la procrastinación y fomentar una rutina de

trabajo saludable. Sin embargo, su potencial no ha sido completamente aprovechado en aplicaciones que integren también la planificación de tareas diarias. Este proyecto ofrece la oportunidad de diseñar una solución tecnológica con valor práctico, que beneficie a quienes buscan un enfoque estructurado y amigable para administrar su tiempo.

No abordar esta problemática implica continuar con la fragmentación entre herramientas de gestión de tareas y de control del tiempo, lo que puede generar una experiencia de usuario poco eficiente y desmotivadora. En cambio, una solución integral podría representar un aporte significativo tanto en el ámbito académico como en el mercado de aplicaciones de productividad.

Propuesta de Desarrollo

La propuesta consiste en escalar el sistema **Cronos Notes**, una plataforma integral web ya desarrollada por el mismo equipo, diseñada para el control de tu tiempo y aumento de la productividad. El sistema está pensado especialmente para personas que necesitan un orden claro en sus tareas diarias para evitar la saturación y el agotamiento.

La aplicación se basa en cuatro pilares fundamentales:

Organización por Perfiles: Podés dividir tus responsabilidades en diferentes ámbitos (como "Estudio", "Trabajo", "Proyectos Personales") para visualizar solo lo necesario en cada momento.

Gestión Inteligente de Tareas: Permite crear, editar y clasificar las tareas pendientes según su nivel de prioridad (Alta, Media o Baja), también ofrece la posibilidad de establecer fechas límite para una mejor organización.

Enfoque con Técnica Pomodoro: Incluye un temporizador ajustable que divide el trabajo en intervalos de concentración profunda (por defecto 25 minutos) seguidos de descansos cortos y largos.

Motivación y Progreso: Cronos implementa un sistema de rachas que premia tu constancia diaria. Además, podés consultar estadísticas visuales detalladas sobre tu rendimiento, permitiéndote identificar cuántas tareas has completado y cuánto tiempo real has dedicado al estudio.

La propuesta es evolucionar dicho sistema para que pase de una aplicación web básica a una plataforma de alto rendimiento que no solo organiza tareas, sino que acompañe activamente el proceso cognitivo del usuario.

Alcances del Sistema

Las mejoras del sistema Cronos Notes se centrarán en escalar la plataforma para equilibrar las herramientas de gestión de proyectos con las de estudio, respondiendo a las demandas de los usuarios encuestados. El alcance de esta actualización incluye:

- **Integración con Herramientas Externas:** Sincronización del sistema con herramientas de uso cotidiano del usuario, específicamente orientadas a calendarios (ej. Google Calendar) y reproductores de música (ej. Spotify) para evitar el salto entre aplicaciones.
- **Restricciones de Perfiles:** Para este trabajo, los perfiles se limitarán a cinco por usuario.
- **Modo Zen/Minimalista:** El sistema debe poder intercambiar entre un escritorio visual y un espacio minimalista, donde solo se visualice el cronómetro y la tarea a completar.
- **Inicio Rápido:** Capacidad de iniciar un cronómetro de 25 minutos con un solo clic, sin pasar por la configuración de la sesión de estudio.
- **Calendario Visual:** El sistema debe permitir visualizar tareas y fechas límite en una interfaz de calendario mensual.
- **Modo Invitado:** El sistema debe tener una opción de invitado, que permitirá probar ciertas funcionalidades del sistema sin la necesidad de registrarse.
- **Inteligencia Artificial:** El sistema debe utilizar inteligencia artificial para ayudar al usuario a priorizar sus tareas en base a las fechas límite.
- **Módulo de Toma de Apuntes:** El sistema debe permitir crear notas asociadas a cada perfil, con la posibilidad de implementar métodos estructurados como Cornell o esquemas jerárquicos.
- **Mezclador de Sonido Ambiente:** interfaz para combinar sonidos como lluvia, ruido blanco y jazz de forma personalizada.

Límites del Sistema

Ecosistema de Niveles y Ranking Semanal

No se desarrollará un sistema competitivo de experiencia ni rankings sociales entre usuarios, ya que las encuestas demostraron que el público prefieren otras funcionalidades antes que la competencia social. De todos modos, se evalúa su implementación a futuro.

Sala de Estudio Virtuales

Aunque algunos usuarios sugirieron funciones de colaboración o listas grupales, la sincronización simultánea y el desarrollo de salas compartidas quedan fuera del alcance de esta versión, delegando así a actualizaciones futuras.

Personalización Estética

La posibilidad de personalización propia del usuario o la entrega de recompensas visuales (medallas, íconos) quedará relegada a una próxima actualización.

Pagos Internos

El sistema no contará con pasarelas de pago ni monetización de funciones o recompensas dentro de esta iteración.

Objetivos

Objetivo General

Desarrollar un sistema informático de gestión de tareas que implemente la técnica Pomodoro, con el fin de mejorar la organización personal y la productividad de los usuarios, mediante la combinación de planificación de actividades con el control del tiempo de trabajo.

Objetivos Específicos

- Realizar el análisis de requerimientos para la integración de APIs externas y servicios de IA.
- Rediseñar la interfaz (UX/UI) de Cronos Notes, con la finalidad de mejorar la experiencia del usuario.
- Implementar un motor de IA que organice el flujo de trabajo según fechas límite y prioridad.
- Validar la usabilidad y eficacia del sistema mediante pruebas con usuarios reales en contextos cotidianos.
- Implementar un módulo de gestión de perfiles que permita la creación, edición y asignación de roles específicos para garantizar el control de acceso a los perfiles de los usuarios.
- Implementar un módulo de toma de apuntes con soporte para el método Cornell, vinculado a la grabación de la clase.
- Añadir un calendario de eventos general relacionado a las tareas de cada perfil.
- Implementar un módulo de racha que pueda incentivar a la persona a mantener el uso diario y concentración de la aplicación web.
- Documentar el proceso de desarrollo, incluyendo diseño, flujo de trabajo y consideraciones futuras.

Planificación

Metodología de Desarrollo

Respecto a la metodología de desarrollo que empleara el equipo para el desarrollo del proyecto se decantó por llevar la metodología ágil conocida como “**Extreme Programming**” o “Programación extrema”. Esta es una metodología ágil centrada en mejorar la calidad del software y la capacidad de adaptación a los cambios.

Se basa en ciclos de desarrollo cortos, pruebas continuas, integración frecuente y una estrecha colaboración con el cliente. La razón por la que es más viable es gracias a la metodología de programación en pareja que permite al equipo formar parejas encargadas del desarrollo del front-end del proyecto y otra del back-end.

Del mismo modo, el equipo de desarrollo implementará un **tablero Kanban** con las siguientes columnas:

Tarjetas Principales. Esta columna contendrá tarjetas que, a su vez, contendrán las subtareas a completar para el desarrollo **FrontEnd y BackEnd**. También se indica el nombre del proyecto y las tecnologías a utilizar para desarrollar el mismo.

BackLog. En esta sección, se dividen las subtareas de las tarjetas principales, y se ponen a la espera para completar su desarrollo.

Diseño. La tarjeta llega a esta columna cuando se empiezan a planificar/configurar los detalles y la forma de llevar adelante el requerimiento. Incluye los diagramas, bocetos, etcétera.

Codificación. Si la tarjeta llega a esta columna, indica al equipo de trabajo que se comenzó con la codificación de diseño realizada previamente.

Pausado. Si se necesita de otro requerimiento para poder continuar con este, la tarjeta pasa a esta columna.

Prueba. Una vez terminada la codificación, la tarjeta del requerimiento llega a esta columna para indicar que se realizarán las pruebas necesarias para validar errores.

Documentación. Pasadas las fases de codificación y prueba, la persona encargada del requerimiento debe tomarse el tiempo de documentar lo realizado para poder realizar una explicación posterior.

Review. Luego de las pruebas, se pasará a mostrar el requerimiento implementado a las personas necesarias (equipo de trabajo, usuarios, profesores) para obtener la retroalimentación correspondiente.

Hecho. Una vez que el requerimiento completó el ciclo de vida completo, llega a esta columna.

A Mejorar. Columna opcional/secundaria. Si luego de la Review es necesaria alguna mejora del requerimiento, la tarjeta llega a esta columna.

Cronograma

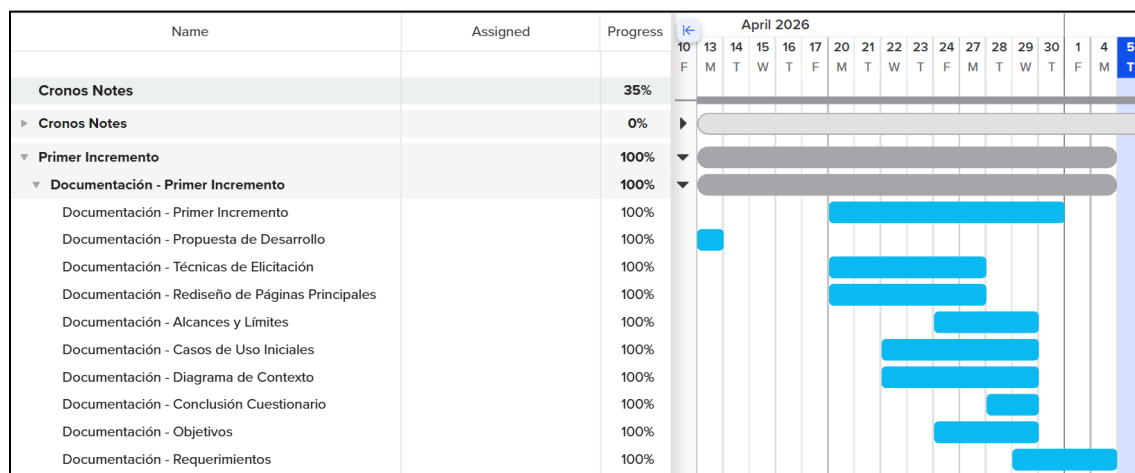
Duración Total del Proyecto

- **Fecha de Inicio:** Lunes, 20 de Abril de 2026.
- **Fecha de Cierre (Planificada):** Domingo, 14 de Junio de 2026.
- **Límite de Entrega Oficial:** Martes, 16 de Junio de 2026.
- **Duración Neta Planificada:** 56 días calendario (Exactamente 8 semanas / 4 incrementos). Al cerrar el 14 de junio, se cuenta con 2 días netos de holgura global para absorber demoras de entrega y empaquetado.

Incremento N°1: Semanas 1 y 2 (del 20 de abril al 3 de mayo)

- **Foco del Incremento:** Documentación, elicitación y diseño (UX/UI)
- **Tareas Realizadas:**
 - **Análisis del Cuestionario (20/04 - 25/04).** Análisis de los cuestionarios para priorizar requerimientos.

- **Propuesta, Alcance y Objetivos (20/04 - 25/04).** Definición de propuesta, alcances, límites y requerimientos del usuario.
- **Rediseño UX/UI de las Páginas (20/04 - 30/04).** Rediseño de las páginas principales (bocetos, paletas de colores, tipografías y prototipos)
- **Casos de Uso y Diagrama de Contexto (24/04 - 30/04).** Modelado de Casos de Uso Iniciales y Diagrama de Contexto.
- **Documentación de Requerimientos (27/04 - 03/05).** Registro de los requerimientos funcionales, no funcionales y de usuario
- **Inicialización del Proyecto (03/05).** Hacia el final del incremento, el equipo realiza la inicialización del proyecto.



Incremento 2: Semanas 3 y 4 (del 4 de mayo al 17 de mayo)

- **Foco del Incremento.** Modelado, autenticación y gestión de perfiles y tareas.
- **Tareas de Backend (Dellagnolo y Fornari):**
 - **Base de Datos y Diagramas (04/05 - 08/05):** Diseño de BD en MySQL y Diagrama de Clases.
 - **Autenticación de Usuario (08/05 - 12/05):** Configuración de Laravel Breeze, correos SMTP y login con Google.

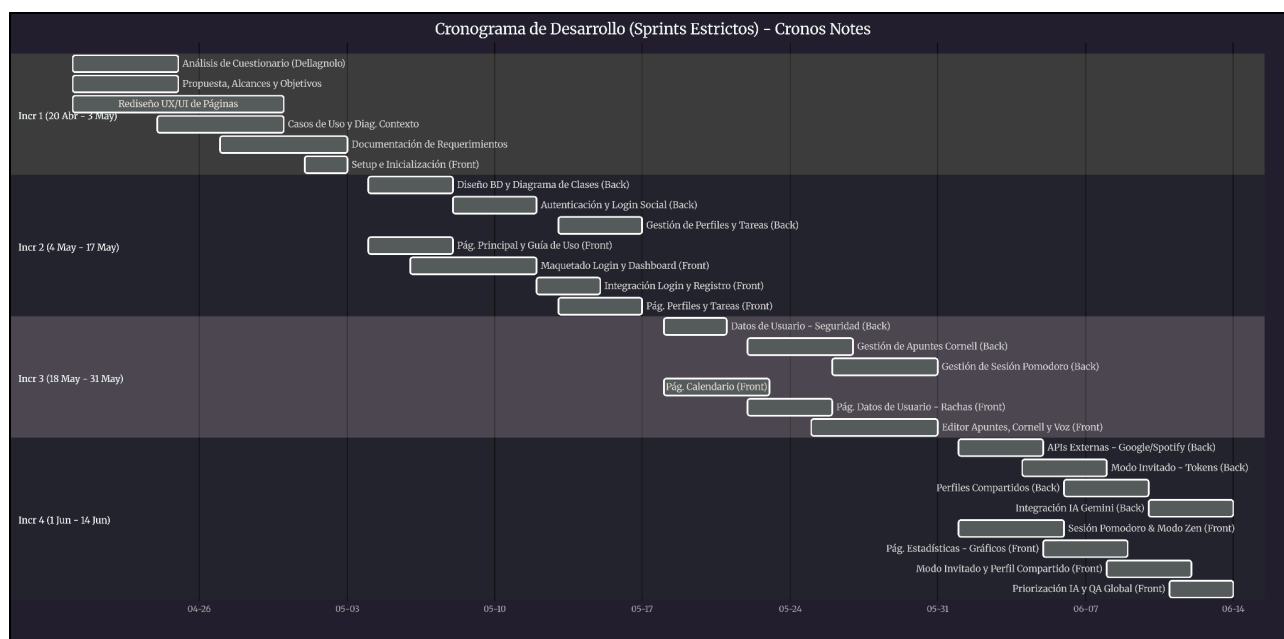
- **Gestión de Perfiles y Tareas (13/05 - 17/05):** Modelos y migraciones base para asociar tareas a perfiles.
- **Tareas de Frontend (Ayala y Romea):**
 - **Vistas Estáticas (04/05 - 12/05):** Desarrollo de la página principal, guía de uso y maquetado de Login/Dashboard.
 - **Integración y Reactividad (12/05 - 15/05):** Conexión del Login y Registro con el Backend
 - **Página de Perfiles (13/05 - 17/05):** Implementación inicial de la vista de perfiles.

Incremento 3: Semanas 5 y 6 (del 18 de mayo al 31 de mayo)

- **Foco del Incremento:** Módulos de productividad (Calendario, Datos y Apuntes)
- **Tareas de Backend (Dellagnolo y Fornari):**
 - **Datos de Usuario (18/05 - 21/05):** Lógica de validación para cambio de contraseña y cambios sensibles.
 - **Gestión de Apuntes (22/05 - 27/05):** Controladores para el Método Cornell, grabación y guardado de archivos de audio.
 - **Gestión de Sesión Pomodoro (26/05 - 31/05):** Persistencia de minutos enfocados.
- **Tareas de Frontend (Ayala y Romea):**
 - **Página de Tareas y Calendario (18/05 - 23/05):** Filtros de tareas y calendario mensual interactivo.
 - **Página de Datos de Usuario (22/05 - 26/05):** Conexión del panel de configuración con el widget de Rachas.
 - **Editor de Apuntes (25/05 - 31/05):** Panel Cornell y desarrollo de la grabadora de voz.

Incremento 4: Semanas 7 y 8 (del 1 de junio al 14 de junio)

- **Foco del Incremento:** IA, Modo Zen, Perfiles Compartidos, Estadísticas Visuales y Cierre.
- **Tareas de Backend (Dellagnolo y Fornari):**
 - **Modo Invitado (01/06 - 08/06):** Gestión del modo invitado implementando tokens temporales.
 - **Perfiles Compartidos (06/06 - 10/06):** Sistema transacciones de invitaciones y gestión entre cuentas.
 - **Integración IA (10/06 - 14/06):** Conexión con Gemini API para la priorización de tareas.
- **Tareas de Frontend (Ayala y Romea):**
 - **Sesión Pomodoro y Modo Zen (01/06 - 06/06):** Temporizador, mezclador de sonidos y panel inmersivo.
 - **Estadísticas y Gráficos (05/09 - 09/06):** Desarrollo de página de estadísticas y gráficos de barras semanales.
 - **Modo Invitado y Perfiles Compartidos (08/06 - 12/06):** Definición de pantalla de invitado y correos para invitaciones
 - **Refinamiento y UX Final (11/06 - 14/06).** Integración visual del priorizador de IA y reemplazo de modales nativos.



Recursos

Recursos Humanos

- **Gestor o Jefe del Proyecto.** Romea Acevedo, Clara Agustina
- **Desarrolladores Front-End.** Ayala, José Andrés y Romea Acevedo, Clara Agustina
- **Desarrolladores Back-End.** Dellagnolo, Ricardo Agustín y Fornari, Nahir Agustín
- **Análisis del Cuestionario.** Dellagnolo, Ricardo Agustín
- **Diseño de la Base de Datos.** Dellagnolo, Ricardo Agustín y Fornari, Nahir Agustín
- **Diseño de Diagrama de Clase.** Dellagnolo, Ricardo Agustín y Fornari, Nahir Agustín
- **Diagrama de Casos de Uso.** Fornari, Nahir Agustín; Ayala, José Andrés y Romea Acevedo, Clara Agustina
- **Diseño de Páginas Web.** Romea Acevedo, Agustina Clara y Ayala, José Andrés

Recursos Técnicos (Hardware y Software)

Desarrollo / Recursos del Equipo

Equipos/Dispositivos. Se necesitaran equipos suficientes (4) con una capacidad de cómputo media capaz de estar al nivel de las herramientas de desarrollo que empleará el equipo en el desarrollo.

Conexión a Internet. El equipo contará con conexión a la red de internet para investigación, comunicación y testing de desarrollo.

Servidor/Servicio de Hosting. Un servicio de Hosting que permitirá subir y mantener en línea el proyecto web en caso de ser necesario.

GitHub. Se utilizará GitHub como software de control de versiones, que facilitara el trabajo cooperativo y brindara seguridad ante pérdidas o errores.

Visual Studio Code. Es el entorno de trabajo que se utilizará para el trabajo de codificación.

Stack Tecnológico

- **PHP y Laravel**
 - **¿Qué es?:** Un framework de desarrollo web para PHP bajo el patrón de arquitectura MVC (Modelo-Vista-Controlador).
 - **Uso en el Proyecto:** Actúa como nuestro motor de Backend. Se encarga de la seguridad, el enrutamiento web, la lógica de negocio pesada, la interacción con la base de datos (MySQL), el envío de notificaciones y la integración con la API de Inteligencia Artificial (Gemini).
 - **Funcionamiento Clave:**
 - **Eloquent ORM (Object-Relational Mapping):** Es la herramienta que nos permite interactuar con la base de datos MySQL como si fueran objetos

de PHP, sin escribir SQL a mano. Sigue el patrón Active Record, lo que significa que cada modelo (*como app/Models/Tarea.php o app/Models/Perfil.php*) representa una tabla de la base de datos, y cada instancia de ese modelo representa un registro físico.

- **Laravel Socialite:** Librería oficial utilizada para el inicio de sesión con Google (OAuth 2.0). Simplifica la comunicación con los servidores de Google para autenticar al usuario y traer sus datos básicos de forma segura.

- **Vue 3**

- **¿Qué es?:** Un framework progresivo de JavaScript utilizado para construir interfaces de usuario interactivas y dinámicas.
- **Uso en el Proyecto:** Define la interfaz visual de Cronos Notes. Gracias a su reactividad, permite que pantallas como el temporizador Pomodoro se actualicen segundo a segundo en tiempo real sin recargar la página.
- **Funcionamiento Clave:**
 - **Composition API:** Permite estructurar el código de las pantallas organizando la lógica por funciones reactivas (en lugar de opciones fijas), facilitando la creación de *Composables* reutilizables (ej. para el manejo del estado del audio o del temporizador).
 - **Sistema de Reactividad:** Vue utiliza un sistema basado en Proxies de JavaScript. Cuando el estado del temporizador cambia, Vue detecta automáticamente la modificación y repinta en el DOM únicamente el texto de los minutos y segundos, optimizando el rendimiento.

- **Inertia.js**

- **¿Qué es?:** Un framework que funciona como puente. Permite conectar un backend monolítico (como Laravel) con un frontend de componentes SPA (como Vue) sin necesidad de crear una API REST compleja con enrutamiento duplicado.
- **Uso en el Proyecto:** Conecta nuestros controladores de Laravel directamente con las páginas Vue en *resources/js/Pages/*.
- **Funcionamiento Clave:** Intercepta todos los clics en enlaces dentro de la aplicación. En lugar de hacer una recarga completa del navegador, Inertia realiza una petición AJAX al servidor. El controlador de Laravel responde con un objeto JSON que contiene los datos (las props) y el nombre de la página Vue que debe cargarse. Inertia reemplaza dinámicamente la vista actual en el navegador, ofreciendo una experiencia SPA fluida.

- **Tailwind CSS**

- **¿Qué es?:** Un framework CSS utilitario orientado al diseño directo en las etiquetas HTML/Vue.
- **Uso en el Proyecto:** Estilizado visual de toda la plataforma (diseño adaptativo, colores, bordes, Modo Zen y Modo Oscuro).
- **Funcionamiento Clave:** En lugar de escribir hojas de estilos separadas (.css), aplicamos clases preexistentes en los componentes de Vue (ej. *class="flex items-center justify-between p-4 bg-gray-800 text-white"*). Al momento de compilar, el motor de Tailwind analiza los archivos y genera un archivo CSS compilado ultraligero que contiene estrictamente las clases que usamos.

Librerías Específicas y Utilidades

- **Howler.js**

- **¿Qué es?:** Una librería de JavaScript para el control de audio en navegadores web que simplifica el uso de la API nativa Web Audio API.
- **Uso en el Proyecto:** Motor de audio del Mezclador de Sonidos Ambientales.
- **Funcionamiento Clave:** Permite cargar múltiples pistas de audio (lluvia, teclado, café, bosque) de forma asíncrona, reproducirlas en loop de forma concurrente (simultánea) y controlar el volumen de cada canal de forma independiente mediante deslizadores (sliders).

- **Chart.js y Vue-Chartjs**

- **¿Qué es?:** Una librería de gráficos HTML5 basada en Canvas, junto con su envoltorio oficial para Vue.
- **Uso en el Proyecto:** Generación de gráficos interactivos en la pantalla de Estadísticas de Usuario (ej. horas de estudio diarias, pomodoros finalizados, tareas completadas por perfil).
- **Funcionamiento Clave:** Recibe los datos procesados desde *app/Http/Controllers/EstadisticaController.php* como *props*, y dibuja en un elemento **<canvas>** del navegador gráficos de barra y dona que responden dinámicamente al pasar el mouse por encima.

- **Ziggy**

- **¿Qué es?:** Una librería que expone las rutas del backend de Laravel al frontend de JavaScript.
- **Uso en el Proyecto:** Permite llamar a la función helper de Laravel *route()* dentro de nuestros archivos *.vue*
- **Funcionamiento Clave:** Ziggy genera un archivo de configuración dinámico en JS con la tabla de enrutamiento de Laravel. Esto nos permite escribir cosas como *route('tareas.store')* en el frontend, evitando hardcodear URLs manuales como */tareas/crear*. Si mañana cambiamos la URL en Laravel, el frontend se actualiza automáticamente.

- **Lucide Icons**

- **¿Qué es?:** Un set de iconos vectoriales SVG de código abierto optimizado para Vue.
- **Uso en el Proyecto:** Provee toda la iconografía visual (los iconos del reproductor de audio, los perfiles de estudio, configuración de pomodoros, etc.).
- **Funcionamiento Clave:** Se importan de forma modular únicamente los iconos necesarios (ej. `import { Play, Pause } from 'lucide-vue-next'`), reduciendo el tamaño final de la aplicación compilada.

Herramientas de Construcción y Base de Datos

- **Vite**

- **¿Qué es?:** Un empaquetador de frontend de última generación enfocado en la velocidad de desarrollo.
- **Uso en el Proyecto:** Compila y procesa nuestro JavaScript, Vue, Tailwind CSS y recursos estáticos.

- **MySQL**

- **¿Qué es?:** Un sistema de gestión de bases de datos relacionales (RDBMS).
- **Uso en el Proyecto:** Motor de persistencia del sistema. Almacena usuarios, perfiles, tareas, sesiones pomodoro y configuraciones de personalización.

Recursos del Cliente

Hardware Requerido

- **Dispositivo de acceso:** Computadora de escritorio o portátil (laptop).
- **Memoria RAM:** Se recomienda un mínimo de 4 GB de RAM. Al ser una aplicación web que mantiene temporizadores activos, reproduce múltiples canales de audio simultáneos (mezclador de ambiente) e interactúa con APIs externas, una memoria adecuada evitará que el navegador suspenda la pestaña y se pierda el progreso de la sesión Pomodoro.
- **Salida de Audio:** Auriculares o altavoces funcionales. Son indispensables para percibir las alertas de inicio/fin de los temporizadores y para utilizar el mezclador de sonido inmersivo.
- **Micrófono (Opcional pero recomendado):** Necesario para utilizar la funcionalidad de grabación de voz dentro del módulo de "Toma de Apuntes" detallado en los casos de uso.

Software Requerido

- **Navegador Web Moderno:** Versiones actualizadas de Google Chrome, Mozilla Firefox, Microsoft Edge, Opera o Safari. Es crucial contar con un navegador moderno para asegurar la correcta ejecución del entorno desarrollado en Vue.js y la correcta renderización de Tailwind CSS, así como el manejo óptimo de los flujos de audio.

- **Sistema Operativo:** Independiente (Windows, macOS, Linux), siempre y cuando tenga soporte para los navegadores web mencionados anteriormente.
- **Conexión a Internet:** Conexión estable de banda ancha. Es de carácter obligatorio, ya que la plataforma está alojada en la web y depende de conexión continua para guardar el progreso, consumir el motor de Inteligencia Artificial y sincronizar con servicios de terceros.
- **Cuentas de Terceros (Opcional):** Para aprovechar al máximo las funcionalidades de integración (Requerimiento RF-M8.1), el usuario se verá beneficiado de poseer una cuenta activa de Google (para Google Calendar) y una cuenta de Spotify.

Requerimientos

Es importante aclarar que, al ser un proyecto que cuenta con ciertas funcionalidades, para el desarrollo de las mejoras de este trabajo solamente se tendrán en cuenta los

Requerimientos de Usuario (RU) y los Requerimientos Funcionales de Mejora (RF-M). Sin embargo, consideramos que es importante indicar los requerimientos funcionales que ya fueron implementados en la versión anterior:

ID	Nombre	Descripción
RF 1	Autenticación de Usuario	Para acceder a las funcionalidades del sistema, el usuario deberá crear una cuenta aportando ciertos datos.
RF 1.1	Datos de usuario	Almacenamiento de la información básica como nombre, apellido, correo electrónico y contraseña segura.
RF 1.2	Validaciones de datos	El sistema debe verificar que los datos sean correctos (formato) y registros mínimos de seguridad de contraseña.
RF 1.3	Sesiones de usuario	El sistema debe permitir que el usuario se mantenga autenticado hasta que cierre la sesión manualmente o se agote la sesión por inactividad.
RF 2	Gestión de Perfiles	El sistema debe ser capaz de crear espacios de trabajo (estudio, trabajo, etc) para organizar información.
RF 2.1	Personalización de perfiles	Ajuste de nombres y configuraciones visuales básicas para cada perfil definido por el usuario
RF 3	Gestión de Tareas	El sistema permitirá crear, editar o eliminar tareas. El usuario también podrá marcarlas como completadas.
RF 3.1	Datos de Tareas	Registro de título, descripción, prioridad y vinculación al perfil específico
RF 3.2	Visualización de Tareas	Listado de las tareas, que permite filtrar y ver las tareas completadas o activas.
RF 3.3	Etiquetas Visuales	Uso de colores para identificar el nivel de

		prioridad de cada tarea pendiente.
RF 4	Sesión Pomodoro	El sistema permitirá trabajar en sesiones pomodoro, las cuales estarán asociadas a una tarea pendiente de realización. En esta sesión el usuario trabajará en su tarea durante el tiempo de concentración y descansará en intervalos de tiempo.
RF 4.1	Temporizador Pomodoro	El usuario podrá utilizar el temporizador Pomodoro integrado para medir intervalos de trabajo (por defecto de 25 minutos) y descansos (por defecto de 5 minutos). Este temporizador podrá pararse o detenerse. En caso de detenerse antes de finalizar, el usuario podrá ajustar manualmente su duración antes de registrarlo.
RF 4.2	Alertas de sesión	Finalizada una sesión, el sistema emitirá una alerta e invita a iniciar el descanso correspondiente.
RF 4.3	Configuración de Sesión	El usuario podrá crear configuraciones específicas a su gusto. Cada configuración tendrá definida los valores de tiempo que empleará en los intervalos de concentración y descanso.
RF5	Sistema de Rachas	El sistema registra rachas diarias de uso basado en actividad (ya sea tareas completadas o pomodoros realizados) para motivar al usuario a mantener la constancia.
RF6	Sistema de Estadísticas	El sistema mostrará una sección de "Estadísticas" donde el usuario podrá visualizar su rendimiento mediante gráficos.
RF 6.1	Estadísticas de Usuario	El sistema de estadísticas mostrará datos como la cantidad de tareas finalizadas, tareas finalizadas con retraso, tiempo total de concentración, racha más larga y racha actual. Estas estadísticas ayudarán al usuario a evaluar su productividad y detectar patrones de uso.

Requerimientos Funcionales de Mejora

Para cumplir con las mejoras solicitadas por los usuarios, el sistema debe cumplir con los siguientes requerimientos funcionales técnicos:

ID	Nombre	Descripción	Prioridad
RF-M01	Alternancia a Modo Zen	El sistema debe permitir al usuario ocultar los paneles laterales y widgets secundarios mediante un botón, dejando visible únicamente el contador de tiempo y el título de la tarea en ejecución.	Alta
RF-M02	Ejecución de Inicio Rápido	El sistema debe proveer un atajo (botón) en la interfaz principal que inicie inmediatamente un temporizador Pomodoro de 25 minutos sin requerir la selección de perfil o tarea previa.	Alta
RF-M03	Renderizado de Calendario Visual	El sistema debe generar una vista de calendario mensual y mapear las tareas pendientes del usuario según la fecha límite registrada en la base de datos.	Alta
RF-M04	Procesamiento de IA para Prioridad	El sistema debe enviar los metadatos de las tareas creadas (nombre y fecha límite) a una API de Inteligencia Artificial para recibir un orden específico de desarrollo de las tareas a completar, cooperando con la organización del usuario.	Alta
RF-M05	Editor de Apuntes en Tiempo Real	El sistema debe proporcionar un editor de texto enriquecido que permite al usuario crear, guardar y estructurar apuntes, vinculándolos al perfil de estudio activo.	Alta
RF-M06	Mezclador de Sonidos	El sistema debe reproducir múltiples pistas de audio ambiental en simultáneo, proporcionando controles de volumen independientes para cada canal (lluvia, teclado, océano, etc.)	Media
RF-M07	Sesión de Invitado	El sistema debe asignar un token temporal a los usuarios no	Media

		autenticados que les permita acceder al temporizador básico y al mezclador de sonido sin persistencia de datos.	
RF-M08	Conexión con APIs Externas	El sistema debe implementar protocolos de autorización (ej. OAuth) para leer eventos de calendarios externos y controlar la reproducción de cuentas vinculadas.	Alta
RF-M8.1	Herramientas a Integrar	El sistema deberá integrar Google Calendar, Spotify y la posibilidad de registrarse utilizando una cuenta de Google.	Alta
RF-M09	Registro de Cancelación de Sesión	Si el usuario detiene el temporizador antes de que el contador llegue a cero, el sistema debe registrar el evento en la tabla de estadísticas como "cancelado" y disparar un mensaje motivacional en la interfaz.	Media
RF-M10	Estimación de Esfuerzo	El sistema debe incluir un campo numérico en el formulario de creación/edición de tareas para ingresar la cantidad estimada de Pomodoros requeridos.	Media
RF-M11	Perfiles de Usuario Compartidos	El sistema debe permitir que un usuario pueda compartir los perfiles que él cree con otros, para compartir el contenido que este contiene	Alta
RF-M11.1	Permisos de Perfil	El usuario creador del perfil puede asignar permisos a los usuarios unidos al mismo: poder crear, modificar, consultar y borrar apuntes y tareas	Alta
RF-M12	Cambios en el Sistema de Rachas	El sistema debe permitir que la racha aumente solamente cuando se complete una sesión de 25 minutos.	Baja
RF-M13	Cambios en las Estadísticas	El sistema deberá implementar nuevas estadísticas: horas de concentración diaria, cantidad de sesiones por perfil	Media

Requerimientos No Funcionales

ID	Nombre	Descripción	Prioridad
RNF 1	Usabilidad e Intuición de Uso	El sistema debe ser fácilmente comprensible desde el primer uso. Su diseño debe enfocarse en la simplicidad, guiando al usuario mediante visuales claros y evitando la sobrecarga informativa. La estructura debe ser coherente entre vistas, asegurando una curva de aprendizaje baja y fomentando su adopción diaria	Alta
RNF 1.1	Interfaz clara	La interfaz de usuario debe ser clara, minimalista y con estructura coherente	Alta
RNF 1.2	Iconos intuitivos	El sistema debe utilizar iconos intuitivos, colores coherentes y mensajes de retroalimentación visual que guíen al usuario en su experiencia.	Alta
RNF 2	Seguridad y Validación de Datos	El sistema debe proteger los datos de los usuarios. Toda información será validada antes de almacenarse o procesarse, evitando ataques comunes de inyección SQL o XSS.	Alta
RNF 2.1	Cifrado de información especial	El sistema almacena las informaciones sensibles con un hash o cifrado que ofrecen una seguridad ante filtraciones.	Alta
RNF 2.2	Enmascaramiento	El sistema ocultará las contraseñas escritas en formularios que la requieran. A excepción de cuando el usuario quiera que se muestren.	Alta
RNF 3	Portabilidad y Entorno de Ejecución Web	El sistema funciona en la web, deberá ser accesible desde cualquier navegador moderno sin necesidad de instalaciones adicionales.	Alta
RNF 4	Escalabilidad del Sistema	El Sistema contará con una organización modular tanto en código como a nivel de base de datos, permitiendo añadir nuevas características y siendo capaz de crecer sin requerir una reestructuración completa.	Media
RNF 5	Personalización de Interfaz	El usuario debe ser capaz de personalizar la interfaz del sistema a su gusto. El sistema le ofrecerá elegir el modo de color	Baja

		que desee (claro u oscuro), y opciones de ambiente donde el usuario elegira el gif que y los sonidos que desea reproducir en la sesion pomodoro	
--	--	---	--

Requerimientos de Usuario

Son todas las mejoras/funcionalidades que fueron mejor puntuadas por los potenciales usuarios:

ID	Nombre	Descripción	Prioridad
RU01	Modo Zen	El usuario necesita una interfaz libre de distracciones visuales para mejorar su concentración durante los intervalos de trabajo.	Alta
RU02	Pomodoro Rápido	El usuario requiere poder iniciar una sesión de estudio de manera inmediata, sin fricciones de configuración, para aprovechar momentos de motivación espontánea.	Alta
RU03	Calendario Visual	El usuario necesita visualizar sus fechas límite (exámenes, entregas) en un formato mensual para organizar sus prioridades semanales.	Alta
RU04	Toma de Apuntes	El usuario necesita poder registrar notas y aplicar métodos de estudio directamente en la plataforma para evitar perder el enfoque cambiando de aplicación.	Alta
RU05	Modo Invitado	El usuario potencial requiere probar el valor y la estética de la aplicación web antes de verse obligado a proporcionar sus datos personales para un registro.	Media
RU06	Estimación de Esfuerzo	El usuario necesita poder asignar un "peso" o cantidad de sesiones a sus tareas para planificar cuántos Pomodoros le exigirá su jornada.	Media
RU07	Mezclador de Sonido	El usuario necesita personalizar los estímulos auditivos (activando o desactivando ruidos blancos y música) para generar un entorno inmersivo que mitigue el estrés.	Media

RU08	Integración con Herramientas Externas	El usuario necesita que Cronos Notes se comunique con las herramientas que ya utiliza en su vida cotidiana para mantener un flujo de trabajo unificado.	Alta
-------------	---------------------------------------	---	------

Diagrama de Contexto

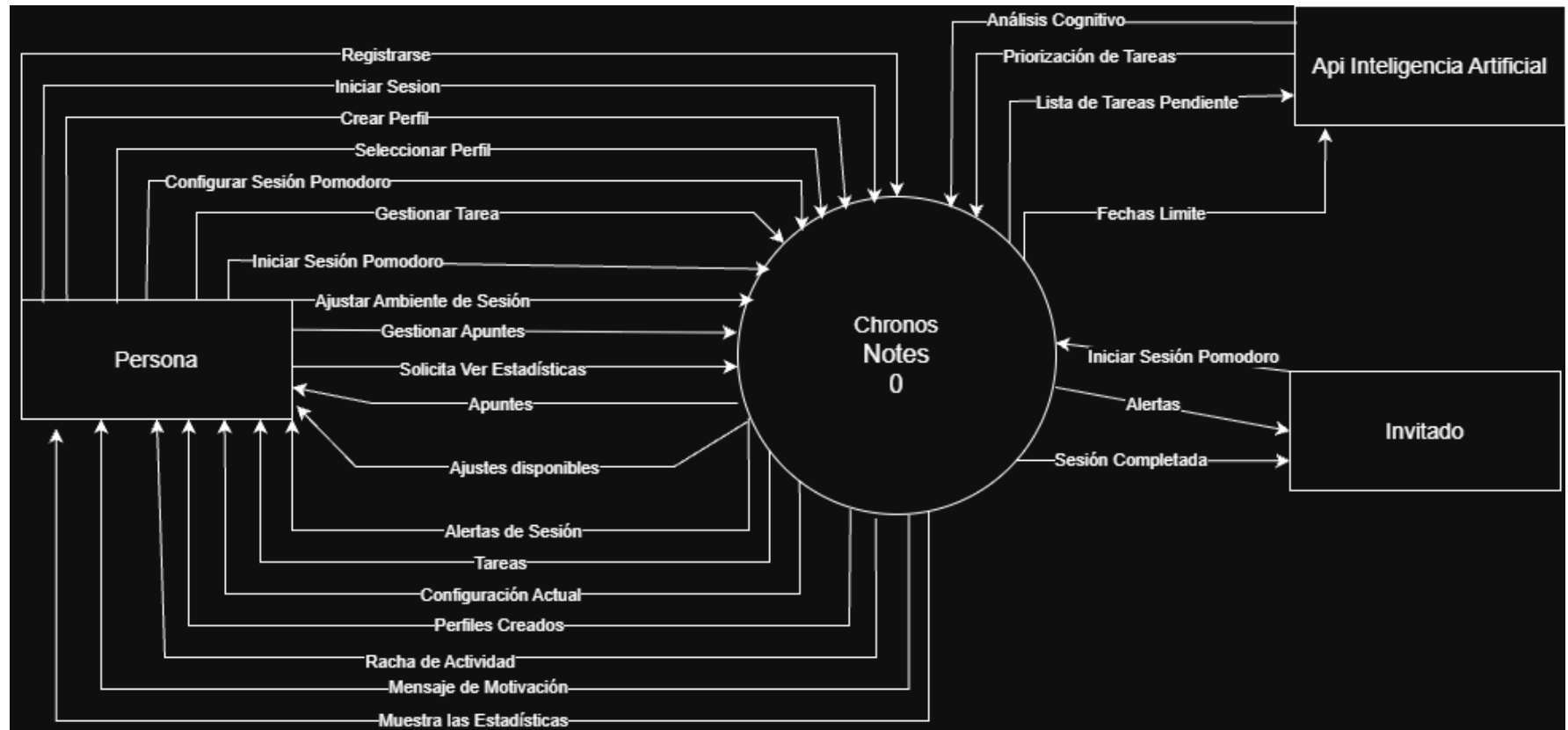
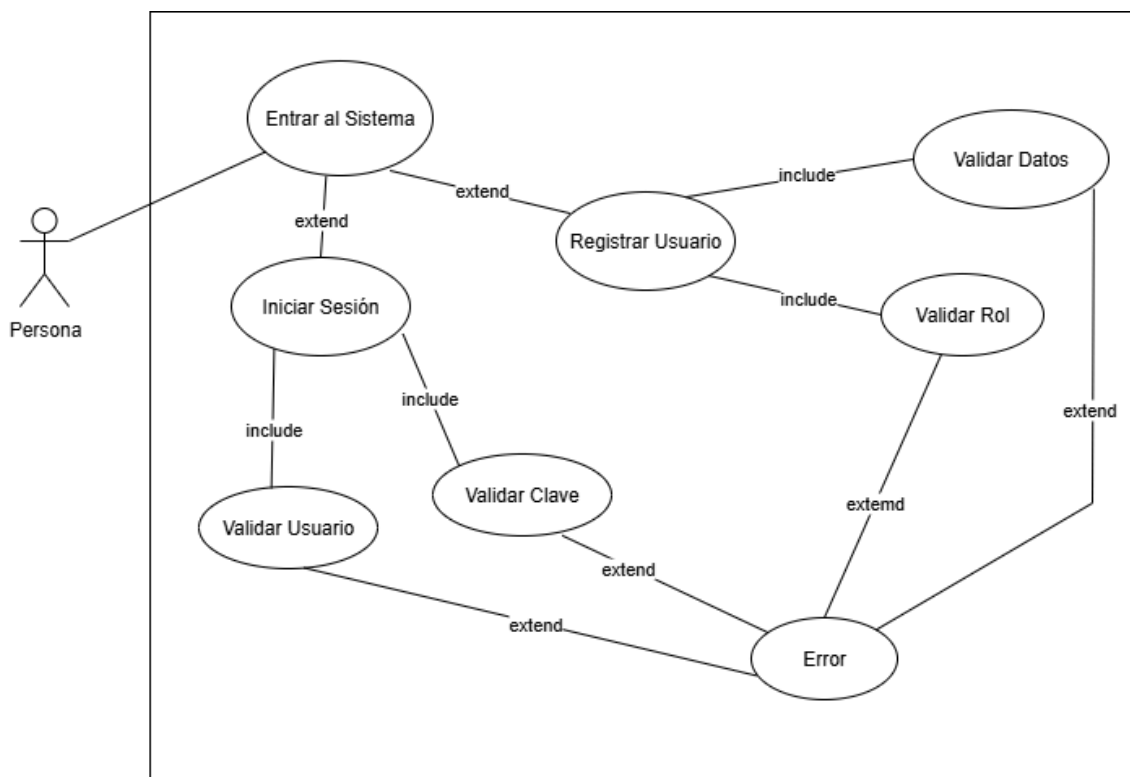
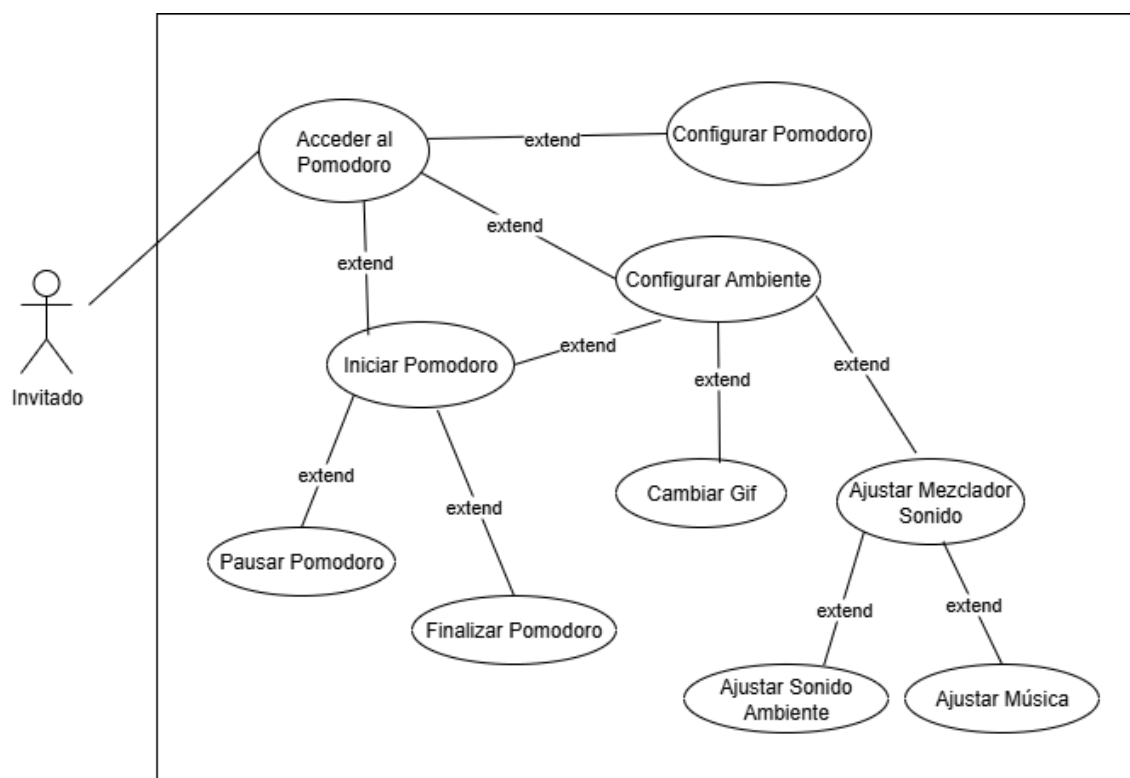


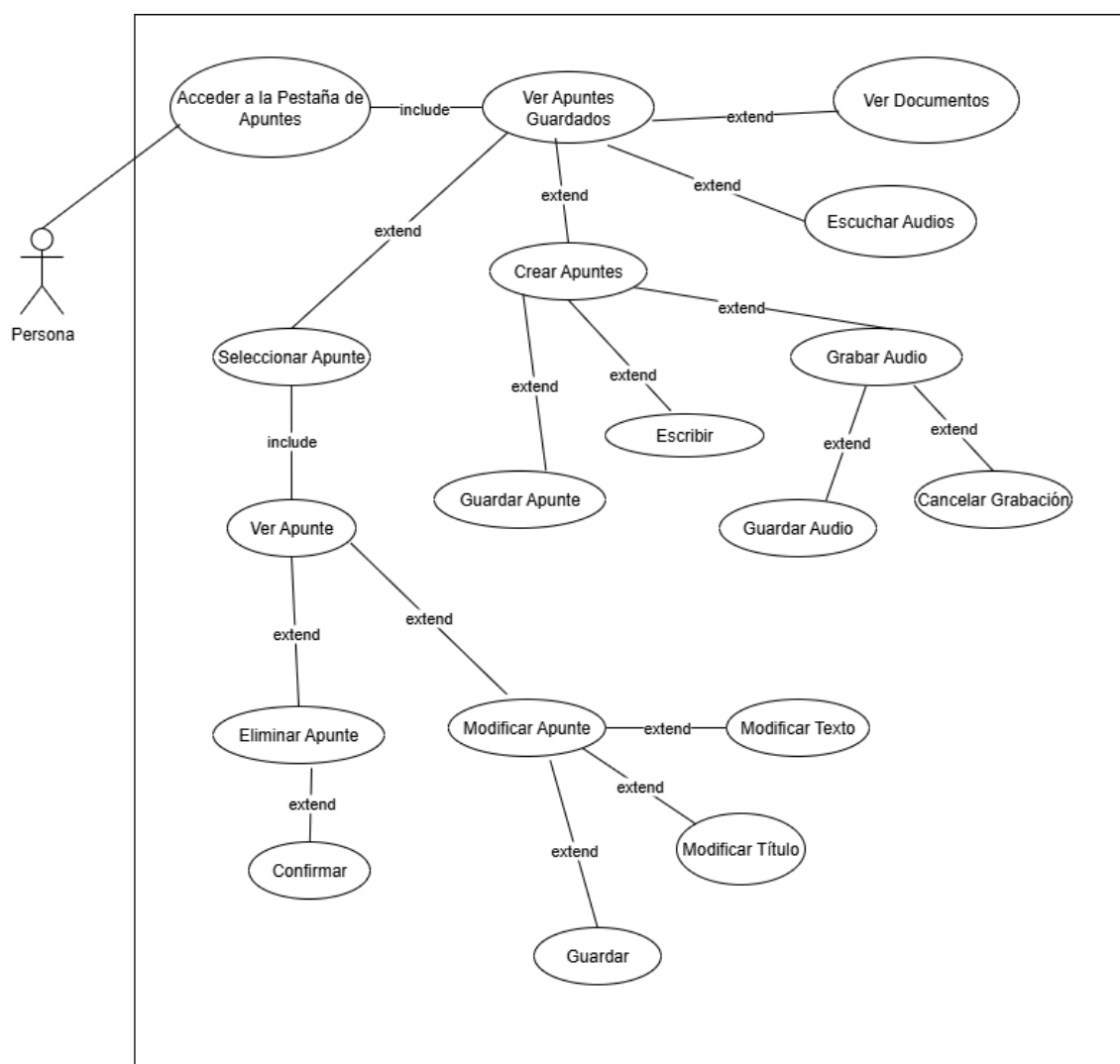
Diagrama de Casos de Uso



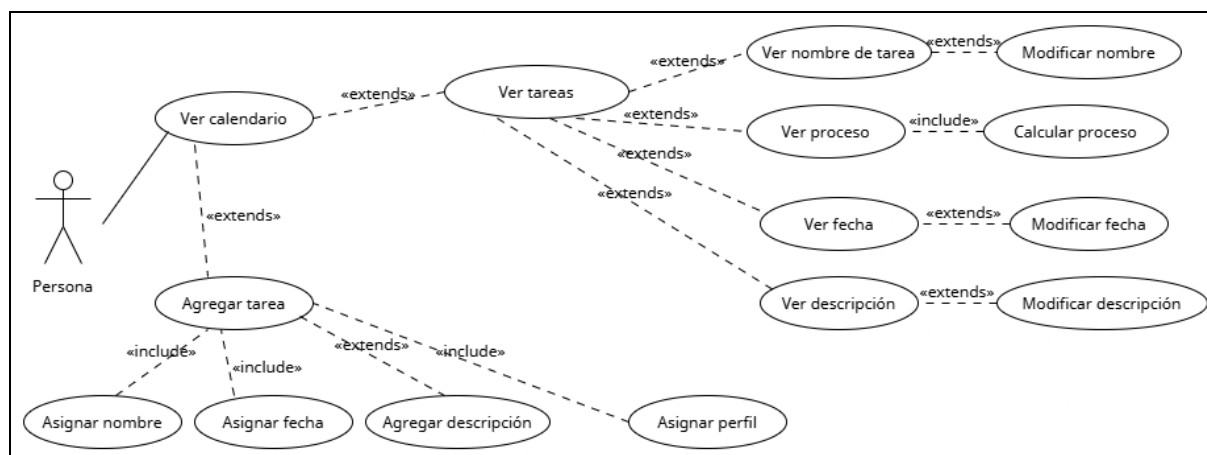
Caso de Uso	01
Nombre	Login de Usuario
Descripción	El usuario entra al sistema, puede registrarse o iniciar sesión en caso de tener una cuenta ya asociada, para el primero, obligatoriamente tendrá que rellenar un formulario con los datos personales, incluyendo su rol dentro del sistema. El inicio de sesión, incluye la validación del usuario y de su clave.
Actores	Persona



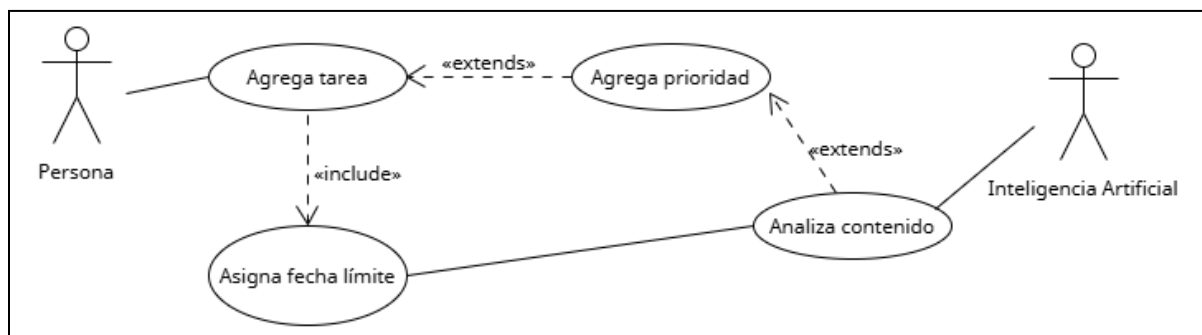
Caso de Uso	02
Nombre	Modo Invitado
Descripción	El usuario invitado (rol), va a acceder al Pomodoro, en donde, puede iniciar el pomodoro que dispone de acciones como pausar pomodoro y finalizar el mismo. Por otro lado, al acceder al pomodoro, podrá configurar el ambiente, que posee opciones, cambiar el gif, ajustar el mezclador (que a su vez extiende funcionalidades como ajustar el sonido o la música) y también de ese caso – configurar ambiente – extiende la función de iniciar el Pomodoro El usuario invitado, una vez accedido al pomodoro, puede configurar el pomodoro.
Actores	Invitado



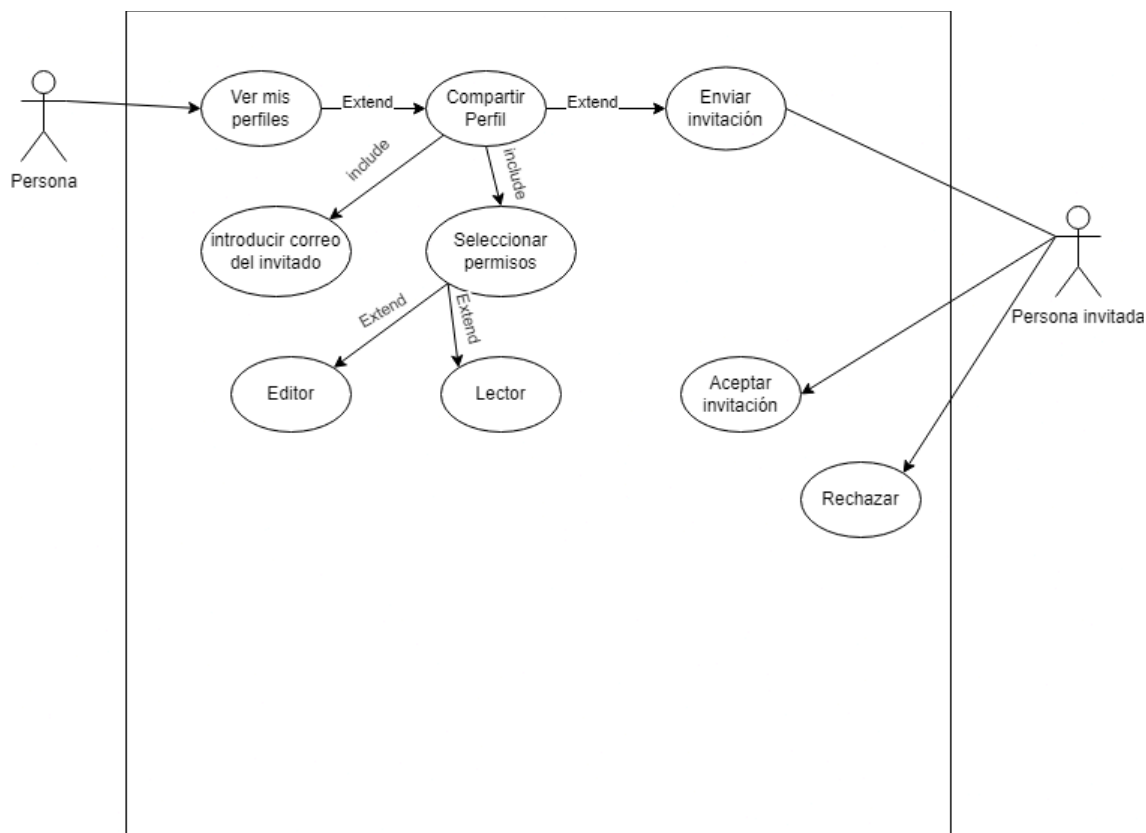
Caso de Uso	03
Nombre	Toma de Apuntes
Descripción	El usuario del sistema (ya registrado), puede acceder a la pestaña de apuntes, el cual incluye ver los apuntes, el usuario puede ver sus documentos, escuchar los audios grabados en sus clases. El usuario también va a poder crear sus apuntes, dicha funcionalidad dispone de la grabación de audios (guardar o cancelar), escritura y guardado de los apuntes. El usuario tiene la capacidad de seleccionar el apunte, eliminar o modificar el mismo, en el cambio puede alterar el texto, el título y guardarlo.
Actores	Persona



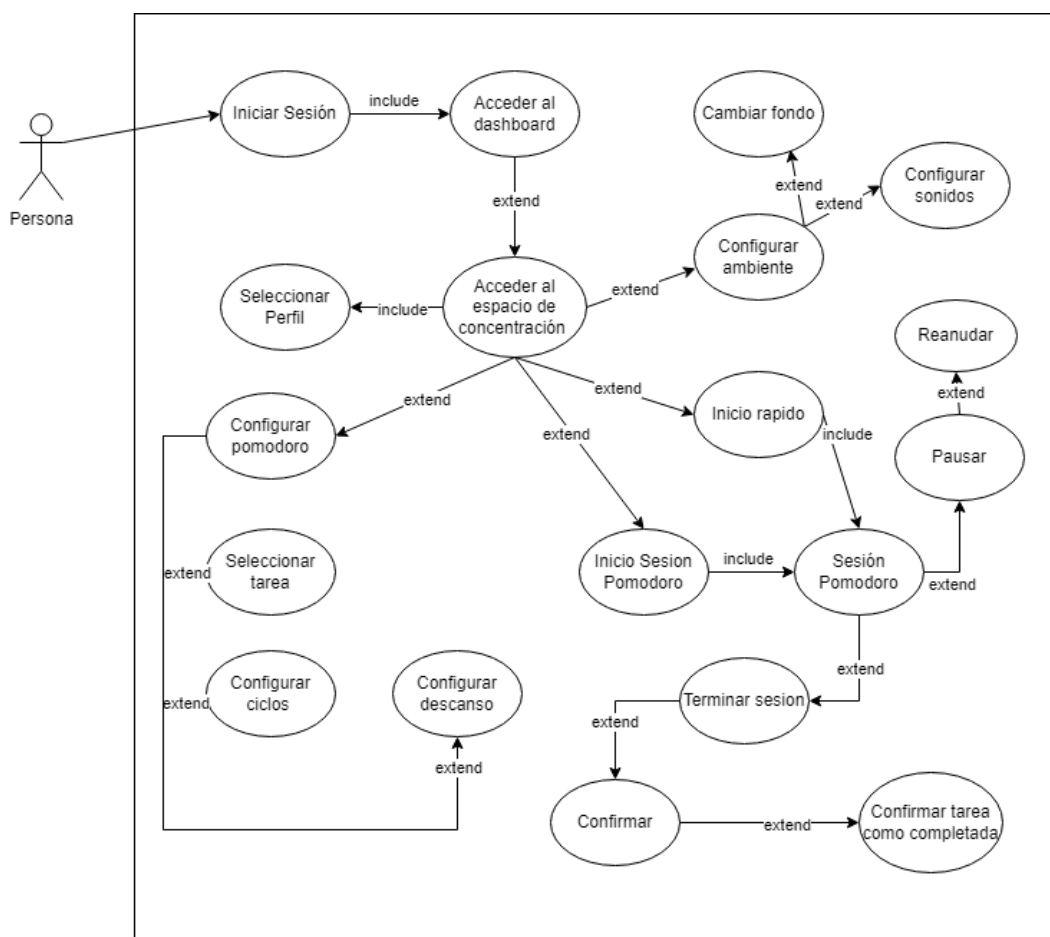
Caso de Uso	04
Nombre	Asignación y vista de tareas
Descripción	El usuario del sistema (ya registrado), puede acceder al calendario, en donde se ordenan las tareas por fecha límite. Puede interactuar con estas tareas viendo el nombre, fecha límite y descripción de cada una, con la posibilidad de modificar estos campos. Además, puede ver el proceso de finalización de la tarea, que incluye el cálculo de progreso. El usuario también puede agregar nuevas tareas. Para esto es obligatorio asignar un nombre, fecha límite, perfil de la tarea, y opcionalmente una descripción.
Actores	Persona



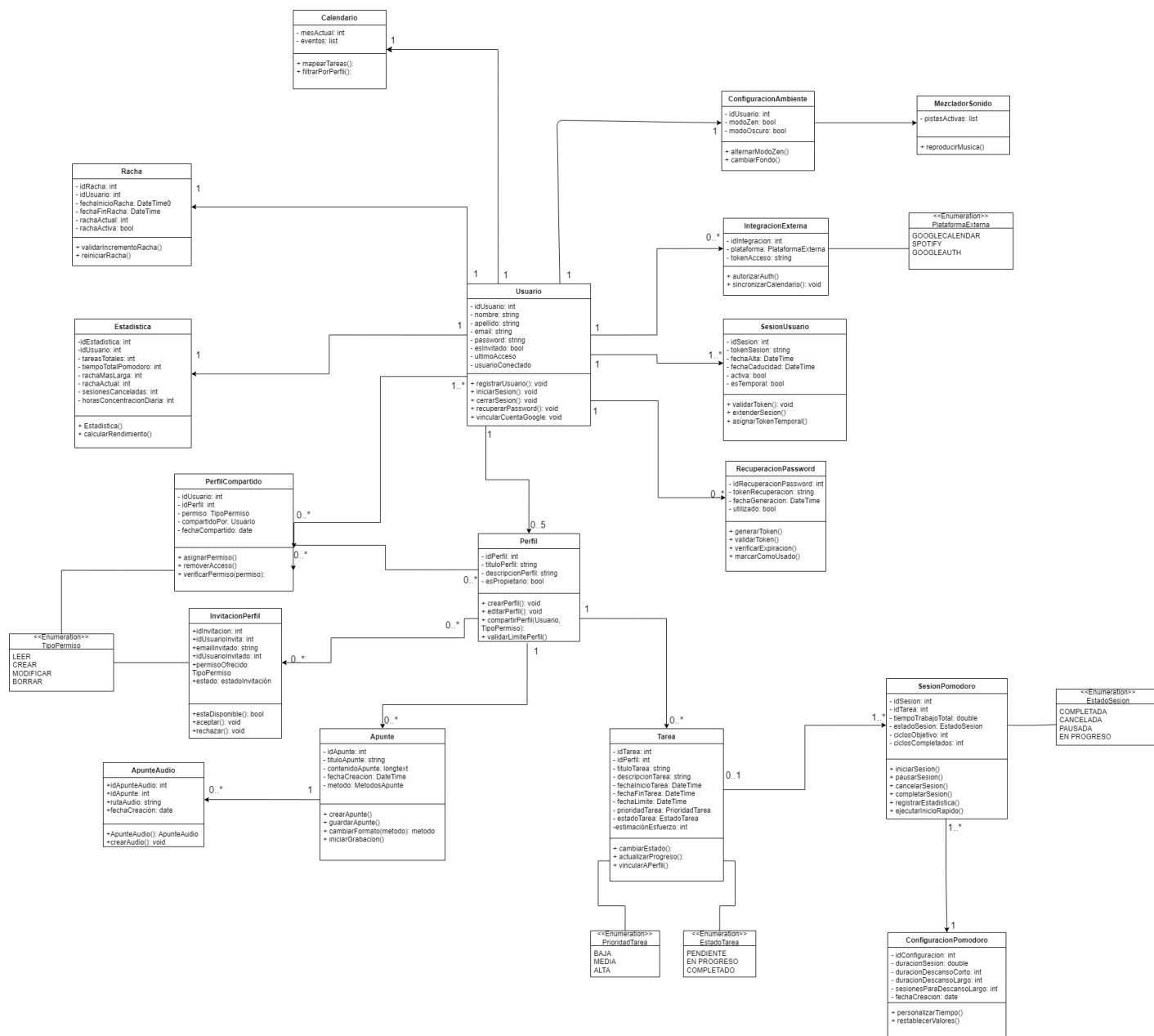
Caso de Uso	05
Nombre	IA en la prioridad de tareas
Descripción	El usuario del sistema (ya registrado), agrega nuevas tareas con fecha límite (ver caso de uso 4). La inteligencia artificial analiza el contenido de la tarea, especialmente la fecha límite, y asigna una prioridad entre tres: alta, media o baja, de manera que pueda guiar al usuario a saber qué tarea realizar primero.
Actores	Persona, Inteligencia Artificial



Caso de Uso	06
Nombre	Perfiles compartidos
Descripción	El usuario del sistema (ya registrado), visualiza sus perfiles donde puede compartir un perfil a otro usuario del sistema. Para ello deberá introducir el correo del invitado y elegir los permisos que desea darle, luego de eso puede enviar la invitación y el invitado podrá aceptarla o rechazarla
Actores	Persona, Persona invitada



Caso de Uso	07
Nombre	Sesión Pomodoro
Descripción	La persona logueada en el sistema, visualizará el dashboard principal del sistema, donde desde el sidebar podrá acceder al espacio de concentración de cronos notes, para lo cual deberá seleccionar un perfil. En el espacio de concentración de cronos podrá configurar el pomodoro seleccionado una tarea, indicando los ciclos de tiempo que desea realizar y el tiempo de los descansos. También podrá configurar el ambiente donde podrá elegir un fondo preferido y podrá configurar el audio que desee. Por último podrá iniciar un pomodoro rápido donde tendrá una configuración por defecto, o podrá iniciar una sesión pomodoro normal donde utilizará la configuración elegida. Ya en el pomodoro podría poner en pausa el cronómetro, y luego reanudarlo, y podrá finalizar la sesión teniendo que confirmar y luego confirmar si la tarea fue completada.
Actores	Persona



Interfaces de Usuario

Metodología a Utilizar

Para el desarrollo de Cronos Notes hemos elegido la metodología **Agile UX**. Esta decisión se fundamenta en la necesidad de integrar de manera fluida el diseño de la experiencia de usuario y el desarrollo técnico con PHP y SQL.

Al ya contar con prototipos diseñados en **Visily**, Agile UX nos permite trabajar en ciclos iterativos (sprints), donde podemos validar funcionalidades de módulos de gestión de tareas y pomodoro mientras se mejora la interfaz. El enfoque ágil nos garantiza que el sistema sea flexible y pueda adaptarse a los cambios de requerimientos que surjan durante el desarrollo del sistema, asegurando que el producto final sea cómodo para el usuario y fácil de usar (**UX**) y que sea visualmente atractiva y clara (**UI**).

Diseño de las Pantallas

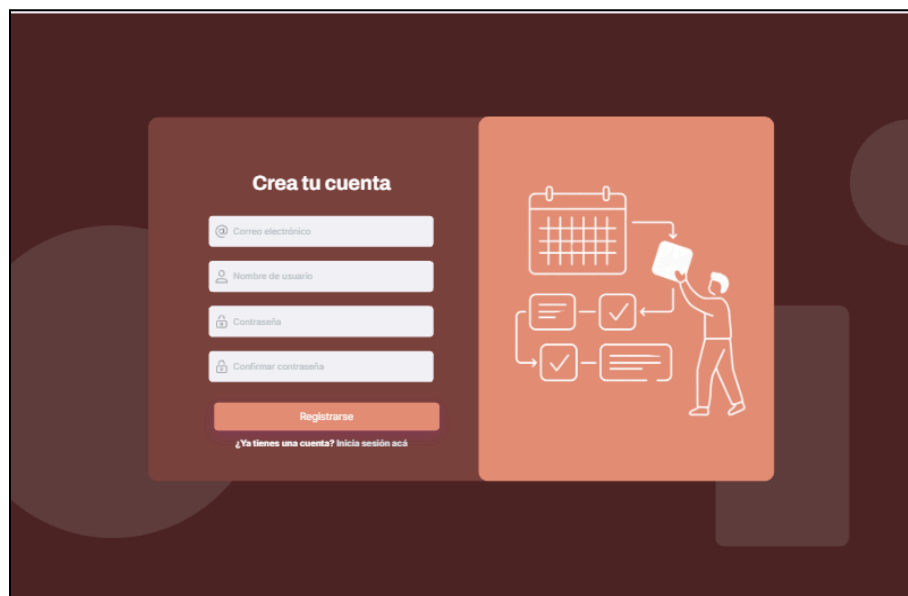
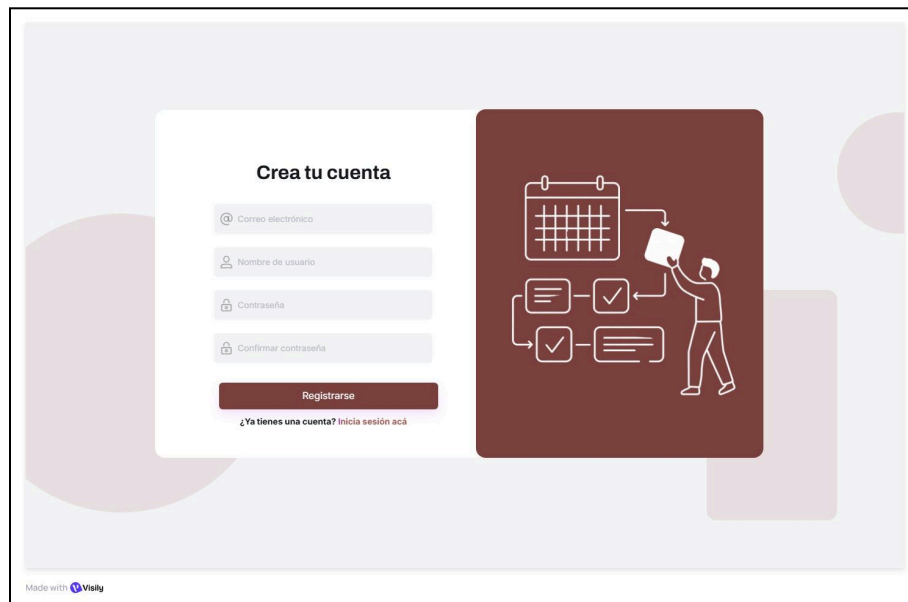
Logo de la Aplicación



Página de Bienvenida

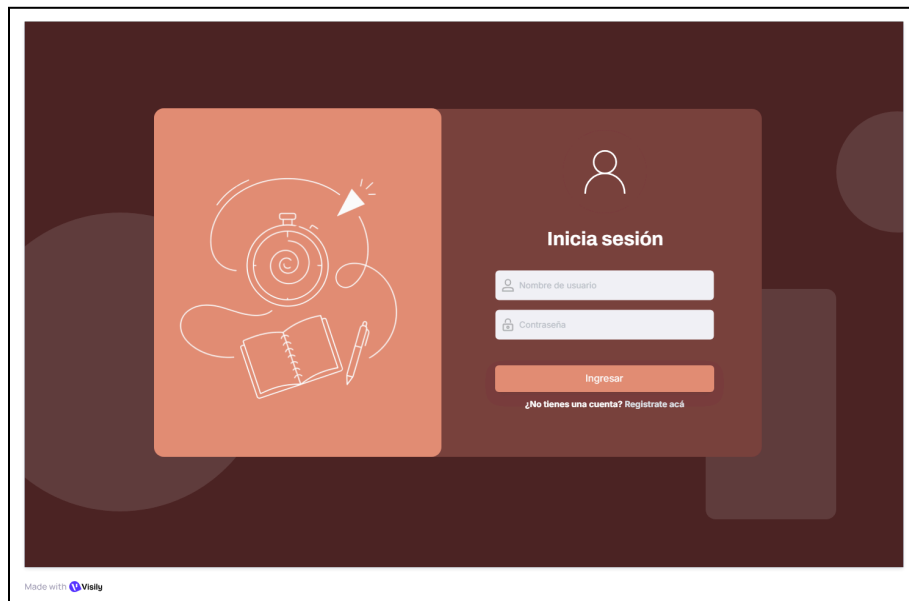
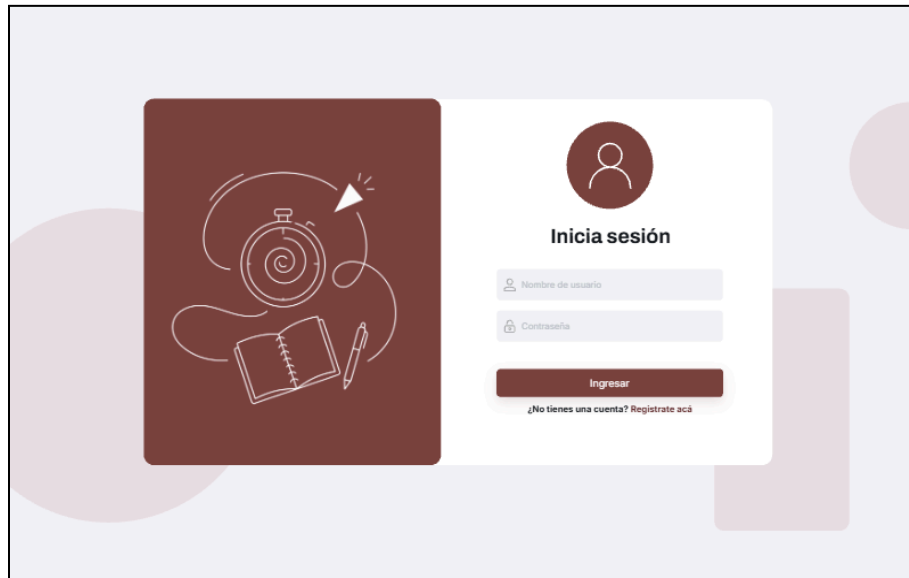
Registro - Modo Claro y Oscuro

- **Descripción:** Es la barrera de entrada para nuevos usuarios. Consiste en un formulario que solicita los datos esenciales (Correo, Nombre de Usuario, Contraseña y Confirmación de Contraseña)



Inicio de Sesión - Modo Claro y Oscuro

- **Descripción:** Pantalla de autenticación. Solicita únicamente el correo y la contraseña para validar las credenciales del usuario contra la base de datos para darle acceso a su entorno.



Página Principal/Dashboard - Modo Claro y Oscuro

- Descripción:** Es la vista principal del usuario. Tiene una barra de navegación lateral para acceder a perfiles, estadísticas, calendario, tareas, espacio de concentración y apuntes. Muestra un resumen rápido de sus perfiles creados, al igual que gráficos estadísticos en conjunto con la racha. El propósito es darle al usuario una visión inmediata de su progreso y organización.



Página de Perfiles

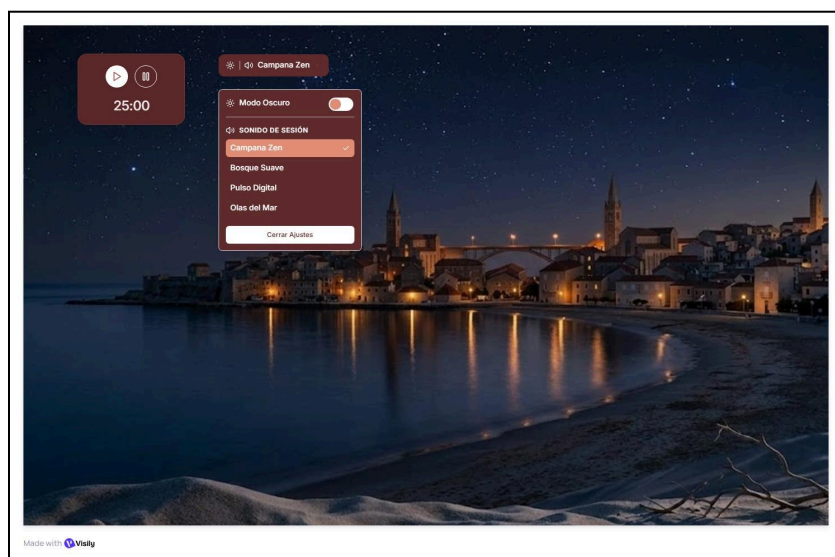
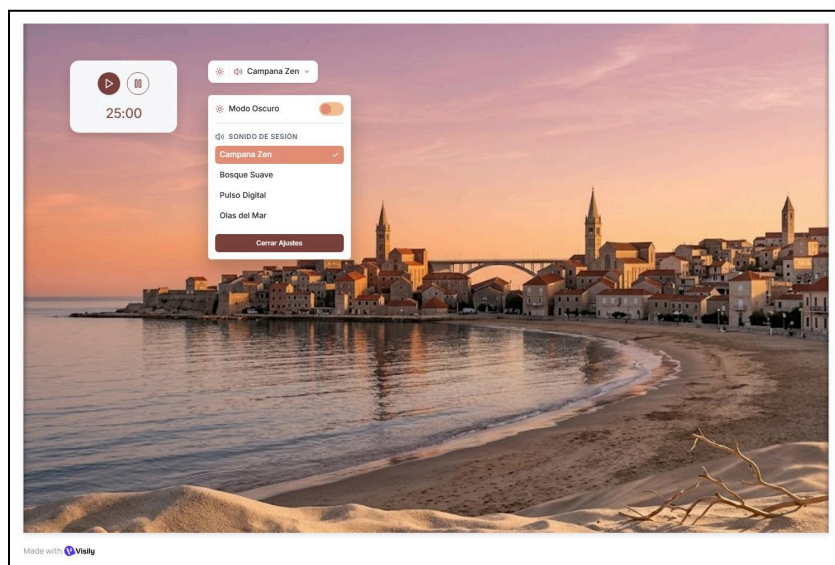
- **Descripción:** Una pantalla estilo grilla o listado donde el usuario visualiza sus “Espacios de Trabajo”. Cada perfil se muestra como una tarjeta.

Página de Tareas

- **Descripción:** La vista detallada de las tareas por perfil de los usuarios. Se presentan en un formato de lista. Incluye filtros rápidos y etiquetas visuales de color.

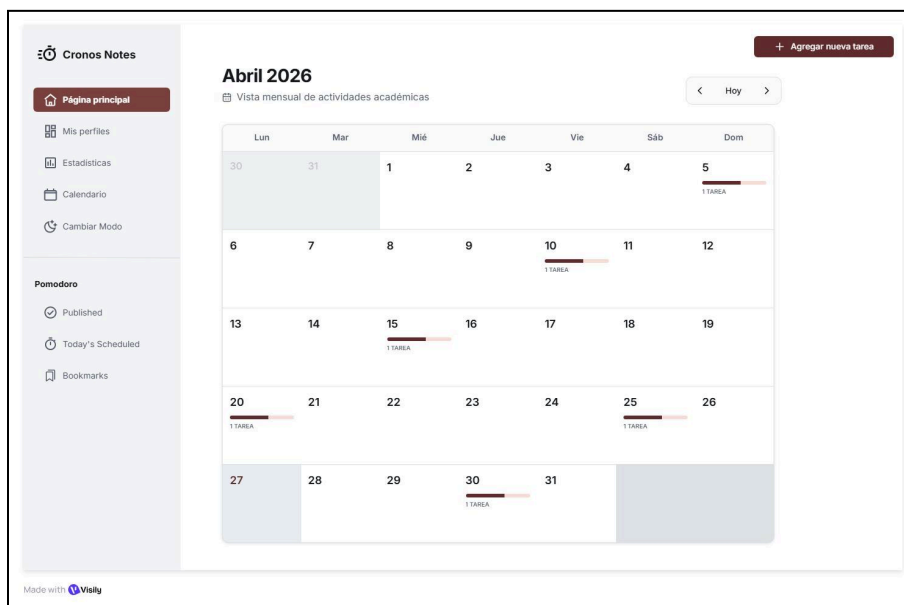
Página de Sesión - Modo Claro y Oscuro

- **Descripción:** Es la pantalla de concentración donde se espera que se ejecuten las tareas. Se caracteriza por tener una imagen de fondo inmersiva. En pantalla se poseen varios widgets, como los controles del temporizador, un mezclador de sonidos y diversos botones con íconos y textos descriptivos. También se permite personalizar la experiencia mezclando los sonidos y modificando el fondo.



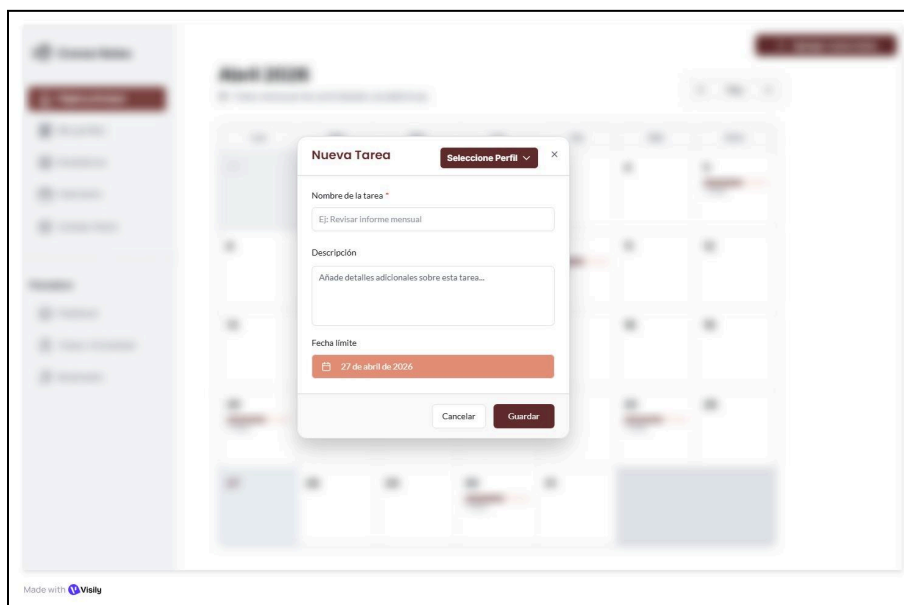
Página de Calendario

- **Descripción:** Es una vista mensual tradicional. Incluye navegación entre meses y un botón para agregar tareas directamente desde ahí. Brinda al usuario una perspectiva a mediano y largo plazo de las responsabilidades del mismo.



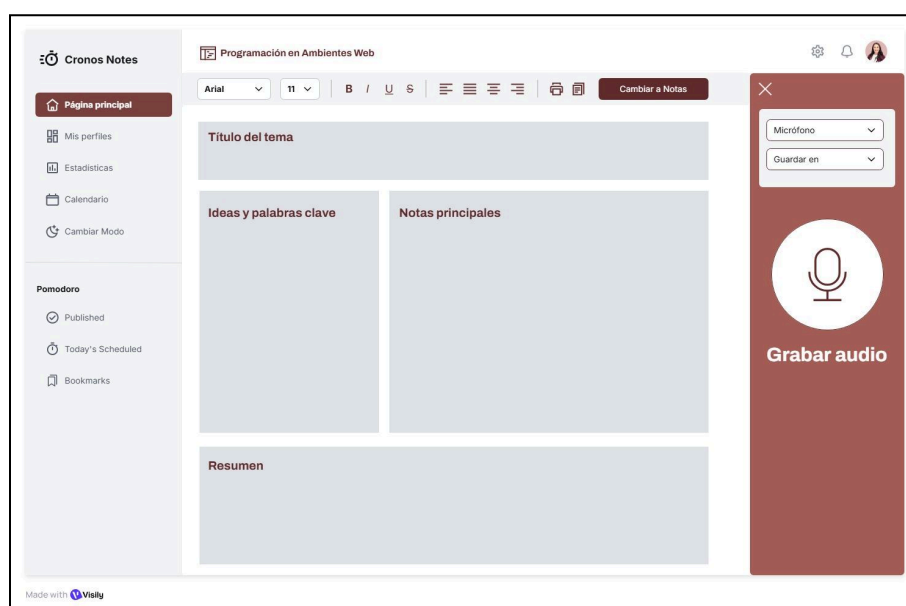
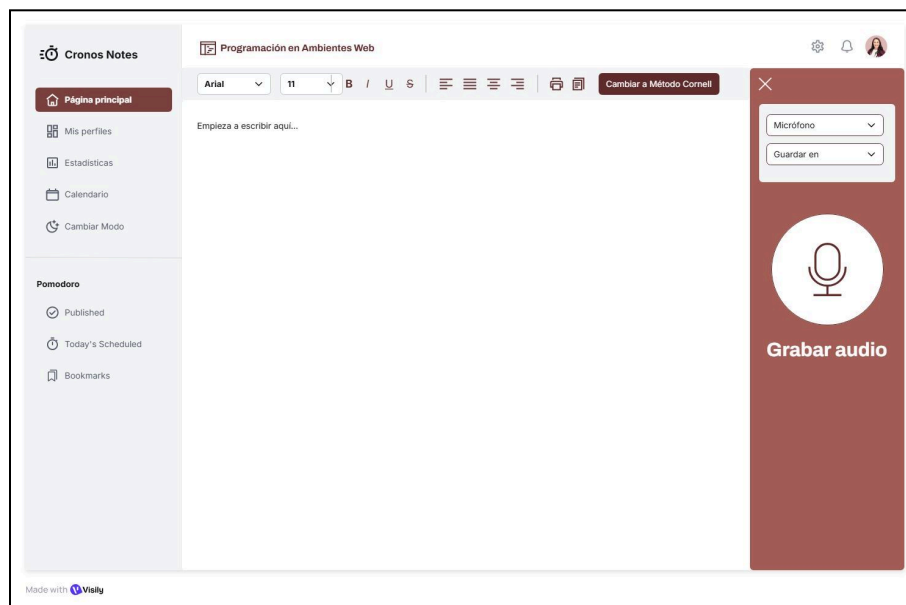
Modal de Nueva Tarea

- **Descripción:** Es una ventana superpuesta que se despliega sobre el calendario. Contiene los campos necesarios para agregar una tarea, con la diferencia de deber asociar un “Perfil” a la tarea.



Página de Toma de Apuntes

- **Descripción:** Es un entorno completo para guardar texto y voz. Presenta una barra de herramientas de texto, un selector de método de tomas de apuntes (como el Método Cornell que divide la hoja en título, ideas clave, notas principales y resumen) y una barra lateral dedicada a grabar audio.



Estructura del Proyecto

Cronos Notes está construido siguiendo una arquitectura **Cliente-Servidor** moderna, utilizando **Inertia.js** como conector entre **Laravel (Backend)** y **Vue (Frontend)**.

Inertia es la tecnología que nos permite construir aplicaciones de una sola página (SPA) modernas utilizando frameworks de frontend, pero sin la necesidad de crear una API REST. Gracias a ella, es posible mantener el enrutamiento en el servidor, usando el sistema de rutas y controladores clásico del framework. Al pasar los datos desde los controladores, Inertia los intercepta y los entrega automáticamente a los componentes de Vue.

Carpetas del Proyecto

Cronos-Notes/	
├─ app/	# Capa Lógica del Backend (PHP)
│ ├─ Http/	# Controladores, Middleware y Requests HTTP
│ ├─ Models/	# Modelos de Datos (Eloquent ORM)
│ ├─ Services/	# Servicios de Lógica de Negocio (Integraciones, IA)
│ └─ Providers/	# Service Providers de Laravel
├─ bootstrap/	# Configuración de inicio del Framework
├─ config/	# Archivos de configuración de Laravel
├─ contexto/	# Documentación inicial y archivos SQL (PDFs, esquemas)
├─ database/	# Persistencia y Esquemas
│ ├─ migrations/	# Definición de tablas de la Base de Datos
│ └─ seeders/	# Datos de prueba para desarrollo
├─ docs/	# Documentación técnica del sistema
├─ public/	# Recursos estáticos compilados y de acceso público
├─ resources/	# Capa Frontend y Vistas
│ ├─ js/	# Código de la aplicación Vue.js
│ │ ├─ Components/	# Componentes Vue reutilizables (Botones, Modals, etc.)
│ │ ├─ Composables/	# Lógica reactiva reutilizable de Vue (Composition API)
│ │ ├─ Layouts/	# Estructuras de página comunes (Sidebar, Zen view, etc.)
│ │ └─ Pages/	# Pantallas/Vistas completas renderizadas por Inertia
│ └─ css/	# Estilos globales y configuración de Tailwind CSS
├─ routes/	# Rutas de la Aplicación
│ └─ web.php	# Rutas web del sistema (manejadas por Inertia)
├─ temp-audio-page/	# Prototipo/Maqueta temporal para la vista de audio
├─ tests/	# Pruebas automatizadas (PHPUnit)
└─ vite.config.js	# Configuración del empaquetador Vite

Explicación de los Directorios

Capa Backend: *app/*

app/Http/Controllers/: Actúan como orquestadores. Su única responsabilidad es recibir la petición del frontend, llamar a los servicios correspondientes para procesar la información y retornar una respuesta Inertia.

app/services/: Para evitar sobrecargar a los controladores o modelos con lógica de negocio compleja, se crearon clases de servicio dedicadas (ej. llamadas a la API de Gemini o integraciones externas). Esto permite reutilizar lógica y facilita las pruebas unitarias aisladas.

app/Models/: Representa nuestro modelo de dominio. Utiliza Eloquent ORM para que la interacción con la base de datos sea mediante objetos, abstrayendo las consultas SQL crudas.

Capa Frontend: *resources/js/*

resources/js/Pages/: Contiene las vistas completas (ej. Dashboard, Calendario, apunte). Cada archivo aquí mapea 1-a-1 con un retorno del controlador Laravel mediante *Inertia::render('Pages/NombrePagina')*.

resources/js/Components/: Componentes pequeños y reutilizables en múltiples páginas para asegurar la consistencia visual y reducir la duplicación de código.

resources/js/Layouts/: Define la estructura exterior de la aplicación (ej. la barra lateral de navegación y la racha del usuario). Las páginas se inyectan dentro de estos layouts, lo que evita renderizar la barra lateral individualmente en cada vista.

resources/js/Composables/: Abstrae la lógica reactiva del frontend (ej. el conteo del temporizador Pomodoro o el control del reproductor de sonido) para que pueda ser importado en cualquier componente que lo requiera.

Base de Datos

database/migrations/: Control de versiones para nuestra base de datos. En lugar de compartir archivos SQL crudos que se desincronizan rápidamente entre desarrolladores, las migraciones permiten que cualquiera configure la estructura de la base de datos local ejecutando *php artisan migrate*.

Codificación del Proyecto

RF-01: Autenticación de Usuario

Descripción y Objetivo

Este requerimiento define la capacidad del sistema para gestionar el acceso seguro de los usuarios a la plataforma. Consiste en la autenticación, el registro de nuevos perfiles, la validación de sus datos y la persistencia de las sesiones.

Se desglosa en los siguientes sub-requerimientos:

- **RF 1.1: Datos de usuario:** Permite almacenar de forma estructurada la información básica del usuario: nombre, apellido, correo electrónico y contraseña segura en la base de datos.
- **RF 1.2: Validaciones de datos:** El sistema debe comprobar que el formato de los datos provistos sea correcto (ej. correo electrónico válido, contraseñas coincidentes y longitud adecuada) antes de procesarlos. Además, protege contra fuerza bruta limitando intentos fallidos de login.
- **RF 1.3: Sesiones de usuario:** Gestiona la persistencia del estado de autenticación a través de una cookie segura vinculada a una sesión física registrada en la base de datos, manteniéndolo conectado hasta que el token expire (24 horas) o cierre sesión voluntariamente.

Tecnologías, Herramientas y Librerías

Este requerimiento se implementa utilizando las siguientes capas tecnológicas del framework Laravel y Vue 3:

- **Laravel Core & Eloquent ORM:** Persistencia de datos en la tabla Usuario y SesionUsuario mediante relaciones de tipo *One-to-Many*.

- **Laravel Rate Limiter:** Protege el endpoint de inicio de sesión de ataques de fuerza bruta. Permite un máximo de 5 intentos por minuto (utilizando IP y email como clave) antes de bloquear temporalmente al usuario.
- **Bcrypt (Laravel Hash Facade):** Cifrado unidireccional de contraseñas para el almacenamiento seguro.
- **Inertia.js & Vue 3:** Interfaces reactivas para las vistas de login y registro, enviando peticiones asíncronas sin recargar toda la aplicación.
- **CheckCustomSession (Middleware Personalizado):** Interceptor de peticiones HTTP que valida la cookie `cronos_session_token` contra los registros de la base de datos para autenticar al usuario de manera programática en cada petición web.

Archivos Involucrados en el Requerimiento

- **Frontend**

Register.vue (/resources/js/Pages/auth/) - Componente que renderiza el formulario de registro y realiza el envío de datos POST a */register*.

Login.vue (/resources/js/Pages/auth/) - Componente del formulario de acceso web con soporte para recordar credenciales y enlace al login de Google.

- **Backend y Controladores**

RegisteredUserController.php (/app/Http/Controllers/Auth/) - Controlador que valida los campos de registro de usuario y guarda la información en la base de datos.

AuthenticatedSessionController.php (/app/Http/Controllers/Auth/) - Maneja el inicio de sesión personalizado, la creación de la cookie y la desactivación física del registro de sesión al cerrar la cuenta.

LoginRequest.php (/app/Http/Requests/Auth/) - Gestiona la validación de credenciales del login y el control de peticiones (Rate Limiting).

CheckCustomSession.php (/app/Http/Middleware/Auth/) - Middleware que intercepta la cookie *cronos_session_token* y autologuea al usuario si el token es válido y no ha caducado.

- **Modelos y Datos (Eloquent ORM)**

User.php (/app/Models/) - Modelo de usuario (*Usuario*) que provee las funciones helper de inicio (*iniciarSesionPersonalizada*) y cierre (*cerrarSesionPersonalizada*) de sesiones en DB.

SesionUsuario.php (/app/Models/) - Modelo para representar y verificar la validez y expiración física de los tokens persistidos.

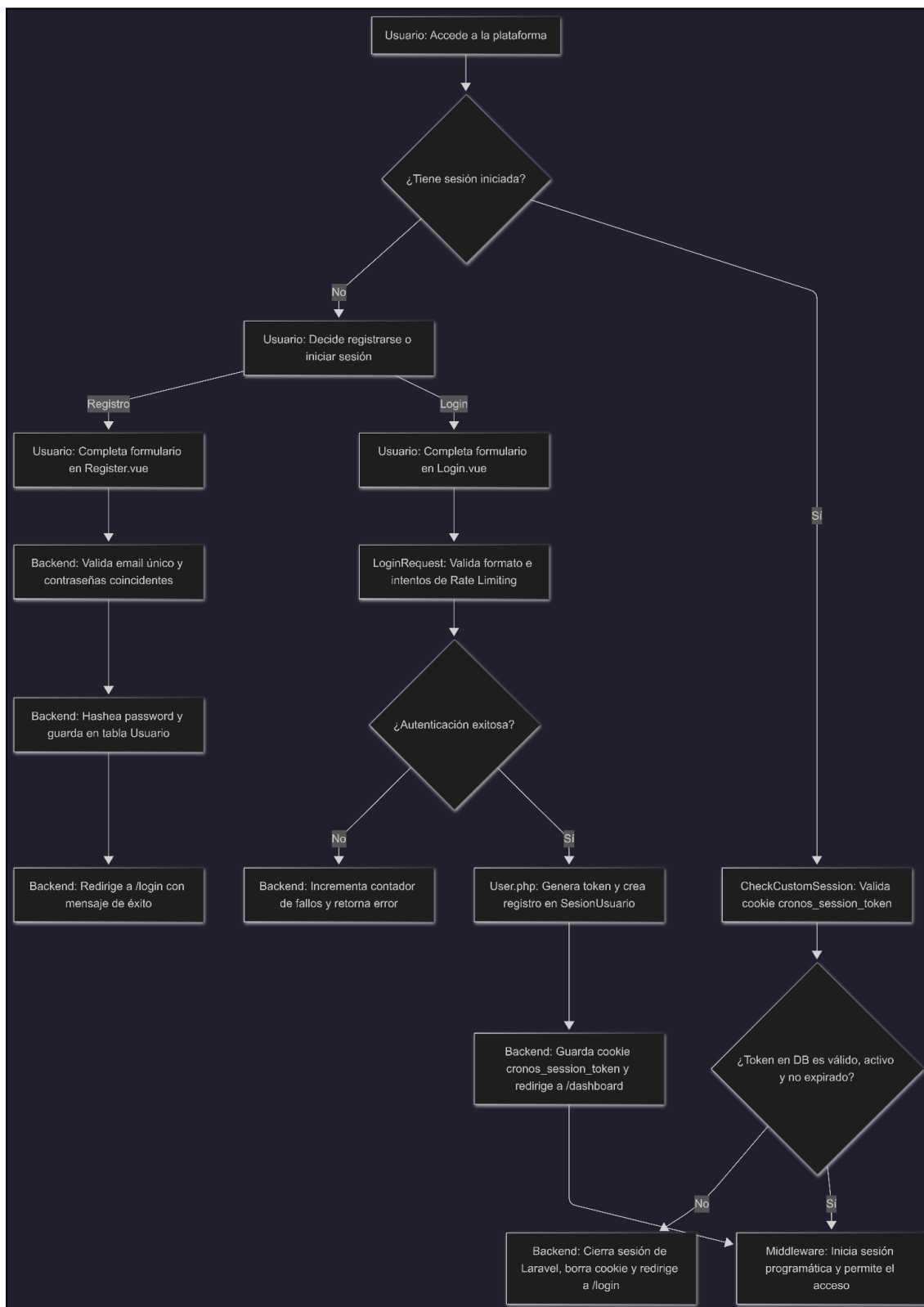
- **Enrutamiento y Base de Datos**

auth.php (/routes/) - Rutas de endpoints /login, /register, /logout y redireccionamiento OAuth.

create_users_table.php (/database/migrations) - Migración de estructura para la tabla *Usuario*.

create_sesion_usuario_recuperacion_password_table.php (/database/migrations) - Migración para la tabla *SesionUsuario*.

Flujo de Datos y Control



1. Acceso Inicial y Validación de Sesión (Middleware):

- Cuando un usuario solicita cualquier ruta protegida, el middleware *CheckCustomSession.php* intercepta la petición.
- Lee el valor de la cookie *cronos_session_token*. Si existe, busca el token correspondiente en la tabla *SesionUsuario*.
- Si la sesión es válida (está activa y no ha caducado), se autentica al usuario programáticamente en el framework y se le permite el acceso.
- De lo contrario, se limpia cualquier rastro de la sesión, se elimina la cookie y se lo redirige al formulario de inicio de sesión.

2. Flujo de Registro (Register.vue):

- El usuario completa los datos en el componente *Register.vue* y se envían por POST a */register*.
- El controlador *RegisteredUserController.php* valida los campos.
- Si las validaciones fallan (ej. contraseñas distintas, correo inválido o ya registrado), se retornan los errores al frontend.
- Si las validaciones son exitosas, se aplica hashing seguro a la contraseña y se crea un registro en la tabla *Usuario*. Luego se redirige a */login* con un mensaje flash de éxito.

3. Flujo de Inicio de Sesión (Login.vue):

- El usuario ingresa sus credenciales en *Login.vue*.
- *LoginRequest.php* verifica el control de tasa de intentos (Rate Limiting). Si se superan los 5 intentos en un minuto, bloquea las peticiones.
- Si las credenciales son incorrectas, se incrementa el contador de fallos y se retorna un mensaje de error.
- Si la autenticación es correcta, se invoca a *iniciarSesionPersonalizada()* en el modelo *User.php* para generar un nuevo token criptográfico, registrar la sesión

activa en la base de datos y guardar la cookie *cronos_session_token* en el navegador con duración de 24 horas.

Pruebas y Validación

- **Pruebas de Registro**

1. **Precondición:** El correo a ingresar no debe estar registrado en el sistema.
2. **Paso 1:** Dirigirse a la URL */register*, llenar el campo correo con un formato inválido (ej: *test_invalido*).
 - a. **Resultado Esperado:** El navegador o el framework debe reportar que el formato no corresponde a un correo electrónico válido.
3. **Paso 2:** Rellenar con datos correctos pero ingresar contraseñas distintas en "Contraseña" y "Confirmar contraseña". Hacer clic en "Registrarse".
 - a. **Resultado Esperado:** Debe retornar un error de validación indicando que la confirmación de la contraseña no coincide.
4. **Paso 3:** Introducir los datos correctos y pulsar "Registrarse".
 - a. **Resultado Esperado:** Debe redirigir automáticamente a la pantalla */login* con un mensaje indicando que el registro fue completado exitosamente.

- **Pruebas de Login y Persistencia**

1. **Precondición:** Tener la cuenta registrada con la contraseña.
2. **Paso 1:** Dirigirse a */login*, escribir el correo válido y una contraseña errónea. Presionar "Ingresar". Repetir esto consecutivamente 6 veces en menos de un minuto.
 - a. **Resultado Esperado:** Del intento 1 al 5 debe mostrar "Estas credenciales no coinciden...". En el intento 6 debe bloquear temporalmente las peticiones indicando que ha realizado demasiados intentos de inicio de sesión y debe esperar un número determinado de segundos.
3. **Paso 2:** Tras superar el bloqueo temporal, ingresar las credenciales correctas.

- a. **Resultado Esperado:** Redirección fluida al Dashboard de la aplicación.
- 4. **Paso 3:** Abrir las herramientas de desarrollador en el navegador (F12), ir a la pestaña Aplicación/Almacenamiento -> Cookies.
 - a. **Resultado Esperado:** Debe existir una cookie llamada *cronos_session_token* marcada como *HttpOnly* (para protección XSS) y con vigencia de 24 horas.
- 5. **Paso 4:** Hacer clic en el botón de cerrar sesión en la aplicación.
 - a. **Resultado Esperado:** Redirección inmediata a la landing page pública (/). En el navegador la cookie *cronos_session_token* debe ser eliminada y en la tabla SesionUsuario de la base de datos el token correspondiente debe cambiar su estado a activa = 0.

RF-02: Gestión de Perfiles

Descripción y Objetivo

Este requerimiento define la capacidad del sistema para organizar y personalizar la información del usuario en diferentes ámbitos o espacios de trabajo (perfiles). Los perfiles actúan como contenedores aislados de tareas y apuntes para ayudar a separar contextos (ej. "Estudio", "Trabajo", "Proyectos Personales").

Se compone de los siguientes sub-requerimientos:

- **RF 2: Gestión de Perfiles:** Permite al usuario crear, editar, eliminar y cambiar el perfil activo del sistema.
- **RF 2.1: Personalización de perfiles:** Ofrece configuraciones visuales y de identificación para cada perfil, tales como modificar el título, agregar una descripción y elegir un icono representativo (ej. carpeta, libro, código).

Reglas de Negocio y Restricciones

- **Límite de perfiles:** Cada usuario puede crear un máximo de 5 perfiles.
- **Nombres únicos:** No se permite que un mismo usuario tenga dos perfiles con el mismo nombre (título).
- **Asociación en cascada:** Al eliminar un perfil, se eliminan todas sus tareas y apuntes relacionados en la base de datos.

Tecnologías, Herramientas y Librerías

Este requerimiento utiliza la arquitectura estándar de Laravel, Vue 3 e Inertia.js:

- **Laravel Core & Eloquent ORM:** Persistencia en la tabla *Perfil* y control de relaciones con *Usuario* y *Tarea*.

- **Laravel Policy (PerfilPolicy):** Gobierna los permisos del perfil según los roles jerárquicos (Lector, Editor, Administrador), asegurando que solo el propietario o usuarios autorizados puedan ver, editar o eliminar el perfil.
- **Inertia.js & Vue 3:** La página *GestionPerfil.vue* maneja el listado dinámico, la apertura de diálogos modales para la creación y edición de perfiles (incluyendo selección de iconos visuales) sin recargas.
- **Sesión de PHP/Laravel:** Se almacena la clave *perfilActivo* en la sesión del servidor para persistir qué perfil está utilizando el usuario en el dashboard.

Archivos Involucrados en el Requerimiento

- **Frontend**

GestionPerfil.vue (/resources/js/Pages/) - Componente principal de la interfaz que permite administrar perfiles, cambiar de perfil activo, editar nombres, descripciones y seleccionar iconos.

- **Backend y Controladores**

PerfilController.vue (/app/Http/Controllers/) - Controlador que gestiona los endpoints CRUD de perfiles y la asignación del perfil activo en la sesión.

- **Políticas de Acceso**

PerfilPolicy.php (/app/Policies/) - Define las autorizaciones para asegurar que los usuarios no propietarios tengan permisos específicos (Lector, Editor o Administrador) y bloquea operaciones indebidas.

- **Modelos y Datos**

Perfil.php (/app/Models/) - Modelo Eloquent que representa la tabla *Perfil*, definiendo las relaciones con *User*, *Tarea* y *Apunte*.

User.php (/app/Models/) - Modelo de usuario con la relación *perfiles()*.

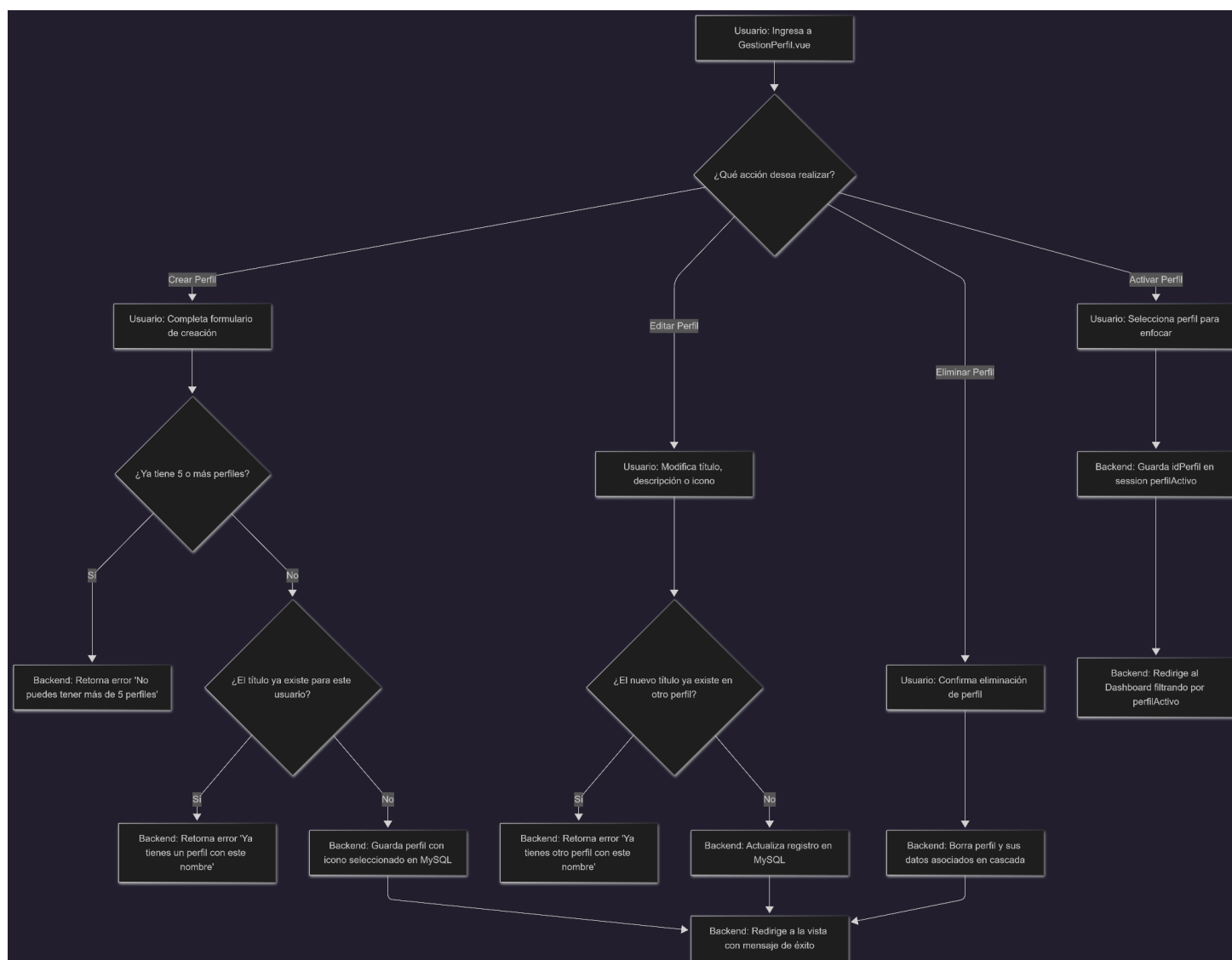
- **Enrutamiento y Base de Datos**

web.php (/routes/) - Define las rutas `/perfiles`, `/perfiles/activar` y `/gestion-perfil`.

create_perfiles_table.php (/database/migrations/) - Migración de estructura para la tabla *Perfil*.

add_icono_perfil_to_perfil_table.php (/database/migrations/) - Migración que añade la columna *iconoPerfil* para almacenar el icono personalizado.

Flujo de Datos y Control



1. Listar Perfiles:

- Al entrar a */gestion-perfil*, *PerfilController.php* consulta todos los perfiles creados por el usuario y los compartidos con él, y los envía como props a *GestionPerfil.vue*.

2. Crear Perfil:

- El usuario abre el modal "Crear Perfil" e ingresa título (máx. 30 caracteres), descripción (máx. 100 caracteres) y selecciona un icono visual.
- Envía la petición POST a */perfiles*. El backend valida que el usuario tenga menos de 5 perfiles y que el nombre sea único para su cuenta. Si pasa, crea el registro y redirige refrescando la vista.

3. Editar Perfil:

- El usuario selecciona un perfil existente y presiona "Editar". Modifica los campos en el modal y hace clic en "Guardar".
- Se envía un PUT a */perfiles/{id}*. El controlador comprueba que el nuevo título no esté en uso por otro perfil del mismo usuario y actualiza los datos.

4. Eliminar Perfil:

- El usuario presiona el botón de eliminar y confirma en el modal. Se envía un DELETE a */perfiles/{id}*.
- El backend elimina el registro, lo cual dispara la eliminación en cascada en la base de datos (tareas y apuntes vinculados).

5. Activar Perfil:

- El usuario hace clic en el botón de activar sobre una tarjeta de perfil. Se envía una petición POST a */perfiles/activar*.
- El backend almacena el *idPerfil* en la variable de sesión *perfilActivo* y redirige al dashboard. Toda vista subsiguiente usará esta variable para filtrar las tareas y apuntes mostrados.

Pruebas y Validación

- **Pruebas de Creación e Íconos**

1. **Precondición:** Tener menos de 5 perfiles creados.
2. **Paso 1:** Ir a "Mis Perfiles", hacer clic en "Añadir Perfil" sin rellenar el título.
 - a. **Resultado Esperado:** El formulario debe impedir el envío indicando que el título es requerido.
3. **Paso 2:** Completar el título con un nombre que ya exista (ej: "Estudio"), elegir cualquier icono y guardar.
 - a. **Resultado Esperado:** Debe mostrar un mensaje de error indicando "Ya tienes un perfil con este nombre".
4. **Paso 3:** Modificar el título por uno único (ej: "Gimnasio"), elegir el icono de la pesa y guardar.
 - a. **Resultado Esperado:** El perfil debe guardarse correctamente y aparecer listado en la interfaz mostrando el icono de la pesa.

- **Pruebas de Límites**

1. **Precondición:** Tener 5 perfiles ya creados en el listado.
2. **Paso 1:** Hacer clic en "Añadir Perfil", rellenar campos con un nombre único y presionar "Guardar".
 - a. **Resultado Esperado:** La acción debe ser rechazada y retornar el mensaje "No puedes tener más de 5 perfiles".

- **Pruebas de Selección Activa**

1. **Paso 1:** Hacer clic en la tarjeta del perfil.
 - a. **Resultado Esperado:** Debe redirigir al Dashboard principal. La barra lateral superior o el estado global de la app deben indicar el perfil activo actual. Todas las tareas y apuntes listados deben corresponder únicamente a este perfil.

RF-03: Gestión de Tareas

Descripción y Objetivo

Este requerimiento define la capacidad del sistema para gestionar las tareas de concentración del usuario dentro del perfil activo. Las tareas organizan el trabajo pendiente y sirven como unidad básica sobre la cual se ejecutan los ciclos de temporización Pomodoro.

Se desglosa en los siguientes sub-requerimientos:

- **RF 3: Gestión de Tareas:** Permite al usuario realizar el ABM (Alta, Baja, Modificación) de tareas y marcarlas como completadas.
- **RF 3.1: Datos de Tareas:** Estructura la información de la tarea en la base de datos (título, descripción, fecha de vencimiento, prioridad inicial, estimación de esfuerzo en pomodoros y vinculación con un perfil específico).
- **RF 3.2: Visualización de Tareas:** Provee una interfaz flexible para listar las tareas del perfil activo, ordenadas cronológicamente por su fecha límite, permitiendo alternar entre vista de Rejilla (Grid) y Lista, así como filtrar dinámicamente según su estado y prioridad.
- **RF 3.3: Etiquetas Visuales:** Identifica visualmente el nivel de prioridad y el estado actual de cada tarea mediante un sistema cromático intuitivo y badges descriptivos.

Tecnologías, Herramientas y Librerías

Este requerimiento se apoya en el stack reactivo del proyecto:

- **Laravel Core & Eloquent ORM:** Almacenamiento en la tabla *Tarea* con integridad referencial hacia la tabla *Perfil* (eliminación en cascada).

- **Inertia.js & Vue 3:** La página *GestionTareas.vue* recibe las tareas cargadas en las props y expone computadas reactivas (*tareasFiltradas*) para filtrar la lista al instante en el navegador del cliente sin realizar nuevas consultas al servidor.
- **EstadisticaService (Backend Service):** Servicio PHP que se activa cuando una tarea cambia a estado '*Completado*', sumando de manera automática el conteo de la tarea resuelta en las estadísticas globales del usuario.
- **Bootstrap Icons:** Librería de iconos vectoriales para identificar los estados de las tareas.

Archivos Involucrados en el Requerimiento

- **Frontend**

GestionTareas.vue (/resources/js/Pages/) - Vista principal del panel de tareas. Maneja el listado, los filtros de estado/prioridad en memoria (*filtroEstado*, *filtroPrioridad*), los modales de creación/edición, la conmutación de rejilla/lista (*esVistaGrid*) y la priorización inteligente con IA.

- **Backend**

TareaController.php (app/Http/Controllers/) - Controlador que gestiona los métodos CRUD de tareas y valida que las peticiones tengan permisos correctos según el perfil activo del usuario.

- **Modelos y Datos**

Tarea.php (/app/Models/) - Modelo Eloquent para la tabla Tarea con casts para formateo de fechas e integridad de datos.

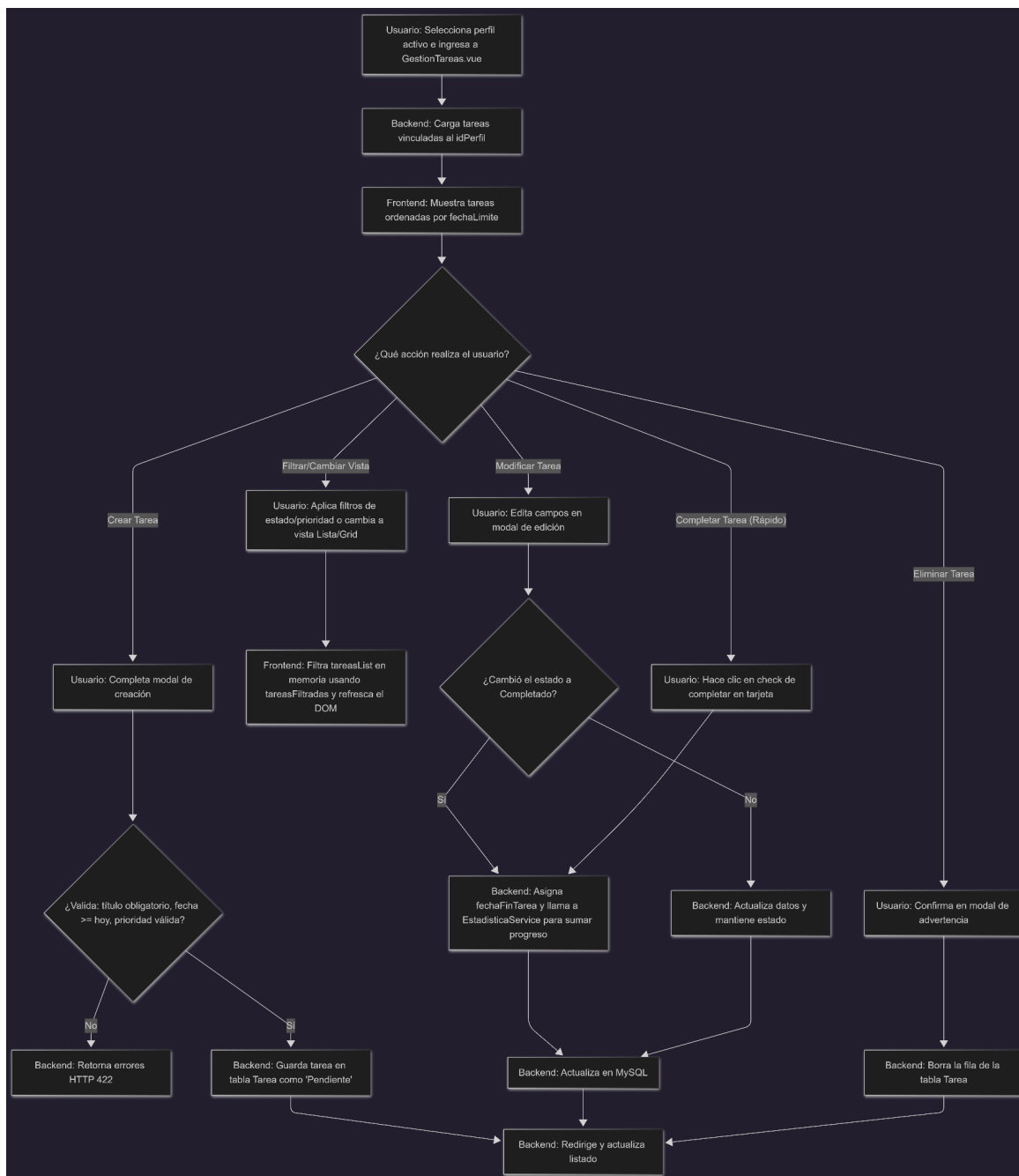
Perfil.php (/app/Models/) - Define la relación inversa de pertenencia *tareas()*.

- **Enrutamiento y Base de Datos**

web.php (/routes/) - Contiene las rutas para las tareas (*/tareas*, */tareas/{id}*, */tareas/{id}/completar*, etc.) bajo el middleware de autenticación.

create_tareas_table.php (/database/migrations/) - Estructura de migración de la base de datos para la tabla Tarea.

Flujo de Datos y Control



1. Carga y Ordenamiento Inicial:

- Al entrar a */tareas*, el controlador obtiene el *perfilActivo* de la sesión.
- Consulta a la base de datos ordenando por *fechaLimite* ascendente (*orderBy('fechaLimite', 'asc')*) para asegurar que el usuario vea primero lo que vence antes.

2. Estructura y Validación de Datos (RF 3.1 & RF 3.2):

- Para Crear una tarea se validan las siguientes reglas:
 - **tituloTarea:** Requerido, string, máximo 45 caracteres.
 - **descripcionTarea:** Opcional, string, máximo 200 caracteres.
 - **fechaLimite:** Requerido, debe ser igual o posterior a hoy (*after_or_equal:today*).
 - **prioridadTarea:** Requerido, valores permitidos: 'Baja', 'Media', 'Alta'.
 - **estimacionEsfuerzo:** Opcional, entero mínimo 1 (indica cantidad de pomodoros que el usuario cree que le llevará completar la tarea).
- Para Modificar se permite cambiar el estado (*estadoTarea en ['Pendiente', 'En Progreso', 'Completado']*). Si el estado pasa a 'Completado', se guarda automáticamente la fecha de hoy en *fechaFinTarea*.

3. Filtros en el Cliente (RF 3.2):

- El frontend cuenta con dos variables reactivas: *filtroEstado* y *filtroPrioridad*.
- La propiedad computada *tareasFiltradas* evalúa las combinaciones seleccionadas y renderiza solo los elementos que coincidan, logrando una respuesta de filtrado instantánea sin tocar el servidor.

4. Diseño de Código Cromático (RF 3.3):

- Las prioridades se mapean a estilos específicos mediante la función *clasePrioridad*:
 - **Alta:** Fondo granate (612c2d), texto blanco.
 - **Media:** Fondo salmón (e38e76), texto blanco.

- **Baja:** Fondo durazno (f4be95), texto granate.
- Los estados se identifican con colores y Bootstrap Icons a través de *claseEstado*:
 - **Pendiente:** Dot gris (adb5bd) e icono *bi-dash-circle*.
 - **En Progreso:** Dot naranja (ff922b) e icono *bi-clock-history*.
 - **Completado:** Dot verde (51cf66) e icono *bi-check-circle-fill*.

Pruebas y Validación

● Pruebas de CRUD y Estructura

1. **Precondición:** Tener un perfil activo asignado en la sesión.
 2. **Paso 1:** Hacer clic en el botón de agregar tarea (botón flotante +), rellenar el formulario omitiendo el título y con una fecha límite del día de ayer. Guardar.
 - a. **Resultado Esperado:** El formulario debe dar error indicando que el título es requerido y la fecha límite debe ser hoy o posterior.
 3. **Paso 2:** Rellenar correctamente con Título, Prioridad: "Alta", Fecha Límite: Hoy. Guardar.
 - a. **Resultado Esperado:** La tarea debe aparecer de inmediato. El fondo de la etiqueta de prioridad debe ser rojo oscuro/granate.
 4. **Paso 3:** Abrir la base de datos en la tabla Tarea.
 - a. **Resultado Esperado:** Debe existir la fila correspondiente con *fechaInicioTarea* establecida en el día actual y *estadoTarea* = 'Pendiente'.
- ### ● Pruebas de Modificación y Completado
1. **Paso 1:** En la tarjeta de la tarea, hacer clic en el botón de modificar (icono de lápiz), cambiar el estado a "En Progreso" y guardar.
 - a. **Resultado Esperado:** El indicador de estado debe cambiar a naranja con el texto "En proceso".

2. **Paso 2:** Marcar la tarea como completada (clic en el botón de verificación).

a. **Resultado Esperado:** La tarea cambia a color verde con un ticket. En base de datos, el campo *fechaFinTarea* debe haberse completado con la fecha de hoy. El contador de tareas finalizadas en el perfil de usuario debe haberse incrementado en 1.

- **Pruebas de Filtros y Conmutador de Vistas**

1. **Paso 1:** Teniendo al menos una tarea en estado "Pendiente" y otra "Completado", cambiar el filtro de Estado a "Pendiente".

a. **Resultado Esperado:** Solo deben renderizarse en pantalla las tareas con estado "Pendiente".

2. **Paso 2:** Presionar el conmutador de vista a "Lista".

a. **Resultado Esperado:** Las tareas deben reorganizarse horizontalmente de manera compacta, alineadas una debajo de otra en lugar de en bloques de cuadrícula.

RF-04: Sesión Pomodoro

Descripción y Objetivo

Este requerimiento define el núcleo funcional del temporizador de productividad basado en la Técnica Pomodoro. Su objetivo es ayudar al usuario a concentrarse en sus actividades dividiendo el tiempo de trabajo en bloques enfocados, alternados con descansos cortos y largos para mitigar el agotamiento cognitivo.

Se compone de los siguientes sub-requerimientos:

- **RF 4: Sesión Pomodoro:** Vincula una sesión física de trabajo a una tarea específica del perfil activo, controlando el inicio, pausa y finalización del ciclo de concentración.
- **RF 4.1: Temporizador Pomodoro:** Implementa el cronómetro regresivo que por defecto corre ciclos estándar de 25 minutos de trabajo y 5 minutos de descanso. Si la sesión se detiene antes de finalizar, el sistema registra proporcionalmente el tiempo transcurrido en minutos completos en la base de datos para no perder el esfuerzo acumulado.
- **RF 4.2: Alertas de Sesión:** Emite una ventana modal emergente de aviso no invasivo al finalizar cada intervalo de tiempo, indicando al usuario que ha completado la fase (estudio/trabajo o descanso) y guiándolo hacia la siguiente etapa.

Tecnologías, Herramientas y Librerías

- **Vue 3 Composables (*usePomodoroTimer*):** Composable reactivo que encapsula la lógica pura del temporizador, los intervalos de segundo a segundo, la persistencia en localStorage ante recargas accidentales de página y el registro automático de minutos trabajados en background.
- **Inertia.js & Laravel Controller:** *PomodoroController.php* recibe peticiones síncronas y asíncronas (Axios) para iniciar, registrar minutos minuto a minuto, pausar y finalizar la sesión.

- **HTML5 Web Audio API & Howler.js:** Carga y reproduce dinámicamente bucles de música y ruido ambiental durante la fase activa, pausándose y deteniéndose automáticamente cuando el temporizador se detiene.
- **AlertModal (Componente Vue):** Componente modal personalizado *AlertModal.vue* utilizado para desplegar las alertas de fin de ciclo en pantalla completa.

Archivos Involucrados en el Requerimiento

- **Frontend**

SesionZen.vue (/resources/js/Pages/pomodoro/) - Interfaz principal del espacio de concentración. Renderiza el cronómetro, controles de reproducción (Iniciar, Pausar, Finalizar), mezclador de sonidos y las ventanas modales de alerta de fin de fase y de confirmación de cierre.

usePomodoroTimer.js (/resources/js/Composables/) - Composable reactivo que coordina el estado de cuenta regresiva, fases (*work*, *shortBreak*, *longBreak*), conteo de ciclos y sincronización del estado actual con el almacenamiento local (*localStorage*).

- **Backend y Controladores**

PomodoroController.php (/app/Http/Controllers/) - Controlador que gestiona la inicialización de sesiones en base de datos, actualización de estados (pausar/reanudar) y la persistencia del tiempo real trabajado tras la finalización o cancelación.

- **Modelos y Datos**

SesionPomodoro.php (/app/Models/) - Modelo Eloquent que mapea a la tabla *SesionPomodoro* para registrar el estado, ciclos completados y minutos de trabajo acumulados.

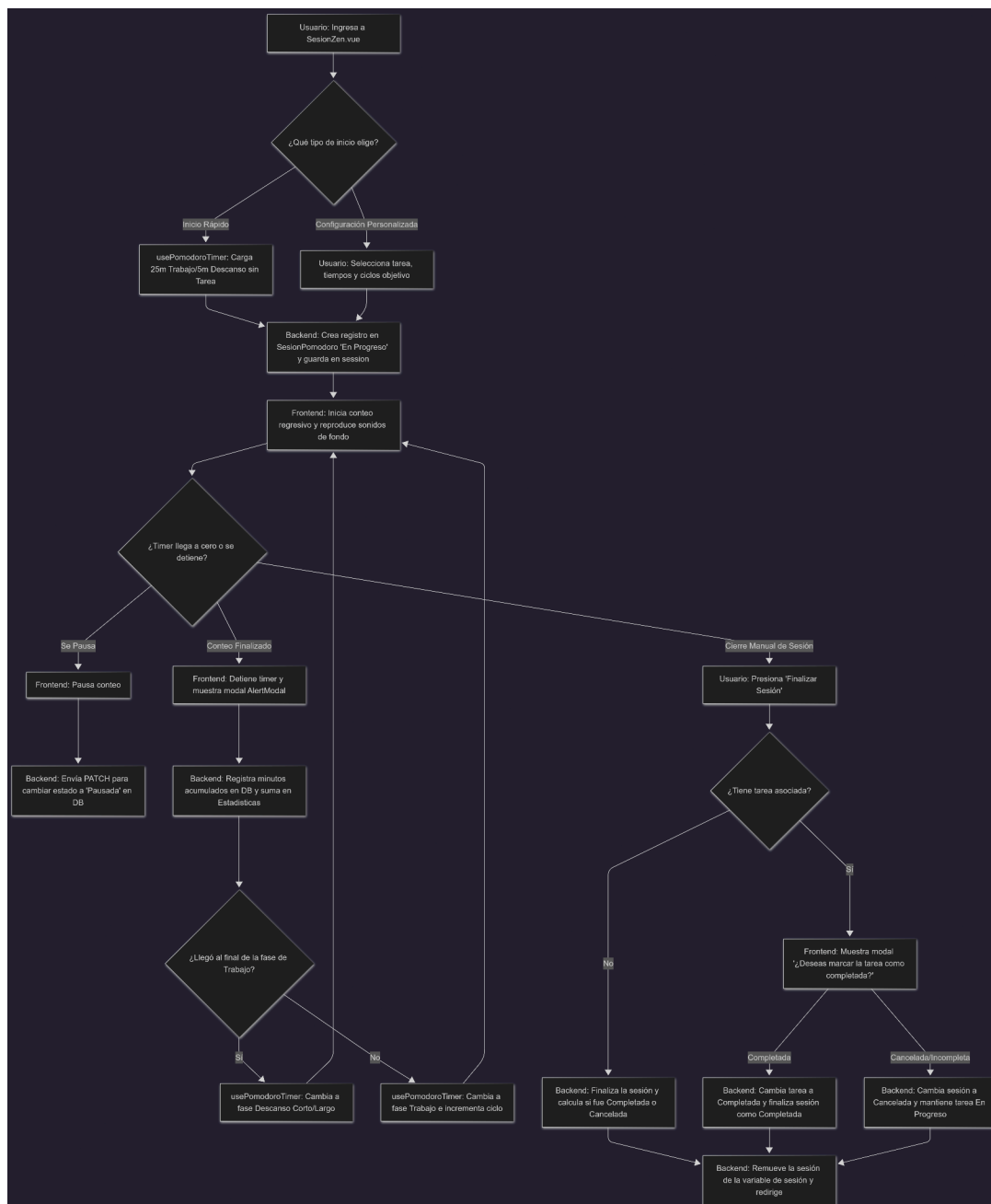
ConfiguracionPomodoro.php (/app/Models/) - Modelo que guarda las preferencias de duración por usuario.

- **Enrutamiento y Base de Datos**

web.php (/routes/) - Define las rutas de API y enrutamiento del temporizador bajo el prefijo /pomodoro.

create_sesion_pomodoro_table.php (/database/migrations/) - Migración para la tabla *SesionPomodoro*.

Flujo de Datos y Control



1. Configuración e Inicialización

- Al invocar *iniciarSesion()*, se valida e inserta un registro en la tabla *SesionPomodoro* en estado '*En Progreso*'. Si hay una tarea seleccionada, esta pasa a estado '*En Progreso*'.
- Se guarda el objeto descriptivo de la sesión en la sesión de Laravel (*sesionPomodoroActiva*) para retornarlo al cliente y evitar que acceda a configurar de nuevo.

2. Ciclo Regresivo en Background

- El front-end inicia el conteo. Cada segundo que transcurre se resta de *timeLeft*.
- Si la fase es de trabajo, cada 60 segundos transcurridos el composable realiza una petición HTTP POST asíncrona en segundo plano a */pomodoro/registrar* informando *minutosTrabajados = 1* y actualizando el acumulado en la base de datos de manera atómica.

3. Pausa y Almacenamiento local

- Al presionar "Pausa", se interrumpe el interval, se detienen los sonidos ambientales y se realiza una petición PATCH a */pomodoro/estado* asignando el estado 'Pausada' en DB.
- El estado actual (tiempo transcurrido, ciclo actual, fase) se guarda en *localStorage* bajo una clave única de la sesión (*pomodoro_session_{idSesion}*) para que si el usuario recarga la pestaña, la sesión continúe exactamente donde estaba.

4. Alertas de Fin de Base

- Cuando el contador llega a *00:00*, se invoca el callback del front-end.
- Se muestra un modal *<AlertModal>* en pantalla indicando la finalización ("¡Sesión de trabajo finalizada!" o "¡Descanso terminado!").

- Tras aceptar, se calcula el siguiente intervalo: si completó la cantidad de ciclos estipulados para el descanso largo (ej. 4 ciclos), el cronómetro se configura con la duración del descanso largo; de lo contrario, inicia un descanso corto y vuelve a comenzar.

5. Cierre de Sesión y Completado de Tarea

- Si el usuario presiona "Finalizar Sesión" manualmente antes de cumplir el objetivo, el front-end calcula los minutos transcurridos en el ciclo de trabajo actual que aún no se registraron en la DB y los envía para su guardado final.
- Si la sesión tenía una tarea asociada, se muestra un modal preguntando si desea dar la tarea por finalizada. De ser afirmativo, la sesión se guarda como 'Completada' y la tarea como 'Completada'. Caso contrario, la sesión se guarda como 'Cancelada' y la tarea se mantiene 'En Progreso'.

Pruebas y Validación

• Pruebas del Temporizador e Intervalos

1. **Precondición:** Estar autenticado y en la pantalla */pomodoro* en estado de Setup.

2. **Paso 1:** Seleccionar una tarea de la lista, configurar 25 minutos de trabajo, 5 de descanso y presionar "Iniciar Sesión".

a. **Resultado Esperado:** El temporizador debe cargar el valor 25:00 y comenzar el descuento segundo a segundo. La tarea elegida debe cambiar su estado en el menú lateral a "En proceso".

3. **Paso 2:** Dejar correr el temporizador durante 1 minuto completo (60 segundos).

a. **Resultado Esperado:** Se debe observar en la consola del navegador (pestaña Red/Network) el envío de una petición HTTP POST a */pomodoro/registrar* con el parámetro *minutosTrabajados = 1*.

4. **Paso 3:** Presionar el botón de "Pausa".

- a. **Resultado Esperado:** El conteo se detiene. Debe dispararse una petición PATCH a `/pomodoro/estado` con `estado = 'Pausada'`.

5. **Paso 4:** Recargar la pestaña del navegador (F5).

- a. **Resultado Esperado:** Gracias a la persistencia en `localStorage`, la interfaz debe reconstruirse mostrando el temporizador detenido en el mismo segundo exacto en el que se pausó.

- **Pruebas de Alertas (Fase Completada)**

1. **Paso 1:** Dejar que el temporizador llegue a `00:00` durante una fase activa de Trabajo.

- a. **Resultado Esperado:** El temporizador se detiene. Debe aparecer una ventana modal emergente con el título "¡Sesión de trabajo finalizada!" y el botón "Aceptar".

2. **Paso 2:** Presionar el botón "Aceptar".

- a. **Resultado Esperado:** El modal debe desaparecer. El temporizador debe configurarse automáticamente con el tiempo de descanso (ej. `05:00`) y quedar listo para iniciar la fase de relajación.

- **Pruebas de Finalización Manual y Tarea**

1. **Precondición:** Tener una sesión iniciada con una tarea asociada.

2. **Paso 1:** Dejar correr la sesión de trabajo por unos minutos y presionar el botón rojo "Finalizar Sesión".

- a. **Resultado Esperado:** El temporizador se detiene y se despliega un cuadro de diálogo preguntando si se desea marcar la tarea como completada.

3. **Paso 2:** Seleccionar "Sí, completar tarea".

- a. **Resultado Esperado:** Se redirige a la pantalla de Pomodoro en estado limpio. En el dashboard la tarea debe figurar como "Finalizada".

RF-05: Sistema de Rachas

Descripción y Objetivo

Este requerimiento define el mecanismo de gamificación y motivación en Cronos Notes. Su objetivo es incentivar la constancia de uso diario y el mantenimiento de hábitos de concentración saludables mediante un contador de "días consecutivos de racha".

Se compone de las siguientes características y reglas de negocio:

- **Racha Activa:** La racha se incrementa en 1 por cada día consecutivo que el usuario complete al menos una sesión pomodoro activa.
- **Regla del Día Consecutivo:** Para mantener la racha, el usuario debe realizar una sesión el día de hoy o haber realizado una el día de ayer. Si transcurre un día completo sin actividad calificada, la racha se rompe y el contador vuelve a cero.
- **Restricción de Duración (RF-M12):** Para evitar fraudes (ej. iniciar y completar sesiones simuladas de 1 minuto), la racha solo aumenta cuando el usuario completa una sesión de trabajo con una duración configurada de exactamente 25 minutos.
- **Límite Diario:** La racha solo puede incrementarse una vez al día. Sesiones completadas el mismo día después del incremento diario son registradas en el historial de trabajo pero no alteran el contador de la racha actual.

Tecnologías, Herramientas y Librerías

El sistema de rachas se implementa mediante servicios y persistencia relacional en el backend:

- **Laravel Core & Carbon:** Para realizar las operaciones de fechas y tiempo, comparando si la última racha finalizó ayer (*Carbon::yesterday()*) o antes para declarar su pérdida.

- **EstadisticaService:** Clase centralizada *EstadisticaService.php* que evalúa las fechas, crea los registros de racha y sincroniza la tabla *Estadistica* global con el desglose histórico de la tabla *Racha*.
- **Inertia.js Props:** El estado actual de la racha (*rachaActual* y *rachaMasLarga*) se inyecta como propiedad del usuario autenticado en las vistas de *Dashboard.vue*, *PerfilUsuario.vue* y *Estadisticas.vue*.

Archivos Involucrados en el Requerimiento

- **Frontend**

Dashboard.vue (/resources/js/Pages/) - Muestra el indicador principal de la racha actual ("X días de racha") en la sección de estadísticas rápidas del usuario.

PerfilUsuario.vue y Estadisticas.vue (/resources/js/Pages/) - Exponen la racha más larga histórica y la racha actual del perfil del estudiante.

- **Backend y Servicios**

EstadisticaService.php (/app/Services/) - Contiene la lógica para comprobar pérdidas de racha (*verificarRachaPerdida*) e incrementar el contador de días consecutivos al completar sesiones de 25 minutos (*evaluarRachaAlCompletarSesion*).

- **Modelos y Datos**

Racha.php (/app/Models/) - Modelo Eloquent que mapea a la tabla *Racha* para almacenar el histórico de rachas del usuario.

Estadistica.php (/app/Models/) - Modelo que unifica y acumula los contadores del usuario para búsquedas rápidas.

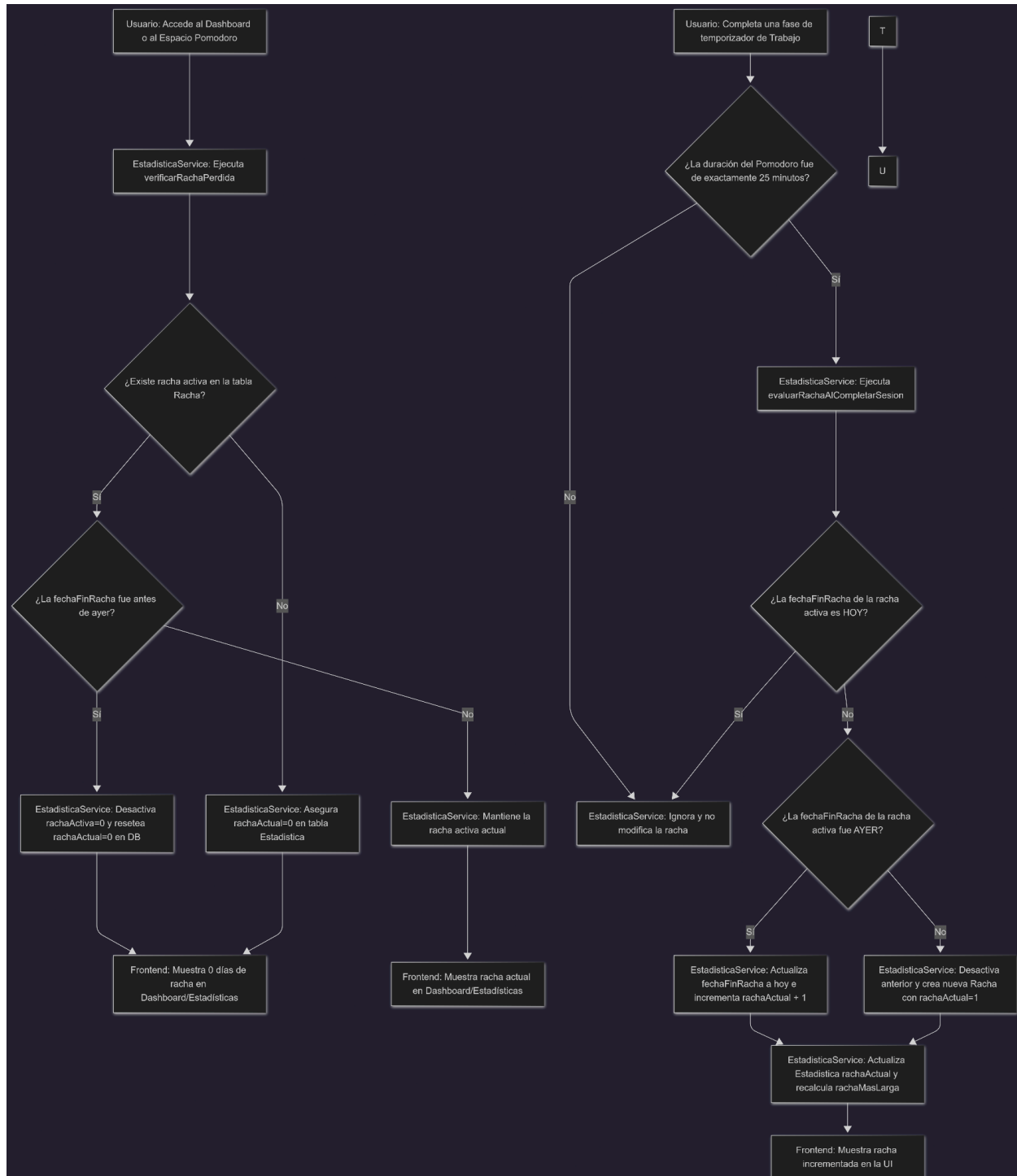
- **Enrutamiento y Base de Datos**

web.php (/routes/) - Pasa por el servicio de racha para resetearla si caducó al cargar el dashboard o la página de perfil.

create_estadistica_racha_table.php (/database/migrations/) - Define las tablas

Racha y Estadística.

Flujo de Datos y Control



1. Comprobación Pasiva de Expiración:

- Cada vez que el usuario carga el Dashboard o su Perfil de Usuario, se ejecuta *verificarRachaPerdida()*.
- Se comprueba la fecha del último pomodoro completado (*fechaFinRacha* de la racha activa). Si esta fecha es anterior al día de ayer (es decir, pasaron 48 horas o más desde la última sesión), se actualiza el estado de la racha a inactiva (*rachaActiva = 0*) y se setea *rachaActual = 0* en la tabla *Estadistica*.

2. Evaluación de Actividad Calificada:

- Cuando el temporizador finaliza y registra una sesión pomodoro completada en el servidor, se dispara *evaluarRachaAlCompletarSesion()*.
- **Filtro de Duración (Regla RF-M12):** Si la sesión duró un tiempo distinto de 25 minutos (ej. 15, 30 o 45 minutos), el flujo termina sin acumular racha.

3. Cálculo de Consecutividad:

- **Caso 1 (Mismo Día):** Si el usuario ya había completado una sesión de 25 minutos hoy (*fechaFinRacha == hoy*), se sale del flujo inmediatamente para no duplicar el conteo.
- **Caso 2 (Día Consecutivo):** Si la última sesión fue ayer (*fechaFinRacha == ayer*), se actualiza la fecha de fin al día de hoy y se incrementa el contador local de la racha en 1.
- **Caso 3 (Racha Nueva/Rota):** Si no hay racha activa o la última actividad ocurrió antes de ayer, se cierra cualquier racha anterior, y se inserta un nuevo registro en la tabla Racha con *fechaInicioRacha = hoy*, *fechaFinRacha = hoy* y *rachaActual = 1*.

4. Sincronización y Réplica:

- El nuevo valor de racha activa calculado se actualiza en el campo *rachaActual* de la tabla *Estadistica*.

- Si la nueva racha supera el récord histórico del usuario (*rachaMasLarga*), se actualiza el campo *rachaMasLarga* con este nuevo valor.

Pruebas y Validación

• **Pruebas de Incremento de Racha**

1. **Precondición:** El usuario debe tener su racha actual en 0 en el Dashboard.
2. **Paso 1:** Iniciar una sesión pomodoro de 25 minutos (vía inicio rápido o personalizado). Esperar o simular que se complete la fase hasta el final (hasta ver el AlertModal).
 - a. **Resultado Esperado:** Al aceptar el modal y recargar el dashboard, la sección de estadísticas debe mostrar "1 días de racha".
3. **Paso 2:** Iniciar otra sesión pomodoro de 25 minutos el mismo día y completarla.
 - a. **Resultado Esperado:** El contador de racha debe mantenerse en "1 días de racha" (no debe incrementarse dos veces el mismo día).
4. **Paso 3:** Iniciar una sesión pomodoro personalizada modificada a 30 minutos de duración. Completar la sesión.
 - a. **Resultado Esperado:** El contador de racha no debe alterarse (debe mantenerse en 1), ya que solo aplican sesiones de exactamente 25 minutos.

• **Pruebas de Expiración y Reseteo**

1. **Precondición:** Tener una racha de 1 o más días con la última fecha de actividad (*fechaFinRacha*) en el día de hoy.
2. **Paso 1:** Modificar en la base de datos la fecha de fin de la racha activa (*fechaFinRacha*) al día de ayer. Ingresar a la aplicación.
 - a. **Resultado Esperado:** La racha debe mantenerse activa y mostrar su contador normal, ya que la última actividad fue "ayer" y el usuario tiene tiempo de mantenerla completando una sesión hoy.

3. **Paso 2:** Modificar en la base de datos la fecha de fin de la racha activa (*fechaFinRacha*) a una fecha de hace dos días. Cargar el Dashboard.
 - a. **Resultado Esperado:** El sistema debe detectar la inactividad, desactivar el registro de la racha en la base de datos (*rachaActiva* = 0) y mostrar "0 días de racha" en la interfaz. El valor de "Racha más larga" en el perfil debe conservar su récord histórico anterior.

RF-06: Sistema de Estadísticas

Descripción y Objetivo

Este requerimiento define el módulo analítico y de autoevaluación de Cronos Notes. Su objetivo es brindar al usuario una visualización clara, interactiva e integrada sobre su rendimiento y hábitos de trabajo diario, permitiéndole identificar patrones de productividad y hacer seguimiento de sus logros.

Se compone de los siguientes sub-requerimientos:

- **RF 6: Sistema de Estadísticas:** Muestra una vista unificada que recopila y calcula métricas clave del comportamiento y dedicación temporal del usuario.
- **RF 6.1: Estadísticas de Usuario:** Expone indicadores de rendimiento numéricos (tareas completadas, tareas completadas con retraso, porcentaje de eficiencia y tiempo acumulado en minutos de concentración) y gráficos interactivos (horas de estudio por día de la última semana y horas de pomodoro dedicadas a cada perfil).

Tecnologías, Herramientas y Librerías

El sistema de estadísticas está diseñado con una arquitectura eficiente que reduce el consumo de librerías externas de renderizado pesado, utilizando SVG nativo y CSS reactivo:

- **Laravel Core & Eloquent ORM:** Consultas relacionales agregadas (conteo, sumatoria y agrupaciones) sobre las tablas *Tarea*, *SesionPomodoro* y *Perfil* para procesar la información del usuario autenticado.
- **Carbon (Date Library):** Utilizado para calcular la ventana temporal de los últimos 7 días en el backend de manera dinámica.
- **Inertia.js & Vue 3 Props:** Traspaso eficiente de arrays de datos precalculados en formato JSON a la vista del cliente.

- **SVG Reactivo y Flexbox (Frontend Render):** El gráfico de dona se genera dinámicamente mediante etiquetas `<circle>` de SVG calculando las proporciones angulares en tiempo real. El gráfico de barras se dibuja usando columnas de HTML y alturas porcentuales basadas en el máximo valor semanal registrado.

Archivos Involucrados en el Requerimiento

- **Frontend**

Estadisticas.vue (/resources/js/Pages/) - Componente principal de la vista analítica. Contiene las tarjetas de métricas, los cálculos reactivos de ejes, las leyendas cromáticas y las estructuras SVG/CSS para pintar los gráficos de dona y barras.

- **Backend y Controladores**

EstadisticaController.php (/app/Http/Controllers/) - Controlador que calcula las tareas creadas, completadas y retrasadas, agrupa los minutos trabajados por perfil y por día para la última semana, y actualiza la concentración diaria.

EstadisticaService.php (/app/Services/) - Servicio que incrementa el total de minutos trabajados globales y suma tareas resueltas a nivel de base de datos.

- **Modelos y Datos**

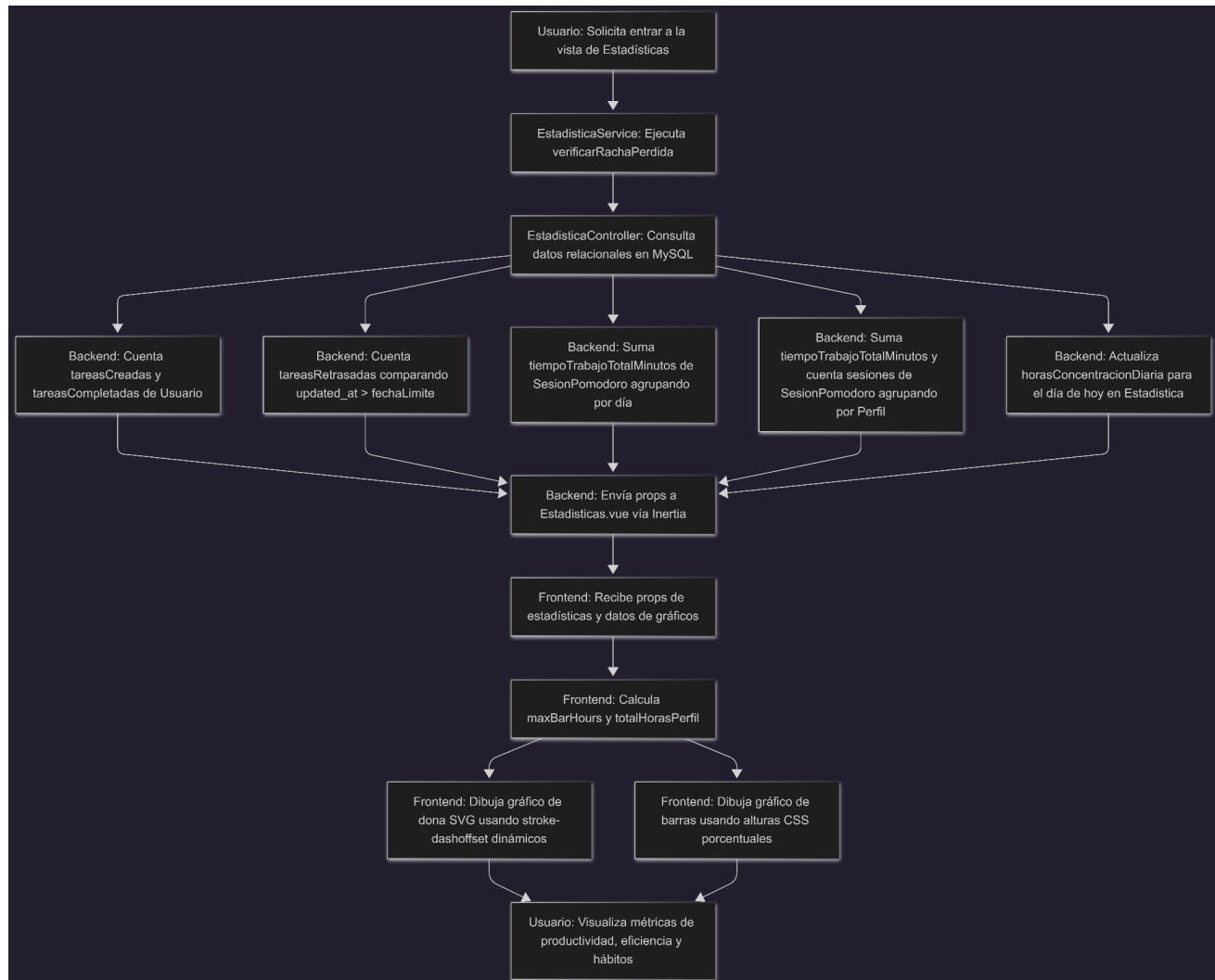
Estadistica.php (/app/Models/) - Mapea la tabla Estadistica donde se acumulan las rachas, el total de minutos trabajados e información consolidada.

- **Enrutamiento y Base de Datos**

web.php (/routes/) - Define la ruta GET /estadisticas protegida por el middleware de autenticación.

create_estadistica_racha_table.php (/database/migrations/) - Migración de estructura para la tabla Estadistica.

Flujo de Datos y Control



1. Petición de Interpretación de Racha

- La petición ingresa por `/estadisticas`. Antes de recopilar métricas, el backend ejecuta `verificarRachaPerdida()` para garantizar que la racha del usuario esté actualizada.

2. Cómputo en Servidor (EstadisticaController)

• Métricas Numéricas:

- tareasCreadas:** Cuenta todas las tareas del usuario.
- tareasCompletadas:** Cuenta tareas en estado 'Completado'.

- **tareasRetrasadas:** Cuenta tareas en estado '*Completado*' cuya fecha de última actualización (*updated_at*) sea posterior a su *fechaLimite*.
 - **eficiencia:** Porcentaje de completadas sobre creadas.
 - **Historial Semanal (Gráfico de Barras):**
 - Itera del día actual hacia atrás 7 veces. Para cada fecha, calcula la sumatoria de minutos trabajados en sesiones activas y divide por 60 para enviarlo en horas.
 - **Distribución por Perfil (Gráfico de Dona):**
 - Itera por cada perfil del usuario. Suma los minutos totales trabajados en sesiones asociadas a tareas de ese perfil y cuenta las sesiones totales.
3. **Cálculo Reactivo en Cliente (Estadísticas.vue)**
- **Columna de Barras:** Obtiene el valor máximo de horas del array de la semana para definir el límite del eje Y (*maxBarHours*). Cada barra calcula su altura relativa con la fórmula $(horas_dia / max_horas) * 100$.
 - **Gráfico de Dona:** Calcula la suma total de horas de todos los perfiles (*totalHorasPerfil*). Cada segmento de círculo calcula su proporción y su posición angular inicial en base a los segmentos acumulados anteriormente.

Pruebas y Validación

- **Pruebas de Métricas y Cálculos**
 1. **Precondición:** El usuario debe tener al menos una tarea creada y completada, y otra tarea completada después de la fecha límite establecida (tarea retrasada).
 2. **Paso 1:** Navegar al apartado "Estadísticas".
 - a. **Resultado Esperado:** Se debe cargar el listado. Comprobar que "Tareas Completadas" sume el valor correcto, "Finalizadas con Retraso" registre las

tareas que excedieron la fecha de vencimiento y el indicador de "Eficiencia" muestre la relación porcentual correcta.

- **Pruebas de Visualización de Gráficos**

1. **Paso 1:** Completar un pomodoro de 25 minutos hoy. Ir a "Estadísticas".

2. **Resultado Esperado:**

- a. En el gráfico "Horas de estudio por día", la barra correspondiente a la fecha de hoy debe mostrar un incremento proporcional a 0.42 horas (25 minutos). El eje Y debe recalcular sus etiquetas de escala de acuerdo con el nuevo valor máximo.
- b. Al pasar el cursor (hover) por encima de la barra, debe aparecer un tooltip con la fecha y las horas exactas.

3. **Paso 2:** Ir a "Estadísticas" habiendo completado pomodoros únicamente en el perfil "Estudio".

- a. **Resultado Esperado:** El gráfico "Horas de pomodoro por perfil" (dona) debe estar compuesto al 100% por el color representativo del perfil "Estudio" y la leyenda debe indicar la sumatoria de horas y sesiones registradas para ese perfil.

RF-M01: Alternancia a Modo Zen

Descripción y Objetivo

Este requerimiento define la capacidad del usuario para alternar a un entorno visual libre de distracciones visuales (Modo Zen) mediante un botón manual en la interfaz del widget de temporizador.

- **Alternancia Manual:** Permite ocultar los paneles laterales (sidebar) y barra superior (topbar) de navegación, expandiendo el área del espacio de concentración para ocupar el 100% de la pantalla.
- **Objetivo:** Ayudar al usuario a evitar interrupciones visuales y carga cognitiva innecesaria durante periodos de estudio o trabajo activo, manteniendo visible únicamente el temporizador y la tarea seleccionada.

Tecnologías, Herramientas y Librerías

Este requerimiento se implementa a nivel de interfaz de usuario mediante control reactivo y clases CSS globales:

- **Vue 3 (Composition API):** Reactividad mediante *isDistractionFree* ref para controlar el estado y actualizar dinámicamente las clases del contenedor.
- **Manipulación de Clases de Body (DOM):** Uso de `document.body.classList.add('distraction-free-mode')` y `document.body.classList.remove('distraction-free-mode')` para ocultar elementos estructurales fuera de la vista Vue principal.
- **Directivas Personalizadas (vDraggable):** Para controlar la interacción física y el movimiento del widget de temporizador, permitiendo moverlo libremente tanto en pantalla completa como en modo minimizado.

Archivos Involucrados en el Requerimiento

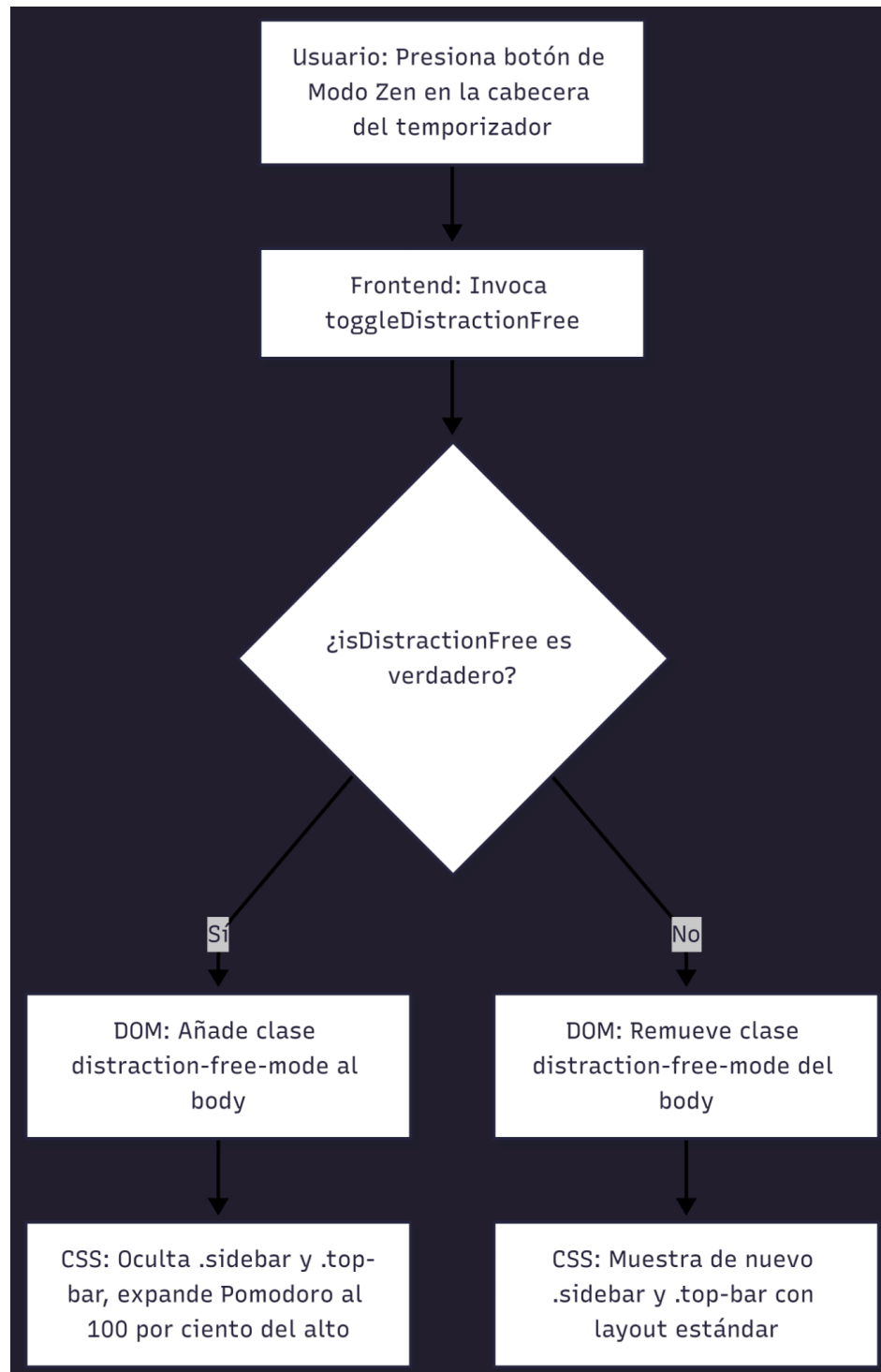
- **Frontend (Vue 3)**

SesionZen.vue (/resources/js/Pages/pomodoro/) - Gestiona la cabecera del widget del temporizador donde se incluye el botón de ojo (``bi-eye-fill`` / ``bi-eye-slash-fill``), el estado de pantalla completa, y los estilos que modifican el layout a modo zen.

AppLayout.vue (/resources/js/Layouts/) - Contiene la estructura general con el menú de navegación lateral (``.sidebar``) y superior (``.top-bar``) que se ocultan ante la presencia de la clase ``distraction-free-mode`` en el ``body``.

- **Directivas de Interacción**

vDraggable.js (/resources/js/Directives/) - Controla el comportamiento de arrastre libre del widget principal del temporizador.

Flujo de Datos y Control

1. **Frontend:** El usuario hace clic en el botón con forma de ojo en la cabecera del temporizador Pomodoro.
2. **Método Controlador:** Se ejecuta `toggleDistractionFree()`, el cual invierte el valor de `isDistractionFree`.
3. **Manipulación del DOM:**
 - Si `isDistractionFree` es `true`, se inserta la clase `.distraction-free-mode` en la etiqueta `<body>` del documento.
 - Si es `false`, se remueve la clase.
4. **Hojas de Estilo (CSS):** El navegador interpreta la regla global `body.distraction-free-mode .sidebar { display: none !important; }` y `.top-bar { display: none !important; }` ocultando de inmediato las secciones y expandiendo el contenedor principal al 100% de la ventana.

Pruebas y Validación

1. **Precondición:** Iniciar sesión e ingresar a la sección de Pomodoro (/pomodoro).
2. **Paso 1:** Con el temporizador maximizado, presionar el botón de ojo (Modo Zen) al lado de minimizar.
 - a. **Resultado Esperado:** La barra de navegación lateral y la barra superior deben desaparecer instantáneamente. La pantalla del espacio Pomodoro debe abarcar todo el espacio del navegador.
3. **Paso 2:** Hacer clic en la pantalla vacía de fondo (fuera de los widgets) o pulsar nuevamente el botón del ojo.
 - a. **Resultado Esperado:** El layout normal de la aplicación debe restablecerse al instante de forma suave y sin desajustes de los widgets.

RF-M02: Ejecución de Inicio Rápido

Descripción y Objetivo

Este requerimiento provee un mecanismo para iniciar una sesión de temporizador Pomodoro de forma inmediata con un solo clic.

- **Evitar Configuración Mandatoria:** El usuario puede presionar el botón de "Inicio Rápido" en el configurador para saltarse la selección de perfiles de trabajo, especificación de tareas activas o configuraciones de tiempo personalizadas.
- **Objetivo:** Facilitar una vía de acceso directa y sin fricción al temporizador de 25 minutos de trabajo y 5 minutos de descanso, aumentando la retención inicial y permitiendo al usuario enfocarse de inmediato ante impulsos espontáneos de concentración.

Tecnologías, Herramientas y Librerías

Este requerimiento interactúa con la lógica de sesiones e interfaz mediante:

- **Vue 3 (Composition API):** Consumo del método ``iniciarInicioRapido()`` expuesto por el composable de temporizador.
- **Laravel / MySQL:** Registro de la sesión activa en la tabla ``SesionPomodoro`` para usuarios registrados una vez que el temporizador inicia su conteo.

Archivos Involucrados en el Requerimiento

- **Frontend**

SesionZen.vue (/resources/js/Pages/pomodoro/) - Contiene la interfaz del panel de configurador y el botón "Inicio Rápido (25 min)" en la pantalla de Setup.

usePomodoroTimer.js (/resources/js/Composables/) - Composable reactivo que inicializa los valores por defecto (25m de trabajo, 5m de descanso corto, 15m de descanso largo) y coordina el disparo del endpoint en el backend.

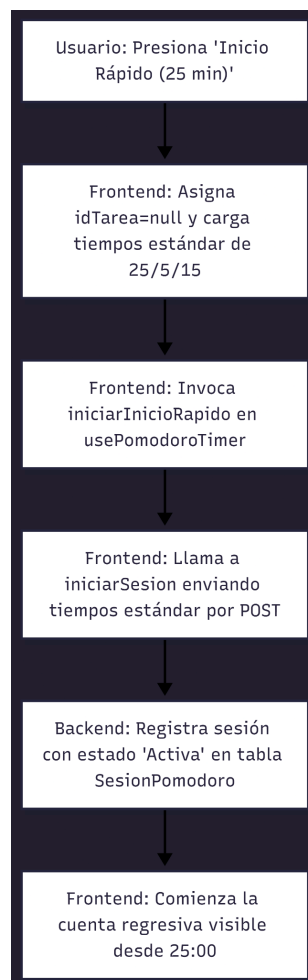
- **Backend & Controladores (Laravel)**

PomodoroController.php (/app/Http/Controllers/) - Recibe las peticiones HTTP para persistir el inicio de la sesión en el backend (`iniciarSesion()`).

- **Modelos y Datos (Eloquent ORM)**

SesionPomodoro.php (/app/Models/) - Modelo relacional que representa el registro físico del bloque de concentración en la base de datos MySQL.

Flujo de Datos y Control



1. **Frontend:** El usuario abre el espacio de Pomodoro en estado de Setup y presiona el botón "Inicio Rápido".

2. **Establecimiento de Parámetros:** El frontend asigna variables nulas para `idTarea` e inicia la fase de trabajo en 1500 segundos (25 minutos).
3. **Petición del Servidor:** El composable realiza una petición HTTP POST a `/pomodoro/iniciar` informando la configuración básica del ciclo.
4. **Backend (Persistencia):** El controlador recibe la petición y, si el usuario está autenticado, persiste una nueva fila en la tabla `SesionPomodoro` con la fecha y hora de inicio actual.
5. **Inicio de Cronómetro:** El frontend recibe la respuesta exitosa y arranca el contador de segundos, refrescando la interfaz del temporizador dinámicamente cada segundo.

Pruebas y Validación

1. **Precondición:** Estar en la pantalla de pomodoro en estado de Setup (sin cronómetro activo en el momento).
2. **Paso 1:** Presionar el botón "Inicio Rápido (25 min)" en el widget.
 - a. **Resultado Esperado:** El widget de Setup debe ocultarse y mostrarse la pantalla activa con la etiqueta "Trabajo" y el temporizador corriendo hacia atrás desde `25:00`.
3. **Paso 2 (Opcional):** Si se está autenticado con una cuenta, ingresar a "Estadísticas" tras completar o pausar esta sesión rápida.
 - a. **Resultado Esperado:** La sesión rápida debe aparecer listada e integrada en el total de horas de concentración o historial de uso.

RF-M03: Renderizado de Calendario Visual

Descripción y Objetivo

Este requerimiento introduce una vista de planificación visual mensual e intuitiva en el sistema.

- **Mapeo de Tareas:** Obtiene todas las tareas creadas por el usuario activo y las ubica en el día correspondiente del calendario según su fecha de límite establecida (``fechaLimite``).
- **Objetivo:** Proveer una herramienta visual de organización semanal y mensual que ayude al usuario a anticipar picos de trabajo y plazos de entrega directamente integrados con sus perfiles de tareas, mejorando la gestión de tiempo de manera global.

Tecnologías, Herramientas y Librerías

El calendario visual combina la recuperación selectiva de registros del servidor con lógica reactiva en el cliente:

- **Laravel (Eloquent ORM):** Recuperación de tareas filtradas por perfil activo utilizando consultas relacionales en la base de datos MySQL.
- **Inertia.js:** Transfiere los datos de la colección de tareas recuperada desde el backend directamente hacia las propiedades (``props``) de la vista Vue en el frontend.
- **Vue 3:** Construcción y renderizado de la grilla de días del mes, posicionamiento reactivo de tareas y filtros dinámicos.

Archivos Involucrados en el Requerimiento

Frontend

Calendario.vue (/resources/js/Pages/) - Página que define el diseño del calendario mensual, lógica de navegación de meses (anterior/siguiente) y renderiza las tarjetas de tareas dentro de los días correspondientes.

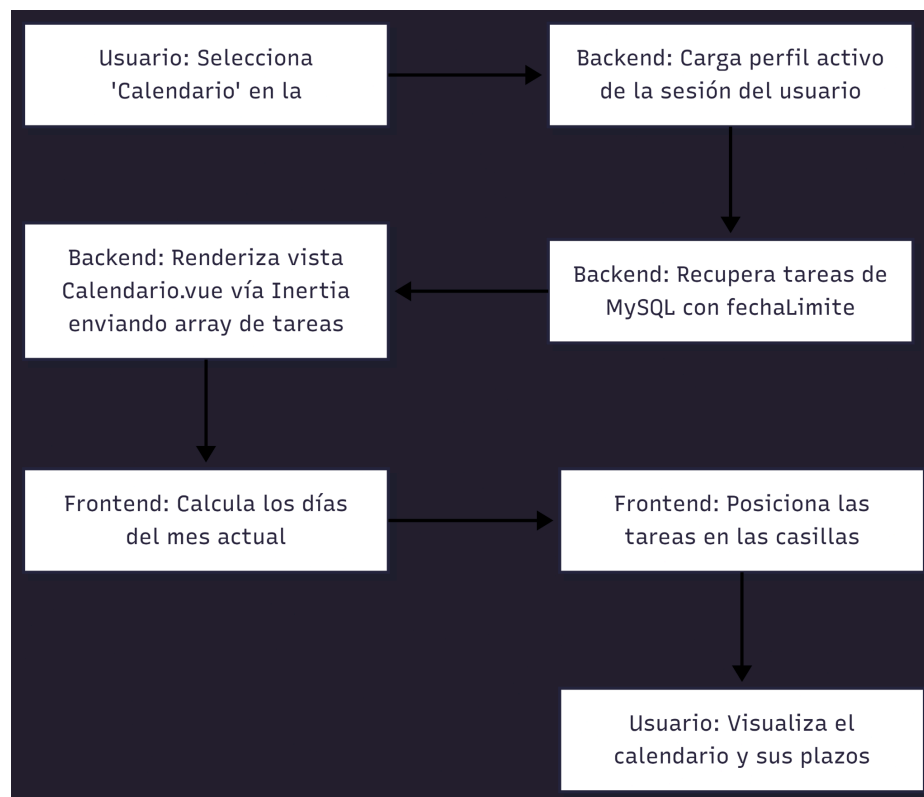
Backend & Controladores

DashboardController.php (/app/Http/Controllers/) - Coordina la recuperación de las tareas y perfiles vinculados para alimentar la visualización inicial de la página.

Modelos y Datos

Tarea.php (/app/Models/) - Contiene los campos `tituloTarea` y `fechaLimite` utilizados para estructurar el calendario.

Flujo de Datos y Control



1. **Frontend:** El usuario solicita ingresar a la pestaña de Calendario.
2. **Backend (Enrutamiento y Carga):** El controlador correspondiente obtiene el ID del perfil activo de la sesión y realiza una consulta Eloquent para buscar tareas no archivadas.
3. **Paso de Datos:** Inertia inyecta el JSON de tareas en las `props` del componente `Calendario.vue`.
4. **Cálculo de Grilla (Frontend):**
 - a. El componente Vue calcula cuántos días tiene el mes actual, el día de la semana en que comienza el mes, y genera las filas de la cuadrícula.
 - b. Itera por cada día del mes, comparando el formato de fecha del día con la propiedad `fechaLimite` de las tareas recibidas.
5. **Renderizado:** Si hay coincidencia de fecha, se inserta una pequeña tarjeta con el título y color según la prioridad de la tarea en la celda del día correspondiente.

Pruebas y Validación

1. **Precondición:** Iniciar sesión y crear una tarea (ej: "Entregar Taller 2") asignándole una fecha límite específica en el mes actual (ej: el día 20).
2. **Paso 1:** Ir a "Calendario" en el menú de navegación izquierdo.
 - a. **Resultado Esperado:** Debe renderizarse la cuadrícula del mes actual de forma responsive.
3. **Paso 2:** Navegar hasta la casilla del día asignado (ej: día 20).
 - a. **Resultado Esperado:** La tarea "Entregar Taller 2" debe mostrarse visible dentro del día correspondiente con el color identificativo de su prioridad.

RF-M04: Procesamiento de IA para Prioridad

Descripción y Objetivo

Este requerimiento define la integración de Inteligencia Artificial para automatizar y mejorar la priorización de tareas pendientes.

- **Priorización Inteligente:** Envía los títulos, descripciones y fechas límite de las tareas del perfil seleccionado a la API de Inteligencia Artificial de Google Gemini.
- **Objetivo:** Proveer al usuario una categorización de prioridades (Alta, Media, Baja) optimizada y justificada por un modelo de lenguaje avanzado, resolviendo problemas de indecisión o sobrecarga y ayudando a estructurar el flujo diario de trabajo.

Tecnologías, Herramientas y Librerías

Este requerimiento requiere una integración segura mediante APIs externas y control de flujo de datos estructurados:

- **Google Gemini API:** Modelo fundacional (`gemini-2.0-flash` o similar) utilizado para realizar el análisis y devolver el orden jerárquico.
- **Laravel Services / HTTP Client:** Llamadas HTTP seguras autenticadas mediante `GEMINI\_API\_KEY` definido en el archivo de entorno `.env` del servidor.
- **Inertia.js & Vue 3:** Botón disparador interactivo en la pantalla de gestión de tareas con estados de carga (spinners) para evitar envíos dobles.
- **Eloquent ORM / MySQL:** Actualización de la columna `prioridadTarea` (`Alta`, `Media`, `Baja`) en la tabla `Tarea`.

Archivos Involucrados en el Requerimiento

Frontend

GestionTareas.vue (/resources/js/Pages/) - Interfaz principal del gestor de tareas que incluye el botón para activar la ordenación por Inteligencia Artificial y el spinner de carga.

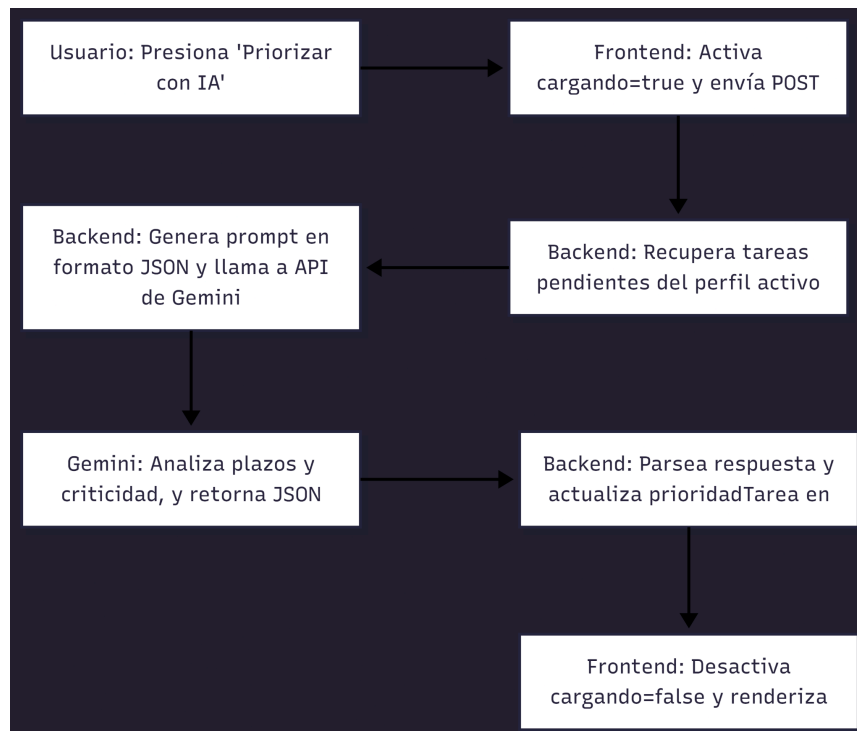
Backend & Controladores

TareaController.php (/app/Http/Controllers/) - Contiene la lógica del endpoint ``priorizarConIA()``. Construye el prompt estructurado, configura los modelos y llama al endpoint de Gemini, parseando la respuesta JSON devuelta.

Modelos y Datos (Eloquent ORM)

Tarea.php (/app/Models/) - Representa la entidad Tarea con los atributos actualizables por la IA.

Flujo de Datos y Control



1. **Frontend:** El usuario inicia la priorización haciendo clic en el botón correspondiente.
2. **Backend (Preparación del Prompt):** El controlador lee las tareas asociadas al perfil y genera una cadena descriptiva con el formato:
 - a. ``[ID: 1, Titulo: "Examen de Software", Limite: "2026-06-15", Descripción: "Preparar unidad 3"]``
3. **Petición HTTP a la API de Gemini:**
 - a. Envía un request POST al endpoint de Gemini incluyendo la clave de desarrollo y el prompt.
 - b. Solicita que la respuesta sea devuelta estrictamente en formato JSON válido con los IDs clasificados en categorías de prioridad.
4. **Persistencia en Base de Datos:** El backend itera sobre el resultado de la IA y actualiza la columna ``prioridadTarea`` de cada registro respectivo en la tabla ``Tarea``.
5. **UI Update:** Inertia devuelve la redirección de éxito y el frontend muestra la lista ordenada visualmente por criticidad mediante etiquetas de colores.

Pruebas y Validación

1. **Precondición:** Tener al menos 3 tareas creadas en el perfil activo con diferentes fechas de vencimiento (por ejemplo: una mañana, otra en 2 semanas, otra sin fecha).
2. **Paso 1:** Ir a la vista de "Tareas" y hacer clic en el botón "Priorizar con IA".
 - a. **Resultado Esperado:** Debe visualizarse un indicador de carga ("Pensando...") y las interacciones del panel de tareas deben bloquearse temporalmente.
 - b. **Resultado Esperado:** Tras unos segundos, el indicador desaparece y las tareas cambian su etiqueta de prioridad. La tarea con vencimiento más cercano debe estar categorizada como prioridad "Alta".

3. **Validación en Base de Datos:** Al revisar el registro de la base de datos MySQL, el campo `prioridadTarea` de las filas correspondientes debe coincidir con el estado de la interfaz.

RF-M05: Editor de Apuntes en Tiempo Real

Descripción y Objetivo

Este requerimiento provee un entorno avanzado para la toma de apuntes o notas de estudio asociadas a perfiles.

- **Editor Enriquecido y Método Cornell:** Ofrece herramientas para formatear texto (negritas, cursivas, listas) y la opción de cambiar al Método Cornell, que divide la nota en secciones de "Ideas", "Notas principales" y "Resumen".
- **Grabadora de Audio:** Permite al usuario capturar audio del micrófono en tiempo real directamente en la interfaz del editor, vinculando grabaciones (audios de clase) al apunte activo.
- **Objetivo:** Integrar la toma de notas escritas con la captura de sonido ambiental en una sola pantalla, eliminando la necesidad de cambiar de dispositivo o aplicación durante el estudio.

Tecnologías, Herramientas y Librerías

El editor enriquecido y la grabadora de voz combinan APIs nativas del navegador con persistencia relacional en Laravel:

- **Web Audio / MediaRecorder API:** API nativa de JavaScript utilizada para acceder de forma segura al hardware del micrófono, capturar el flujo de audio e iniciar/detener la codificación en archivos comprimidos (`.wav` o `.mp3`).

- **Inertia useForm & FormData:** Serializa los apuntes de texto y adjunta el archivo binario del audio grabado en una petición multipart/form-data.
- **Laravel Storage:** Para almacenar físicamente las grabaciones de audio en el disco del servidor (`storage/app/public/audios/`).
- **MySQL / Eloquent ORM:** Tablas `Apunte` y `ApunteAudio` para relacionar los apuntes textuales con sus grabaciones multimedia asociadas.

Archivos Involucrados en el Requerimiento

Frontend

Editor.vue (/resources/js/Pages/apunte/) - Vista del formulario de toma de apuntes, enlazado de texto y estructura Cornell.

AudioPanel.vue (/resources/js/Pages/apunte/components/) - Componente encargado de manejar la interacción con el micrófono mediante `MediaRecorder`, gestionar estados de grabación y reproducir el audio guardado.

EditorToolbar.vue (/resources/js/Pages/apunte/components/) - Barra de herramientas de formateo de texto del editor.

NoteEditor.vue (/resources/js/Pages/apunte/components/) - Área editable del apunte con soporte Cornell o estándar.

Backend & Controladores

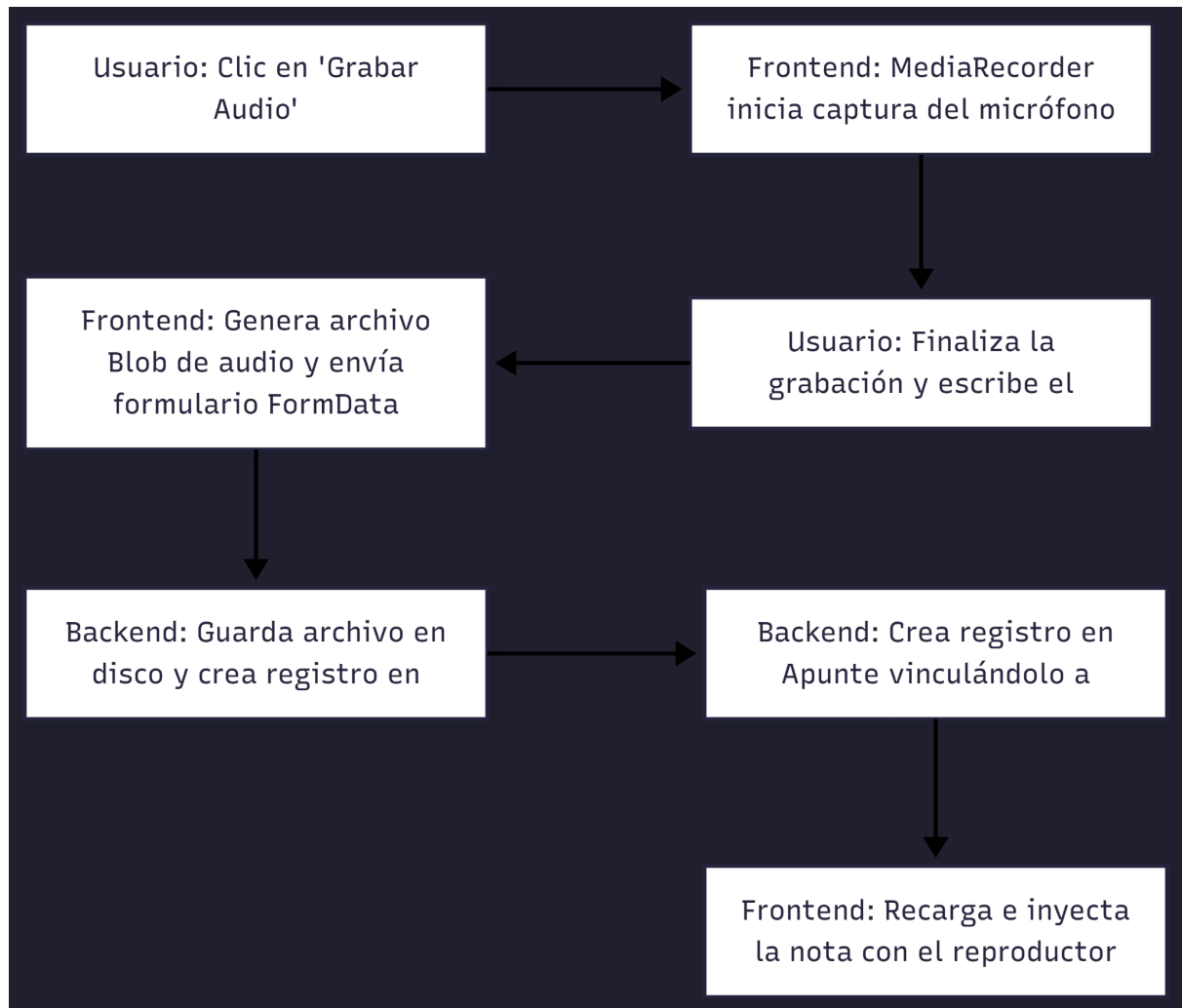
ApunteController.php (/app/Http/Controllers/) - Recibe las peticiones HTTP, almacena el audio físico en disco y persiste las referencias en base de datos.

Modelos y Datos (Eloquent ORM)

Apunte.php (/app/Models/) - Modelo de Eloquent para las notas.

ApunteAudio.php (/app/Models/) - Asocia los archivos de audio físicos con el apunte correspondiente.

Flujo de Datos y Control



- 1. Frontend (Grabación):** El usuario pulsa en "Grabar Audio". La aplicación solicita permiso de micrófono y, tras concederse, inicializa la clase `MediaRecorder` recolectando porciones de audio en un array de datos.
- 2. Finalización:** Al detener la grabación, se compilan los datos en un objeto `Blob` y se crea una URL temporal local para permitir la preescucha antes de guardar.
- 3. Envío de Datos (POST):** El usuario pulsa "Guardar Apunte". Inertia envía un `FormData` conteniendo los campos de texto, formato seleccionado (Cornell/estándar), y el archivo binario del audio.

4. Backend (Persistencia):

- a. El controlador `ApunteController` valida que el archivo de audio sea válido.
 - b. Guarda el audio en `public/storage/audios/` asignándole un hash único.
 - c. Crea un registro en la tabla `apunte\_audios` (Modelo `ApunteAudio`).
 - d. Crea un registro en la tabla `apunte` (Modelo `Apunte`), asociando la nota al perfil activo y al ID del audio guardado.
5. **UI Update:** Inertia redirige de vuelta al listado, renderizando el reproductor ``<audio>`` nativo si el apunte tiene una grabación asociada.

Pruebas y Validación

1. **Precondición:** Tener un perfil activo e ingresar a la pestaña de "Apuntes".
2. **Paso 1:** Pulsar en "Nuevo Apunte". Escribir un título e ideas en los campos correspondientes.
3. **Paso 2:** Presionar el botón del micrófono "Grabar", hablar por 5 segundos y presionar "Detener". Escuchar la grabación previa.
4. **Paso 3:** Pulsar en "Guardar Apunte" y esperar el redireccionamiento.
 - a. **Resultado Esperado:** El apunte se crea y aparece en la lista. Al reabrirlo, debe mostrarse el texto formateado tal como se escribió, y debe aparecer un reproductor de audio integrado que permita escuchar la grabación de 5 segundos realizada.

RF-M06: Mezclador de Sonidos

Descripción y Objetivo

Este requerimiento provee un ambiente auditivo personalizable para potenciar la concentración y el aislamiento acústico durante las sesiones de estudio.

- **Reproducción Simultánea y Mezcla:** El usuario puede activar una o más pistas de sonido ambiental (lluvia, tormenta, fogata, viento, ruido blanco, cafetería, etc.) al mismo tiempo y ajustar el volumen de cada pista de forma independiente a través de un panel de controles deslizantes (sliders).
- **Objetivo:** Ayudar al usuario a generar un espacio inmersivo y libre de ruidos externos distractores que ayude a mitigar el estrés académico o laboral.

Tecnologías, Herramientas y Librerías

El mezclador de sonidos aprovecha una combinación de recursos de audio locales y una librería de audio robusta:

- **Howler.js:** Librería JavaScript de control de audio en 3D y multicanal, utilizada para cargar las pistas, reproducirlas concurrentemente en bucle (``loop: true``), y cambiar el volumen dinámicamente sin interrumpir la reproducción.
- **Pistas Locales (Directorio public):** Archivos de sonido comprimidos en formatos de alta calidad ``mp3`` y ``webm`` alojados directamente en el servidor.
- **Vue 3 (Composition API):** Estructuración reactiva del mezclador mediante ``mixerState`` y mapeo de emojis dinámicos según el tipo de sonido.
- **Resolución de Rutas Relativas (XAMPP):** Método ``getAssetUrl`` para garantizar que las pistas de sonido se ubiquen correctamente tanto en servidores locales de subdirectorio (ej. ``http://localhost/public/``) como en entornos virtuales de dominio raíz.

Archivos Involucrados en el Requerimiento

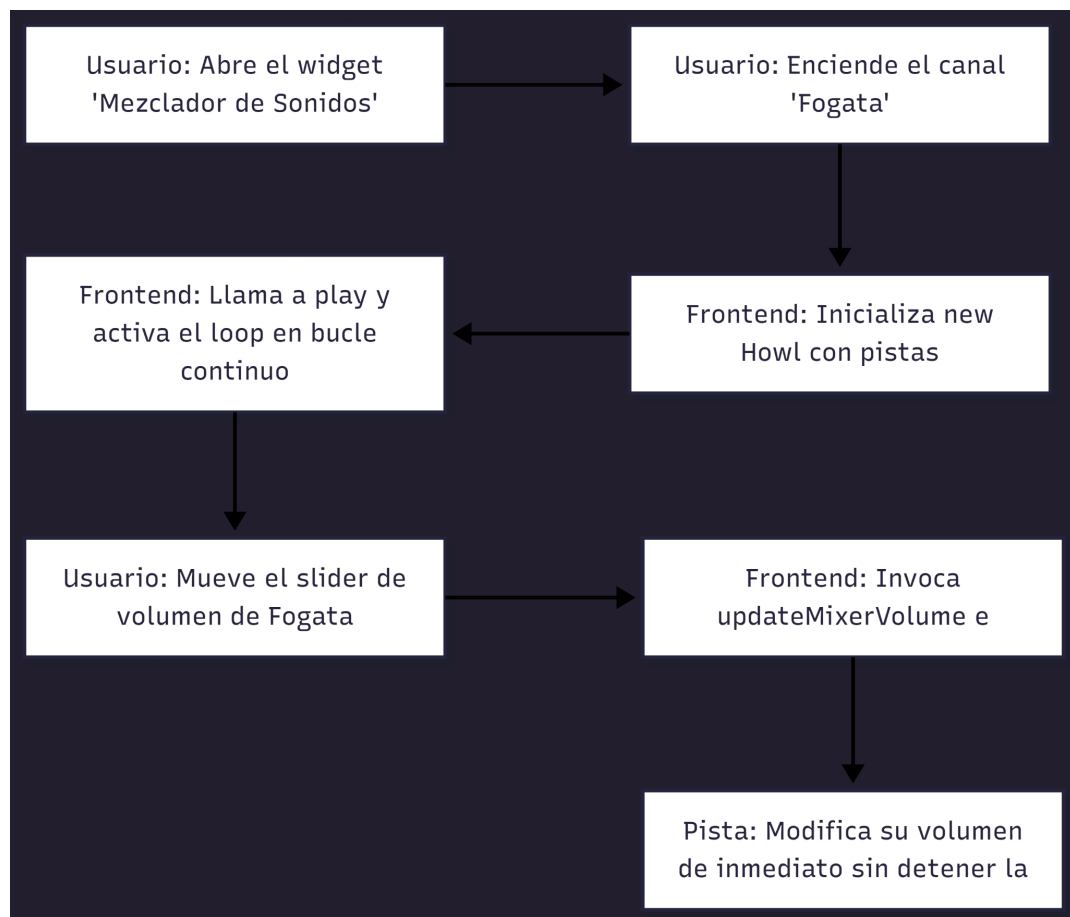
Frontend (Vue 3)

SesionZen.vue (/resources/js/Pages/pomodoro/) - Administra la lógica del mezclador, controla la instanciación de objetos `Howl`, maneja los sliders de volumen individuales y los estados activos.

Recursos Estáticos (Audios)

public/audios/ - Directorio que aloja las pistas de audio locales comprimidas (ej. `storm.mp3`, `campfire.webm`, `cafe.mp3`, etc.).

Flujo de Datos y Control



1. **Frontend:** El usuario abre el Mezclador de Sonidos y activa un canal de audio (ej: Fogata).
2. **Carga y Reproducción:**
 - a. Si no existe una instancia de ``Howl`` para ese sonido en el array de instancias (``howlerInstances``), se instancia una nueva.
 - b. La propiedad ``src`` de la instancia carga las fuentes ``webm`` y ``mp3`` resolviendo las rutas a través del helper ``getAssetUrl`` para evitar errores 404 de servidor.
 - c. Se configuran los parámetros ``loop: true`` y ``html5: true``.
 - d. Se inicia la reproducción ejecutando el método ``play()``.
3. **Control de Volumen:** Al desplazar el slider de volumen, el evento de input dispara ``updateMixerVolume(soundKey)``. Este método ajusta el volumen directamente en la instancia de Howler ejecutando ``howlerInstances[soundKey].volume(state.volume)``.
4. **Desactivación:** Si el usuario apaga el canal de audio, se invoca ``pause()`` en la instancia respectiva, deteniendo la salida de sonido pero conservando la posición de reproducción.
5. **Cierre:** Al abandonar la pantalla de pomodoro, la directiva ``onUnmounted`` limpia todas las instancias llamando a ``stop()`` y ``unload()`` para evitar fugas de sonido en segundo plano.

Pruebas y Validación

1. **Precondición:** Estar en la pantalla de pomodoro (/pomodoro).
2. **Paso 1:** Desplegar el widget "Mezclador de Sonidos" (ubicado en el extremo superior izquierdo).
3. **Paso 2:** Encender la pista "Tormenta" pulsando sobre su botón o icono.
 - a. **Resultado Esperado:** Debe reproducirse el sonido de lluvia y truenos en bucle.
4. **Paso 3:** Encender la pista "Fogata" y mover su slider a la mitad de volumen.

- a. **Resultado Esperado:** Ambos sonidos deben escucharse de forma simultánea, y el crujido de la fogata debe ser más tenue que el sonido de la tormenta.
- 5. **Paso 4:** Salir del módulo de pomodoro y navegar hacia otra sección (ej. Tareas).
 - a. **Resultado Esperado:** Todo sonido del mezclador debe silenciarse por completo de forma inmediata.

RF-M07: Sesión de Invitado

Descripción y Objetivo

Este requerimiento define la capacidad del sistema para ser probado por usuarios anónimos (visitantes) sin forzarlos a registrarse.

- **Acceso Directo (Modo Invitado):** Permite ingresar al espacio de Pomodoro con el temporizador y el mezclador de sonidos ambientales habilitados de forma inmediata.
- **Fidelización y Barrera de Entrada:** Bloquea las funcionalidades persistentes (guardar plantillas de tiempo, asociar tareas, registrar estadísticas, etc.) y muestra un modal promocional invitando a registrarse de forma gratuita cuando el visitante intenta utilizarlas.
- **Objetivo:** Atraer usuarios nuevos permitiéndoles probar el valor estético y la utilidad del espacio de trabajo de manera inmediata antes de solicitar sus datos personales.

Tecnologías, Herramientas y Librerías

La sesión de invitado utiliza rutas públicas y condicionales lógicos en el frontend:

- **Laravel Routing:** Ruta pública `/invitado` excluida del middleware `auth`.
- **Inertia.js:** Paso del flag booleano `isGuest => true` en las propiedades de la vista al renderizar el componente.
- **Vue 3:**

- Evaluación lógica condicional (`v-if="isGuest"`) en plantillas para ocultar la barra lateral de navegación (AppLayout) y el selector de tareas del Pomodoro.
- Teleport del modal de promoción de registro para incentivar la conversión a usuario formal.

Archivos Involucrados en el Requerimiento

Frontend

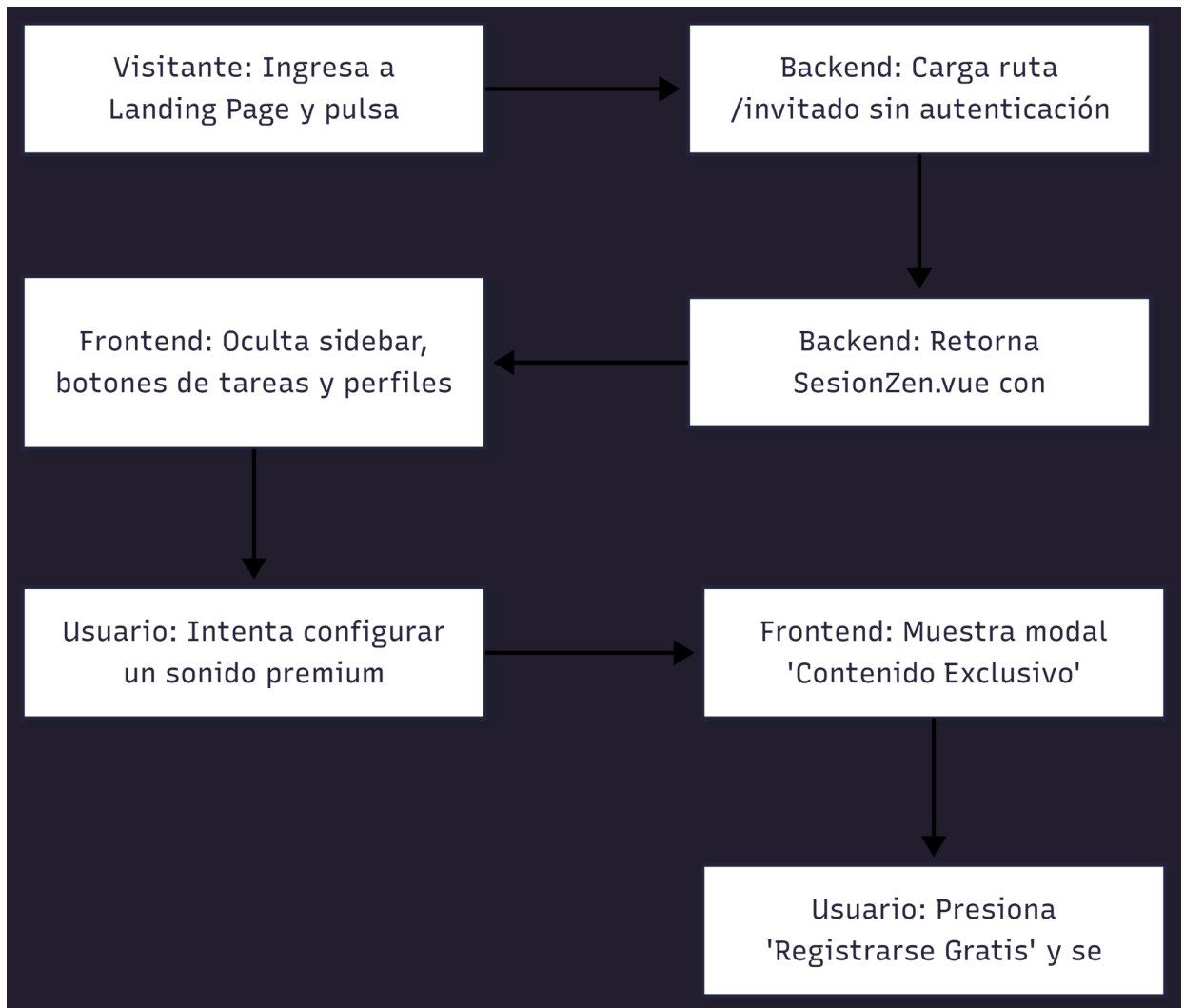
SesionZen.vue (/resources/js/Pages/pomodoro/) - Oculta la selección de tareas en el configurador, bloquea los paisajes y sonidos premium para visitantes, y abre el modal promocional de registro.

Welcome.vue (/resources/js/Pages/) - Landing page que provee el botón de acceso directo "Probar como Invitado".

Backend & Controladores (Laravel)

web.php (/routes/) - Ruta de acceso pública a `/invitado` asociada al controlador.

PomodoroController.php (/app/Http/Controllers/) - Método `invitado()` que carga el componente `SesionZen` pasando `isGuest` como `true`.

Flujo de Datos y Control

1. **Frontend:** Un usuario no autenticado ingresa a la aplicación y pulsa en "Probar como Invitado".
2. **Backend (Enrutamiento):** La ruta web mapea ``/invitado`` y llama al controlador ``PomodoroController@invitado``. Este omite el filtro de autenticación y renderiza la vista ``Pomodoro/SesionZen`` adjuntando la propiedad ``isGuest => true``.

3. Restricción de Layout (Frontend):

- a. El componente Vue principal evalúa `<component :is="isGuest ? 'div' : AppLayout">`.
 - b. Si `isGuest` es `true`, el componente principal se renderiza dentro de un contenedor `div` estándar en lugar de `AppLayout`, eliminando el menú de navegación lateral.
 - c. En el configurador de Pomodoro, se oculta el menú desplegable de tareas.
4. **Validación de Bloqueos:** Si el invitado intenta interactuar con un fondo de pantalla o sonido restringido (ej: paisaje premium), la función `isLandscapeLocked(key)` o `isSoundLocked(soundKey)` retorna `true`.
5. **Modal de Conversión:** Se abre el modal `modalRegistroOpen = true` mostrando el mensaje correspondiente para redirigir al visitante al formulario de registro formal en `/register`.

Pruebas y Validación

1. **Precondición:** Cerrar sesión en la aplicación.
2. **Paso 1:** Navegar a la URL del espacio invitado `/invitado`.
 - a. **Resultado Esperado 1:** La página debe cargar el entorno Pomodoro. La barra lateral de navegación no debe estar visible.
3. **Paso 2:** Abrir el panel de configuración de la sesión del temporizador.
 - a. **Resultado Esperado 2:** El selector de tareas no debe mostrarse (los invitados no tienen tareas registradas).
4. **Paso 3:** Ir a la Configuración Avanzada, seleccionar un paisaje premium (ej. Paisaje 5) e intentar aplicarlo.

- a. Resultado Esperado 3:** Debe mostrarse un modal con el título "Contenido Exclusivo" que explique que la opción solo está disponible para registrados y provea un botón para ir al registro.

RF-M08: Conexión con APIs Externas

Descripción y Objetivo

Este requerimiento define la integración del sistema Cronos-Note con plataformas externas mediante protocolos de autorización seguros (OAuth 2.0). El objetivo principal es permitir la interoperabilidad con servicios externos para enriquecer la experiencia de usuario:

- **Autenticación con Google:** Permitir el registro e inicio de sesión rápido en Cronos-Note mediante una cuenta de Google existente.

Tecnologías, Herramientas y Librerías

Para esta integración se emplean protocolos estándar de seguridad y librerías del ecosistema de Laravel:

- **Laravel Socialite:** Paquete oficial de Laravel que simplifica la integración y el flujo OAuth 2.0 con diversos proveedores (Google, Spotify, etc.).
- **Google API Client (OAuth 2.0):** Proveedor utilizado para autenticar al usuario y solicitar los scopes necesarios para Google Calendar (`.../auth/calendar.readonly`).
- **Tabla `IntegracionExterna`:** Tabla del sistema encargada de persistir de forma segura las credenciales de conexión del usuario (identificador de la plataforma, token de acceso y token de actualización/refresh token).

Archivos Involucrados en el Requerimiento

Frontend

Login.vue (/resources/js/Pages/auth/) - Contiene el botón de "Continuar con Google".

Register.vue (/resources/js/Pages/auth/) - Contiene el botón de "Registrarse con Google".

SesionZen.vue (/resources/js/Pages/pomodoro/) - Panel de control e interfaz de reproducción para Spotify (reproductor integrado).

Backend & Controladores (Laravel)

GoogleAuthController.php (/app/Http/Controllers/Auth/) - Administra la redirección y el retorno (callback) del flujo de autenticación de Google.

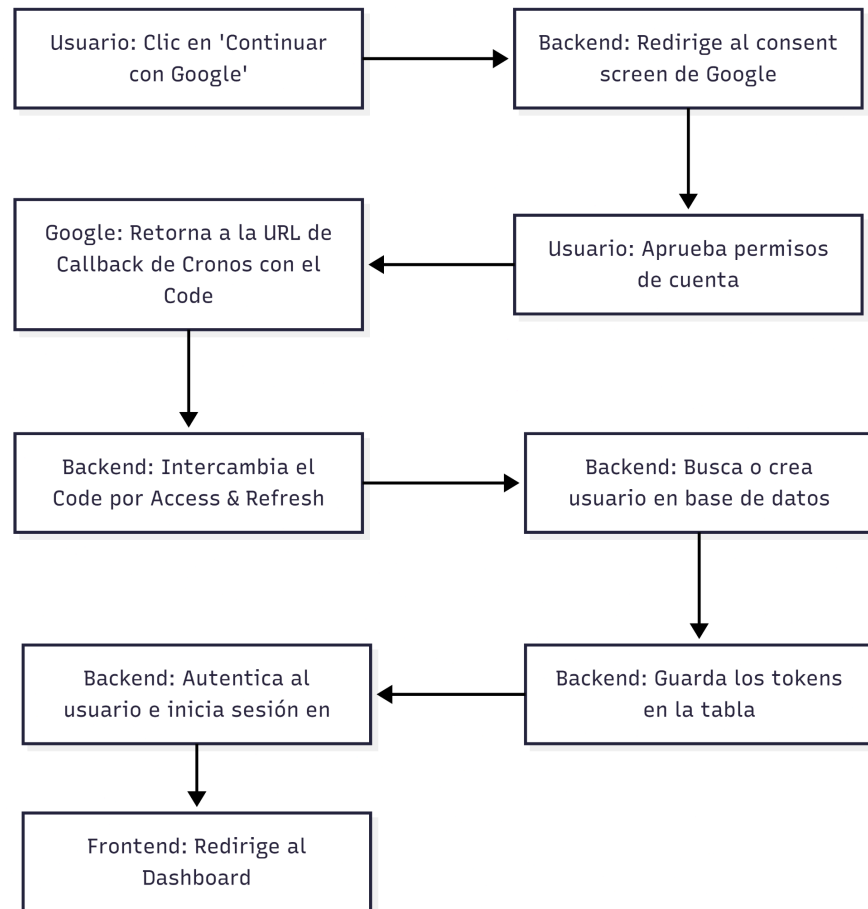
auth.php (/routes/) - Define las rutas de redirección (`auth/google`) y de callback (`auth/google/callback`).

Modelos y Datos (Eloquent ORM)

IntegracionExterna.php (/app/Models/) - Modelo de Eloquent que representa la tabla `IntegracionExterna` para el guardado de tokens.

User.php (/app/Models/) - Define la relación `hasMany` con `IntegracionExterna`.

Flujo de Datos y Control



1. **Petición de Autorización:** El usuario selecciona la acción de vincular su cuenta o iniciar sesión. El frontend redirige a la ruta del backend (`auth/google` o similar).
2. **Redirección de Proveedor:** El backend utiliza `Laravel Socialite` para generar la URL de autorización del proveedor externo (con los scopes requeridos) y redirige al usuario.
3. **Aprobación del Usuario:** El usuario autoriza a Cronos-Note en la plataforma externa.
4. **Manejo del Callback:** El proveedor redirige al endpoint de callback de Cronos-Note enviando un parámetro `code`.

5. Obtención y Guardado de Tokens:

- a. El controlador de Laravel captura el callback, intercambia el código por los tokens de acceso y refresco de forma segura.
 - b. Se busca al usuario autenticado o se registra en caso de inicio de sesión por primera vez.
 - c. Se realiza un `updateOrCreate()` en la tabla `IntegracionExterna` guardando el `tokenAcceso` e `identificadorExterno`.
- 6. Consumo de la API:** Posteriormente, el sistema realiza peticiones HTTP autorizadas (con cabecera `Authorization: Bearer [tokenAcceso]`) para leer eventos de Google Calendar o enviar comandos de reproducción a Spotify.

Pruebas y Validación

- 1. Precondición:** No haber iniciado sesión. Tener una cuenta de Google activa.
- 2. Paso 1:** Ir a la página de Login (`/login`) y hacer clic en "Continuar con Google".
- 3. Paso 2:** Seleccionar una cuenta de Google en la pantalla de consentimiento.
- 4. Resultado Esperado:** El sistema debe redirigir al Dashboard de Cronos-Note, iniciar sesión exitosamente y registrar un registro en la tabla `IntegracionExterna` con `plataforma = 'GoogleAuth'`.

RF-M09: Registro de Cancelación de Sesión

Descripción y Objetivo

Este requerimiento define el comportamiento del sistema cuando un usuario detiene activamente un temporizador de concentración (Pomodoro) antes de alcanzar el objetivo de tiempo establecido (cuando el contador no ha llegado a cero). El objetivo es:

- **Persistencia en Estadísticas:** Registrar el evento incrementando la métrica de sesiones canceladas del usuario en la base de datos para mantener un historial fidedigno de rendimiento.
- **Feedback Motivacional:** Mostrar un mensaje de aliento y motivación en la interfaz de usuario en lugar de un aviso genérico de cancelación, promoviendo la resiliencia y el bienestar del usuario frente a la interrupción del flujo de trabajo.

Tecnologías, Herramientas y Librerías

Este requerimiento utiliza la infraestructura estándar de sesiones e interfaz de Cronos-Note:

- **Eloquent ORM / MySQL:** Actualización de la tabla `Estadistica` para incrementar la columna `sesionesCanceladas`.
- **Sesión de Laravel (Flash Messages):** Mecanismo de persistencia temporal en sesión para transferir el mensaje motivacional desde el controlador del backend hasta la vista del frontend durante el redireccionamiento.
- **Inertia.js & Vue 3:** Recepción automática del flash message success en la propiedad `\$page.props.flash.success` para renderizar el componente visual de notificación de manera interactiva.

Archivos Involucrados en el Requerimiento

Frontend

SesionZen.vue (/resources/js/Pages/pomodoro/) - Administra el inicio del flujo de cancelación de sesión y renderiza los modales de confirmación (`showConfirmEndModal`).

AppLayout.vue (/resources/js/Layouts/) - Captura el mensaje flash de éxito e introduce un toast emergente flotante en el lateral derecho superior de la pantalla.

Backend & Controladores

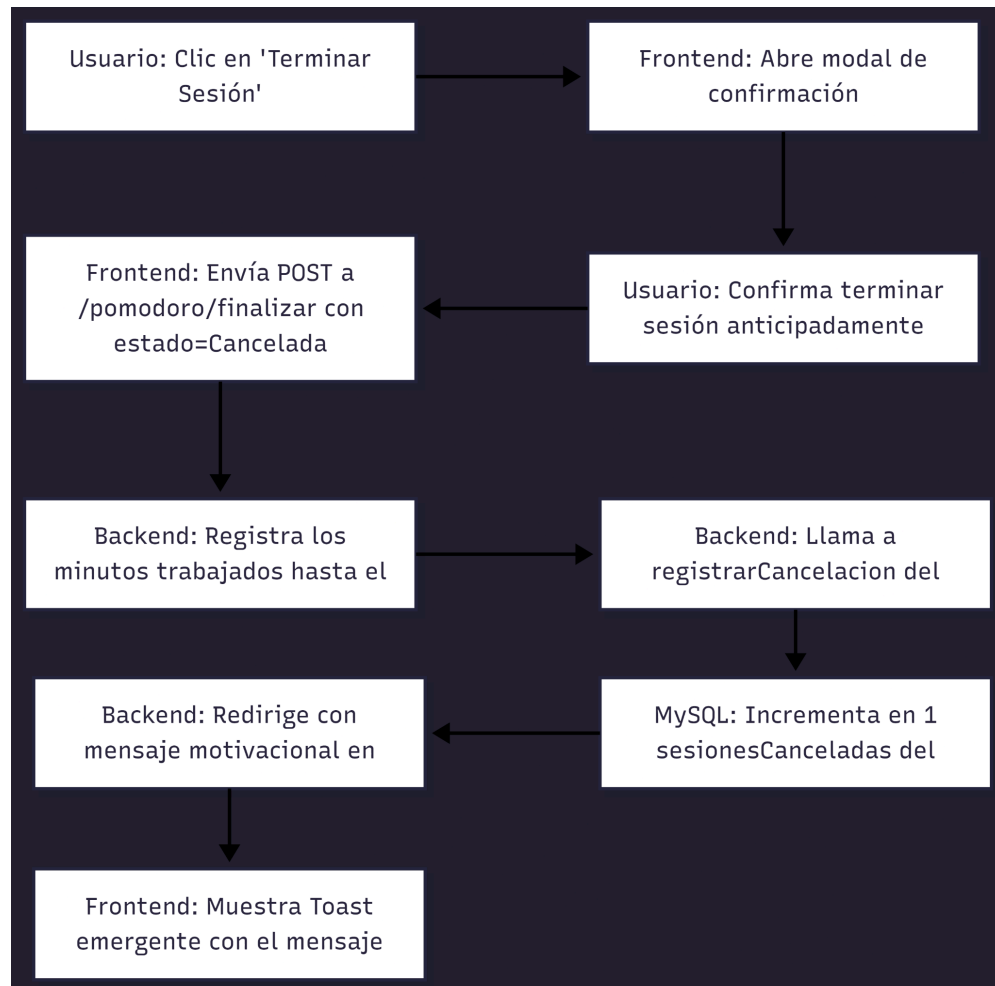
PomodoroController.php (/app/Http/Controllers/) - Modifica la acción `finalizarSesion()`. Gestiona el flujo redirigiendo al usuario y adjuntando el mensaje motivacional.

EstadisticaService.php (/app/Services/) - Proporciona el método `registrarCancelacion()` encargado de persistir el conteo en la base de datos.

Modelos y Datos (Eloquent ORM)

Estadistica.php (/app/Models/) - Modelo Eloquent con la columna `sesionesCanceladas`.

Flujo de Datos y Control



1. **Frontend:** El usuario decide terminar su sesión antes de tiempo. Se detiene el temporizador en el cliente y se solicita confirmación.
2. **Controlador:** Recibe los datos de finalización en `finalizarSesion()`.
3. **Persistencia:**
 - a. Si se han acumulado minutos de concentración en el ciclo activo, se incrementa el tiempo de trabajo acumulado y se actualizan los gráficos de estadísticas semanales.

- b.** Al detectarse un estado ``Cancelada``, se invoca
``$estadisticaService->registrarCancelacion(Auth::id())``.
 - c.** La base de datos incrementa el contador `sesionesCanceladas` en la fila correspondiente al usuario en la tabla `Estadistica`.
- 4. Respuesta y Redirección:** El controlador olvida la sesión activa en el servidor y redirige a la ruta principal de pomodoro agregando un mensaje de aliento motivacional al flash payload:
 - a.** "Sesión cancelada. ¡No te preocupes! Cada pequeño paso cuenta, vuelve a intentarlo cuando estés listo. 💪"
- 5. UI Notification:** `AppLayout.vue` detecta el flash message y renderiza un toast con el mensaje motivacional.

Pruebas y Validación

- 1. Precondición:** Iniciar sesión con un usuario y abrir el panel de concentración Pomodoro (`/pomodoro`).
- 2. Paso 1:** Iniciar una nueva sesión de Pomodoro rápida o personalizada.
- 3. Paso 2:** Esperar a que el temporizador comience a correr (ej. transcurridos algunos segundos o minutos) y hacer clic en el botón rojo "Terminar Sesión".
- 4. Paso 3:** Confirmar la acción en el cuadro de diálogo de advertencia.
 - a. Resultado Esperado:** - El sistema debe redirigir a la pantalla de configuración del Pomodoro. - Debe aparecer un toast verde con el mensaje motivacional de aliento. - En la base de datos (o la página de estadísticas), el número de sesiones canceladas del usuario debe haberse incrementado en exactamente 1.

RF-M10: Estimación de Esfuerzo

Descripción y Objetivo

Este requerimiento define la capacidad del usuario para planificar y predecir la cantidad de esfuerzo/tiempo requerida para completar una tarea de manera estructurada:

- **Campo de Estimación:** Se provee un campo numérico positivo en los formularios de creación y edición de tareas para ingresar la cantidad estimada de ciclos Pomodoro (bloques de 25 minutos) requeridos.
- **Objetivo:** Ayudar al usuario a planificar sus actividades diarias cuantificando las tareas en unidades Pomodoro. Esto se vincula directamente con la visualización del progreso en tiempo real durante la sesión activa de Pomodoro.

Tecnologías, Herramientas y Librerías

Este requerimiento aprovecha el modelo relacional estándar y la validación en Laravel:

MySQL / Eloquent ORM: Columna `estimacionEsfuerzo` de tipo entero (`INTEGER`) en la tabla `Tarea`.

Formularios de Inertia (`useForm`): Para el enlazado bidireccional de datos (`v-model`) en Vue y el envío asíncrono con manejo automático de errores de validación.

Validación de Laravel: Regla de validación `estimacionEsfuerzo` => `nullable|integer|min:1` para asegurar que los datos ingresados correspondan a un entero positivo.

Archivos Involucrados en el Requerimiento

- **Frontend**

GestionTareas.vue(/resources/js/Pages/) - Formulario principal de creación y edición de tareas, y renderizado de la estimación en la tarjeta de detalle de tarea.

GestionPerfil.vue (/resources/js/Pages/) - Formulario modal de creación rápida de tareas vinculadas a perfiles.

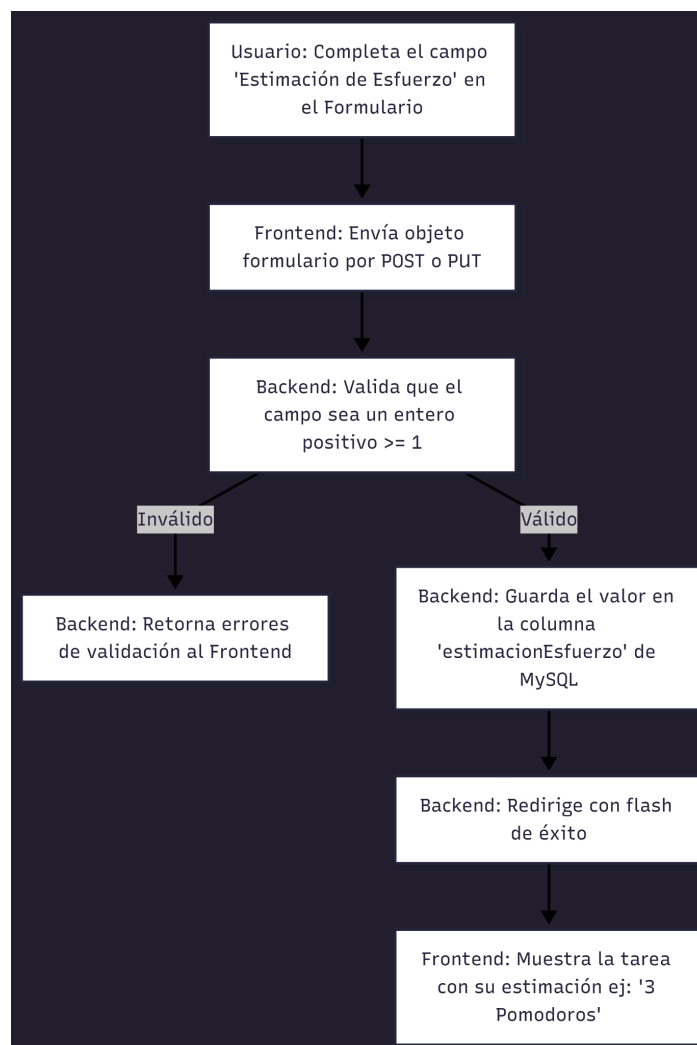
- **Backend & Controladores**

TareaController.php (/app/Http/Controllers/) - Valida e introduce el campo `estimacionEsfuerzo` al crear (`store()`) o actualizar (`update()`) tareas.

- **Modelos y Datos (Eloquent ORM)**

Tarea.php (/app/Models/) - Representación de la tabla `Tarea` con la regla de casteo `estimacionEsfuerzo` => `integer`.

Flujo de Datos y Control



1. **Frontend:** El usuario abre el modal de creación o edición de tarea e ingresa un número (ej. `3`) en el campo "Estimación de Esfuerzo (Pomodoros)".
2. **Envío de Datos:** Al enviar el formulario, Inertia serializa el formulario y lo envía a la ruta `tareas.store` o `tareas.update`.
3. **Backend (Validación):** El controlador ejecuta la validación. Si el campo no es numérico o es menor a 1, se interrumpe el flujo y se devuelven los errores.
4. **Base de Datos:** Se escribe/actualiza el registro en la tabla `Tarea`.
5. **Visualización:** Al recargarse los datos, la tarjeta de la tarea y el reproductor de la sesión de Pomodoro recuperan el valor para calcular e ilustrar el progreso de ciclos completados vs. estimados.

Pruebas y Validación

1. **Precondición:** Iniciar sesión con un usuario de prueba.
2. **Paso 1:** Ir a "Mis Tareas" y pulsar en "Crear Tarea".
3. **Paso 2:** En el campo "Estimación de Esfuerzo (Pomodoros)", escribir `-2` o `texto` e intentar guardar.
 - a. **Resultado Esperado:** La validación del sistema debe fallar, impidiendo el envío o mostrando un mensaje de error ("El campo estimacion esfuerzo debe ser un número entero" o similar).
4. **Paso 3:** Corregir el campo ingresando el número `4` y guardar la tarea.
 - a. **Resultado Esperado:** La tarea se crea de forma correcta. Al abrir sus detalles o tarjeta, debe indicar visualmente `4 Pomodoros` (o `0/4 ` en el reproductor de Pomodoro).

RF-M11: Perfil de Usuario Compartido

Descripción y Objetivo

Este conjunto de requerimientos define la capacidad de trabajo colaborativo dentro del sistema mediante el concepto de Perfiles Compartidos y control de accesos por roles (Permisos de Perfil):

- **Perfiles Compartidos (RF-M11):** Permite que el creador (propietario) de un perfil de trabajo lo comparta con otros usuarios enviando invitaciones mediante correo electrónico. Al aceptar la invitación, los destinatarios acceden a todo el contenido del perfil (apuntes, tareas, y temporizadores Pomodoro asociados).
- **Permisos de Perfil (RF-M11.1):** Establece un control de acceso granular mediante roles jerárquicos estructurados. El creador del perfil puede asignar un nivel de acceso específico a cada usuario invitado:
 - **Lector:** Permite únicamente la lectura y visualización de los apuntes y tareas asociadas al perfil. No puede crear, modificar ni eliminar ningún elemento, y el editor de apuntes/tareas opera en modo de solo lectura.
 - **Editor:** Permite visualizar, crear y modificar apuntes y tareas dentro del perfil compartido.
 - **Administrador:** Además de las acciones del Editor, tiene permisos para eliminar apuntes/tareas y gestionar la asignación/modificación de permisos a otros miembros colaboradores.

Tecnologías, Herramientas y Librerías

- **Laravel Policies (Laravel Gate):** Implementación de la clase `PerfilPolicy` para encapsular la jerarquía y reglas de autorización de accesos de manera centralizada.
- **Inertia.js & Vue 3:** Flujo de modals dinámicos (`CompartirPerfilModal.vue`) y enlace reactivo de variables en el frontend (`v-model`, `useForm`) para actualizar roles de forma asíncrona mediante peticiones `PUT`/`DELETE`/`POST`.
- **Laravel Notifications & SMTP (Mail):** `InvitacionPerfilNotification` construye y envía un correo electrónico personalizado a los invitados con un enlace seguro firmado mediante un token de un solo uso de 64 caracteres.
- **MySQL / Eloquent ORM:**
 - **Tabla `PerfilCompartido`:** Estructura pivote intermedia que gestiona la relación muchos a muchos entre usuarios y perfiles, almacenando el rol (`permiso`: Lector, Editor, Administrador) y metadatos de auditoría (`compartidoPor`, `fechaCompartido`).
 - **Tabla `InvitacionPerfil`:** Almacena el historial de invitaciones pendientes, correos, token y fecha de expiración.

Archivos Involucrados en el Requerimiento

- **Frontend**

CompartirPerfilModal.vue (/resources/js/Components/) - Modal de administración de colaboradores, envío de invitaciones, edición de roles y revocación de accesos.

GestionPerfil.vue (/resources/js/Pages/) - Integra el modal de compartir y gestiona la activación del perfil activo.

Invitacion.vue (/resources/js/Pages/) - Vista para aceptar/rechazar invitaciones.

InvitacionExpirada.vue (/resources/js/Pages/perfil-compartido/) - Vista en caso de token expirado o utilizado.

Index.vue y Editor.vue (/resources/js/Pages/apunte/) - Controlan que la interfaz oculte botones de edición/creación o bloquee el editor en modo solo lectura (`isReadOnly`) para el rol `Lector`.

GestionTareas.vue (resources/js/Pages/) - Bloquea o permite la interacción en tareas según los permisos de perfil.

- **Backend & Controladores (Laravel)**

PerfilCompartidoController.php (/app/Http/Controllers/) - Coordina el flujo de invitaciones, validaciones, confirmación, y edición de roles pivote.

ApunteController.php (/app/Http/Controllers/) - Valida autorizaciones antes de realizar acciones en apuntes.

TareaController.php (/app/Http/Controllers/) - Valida autorizaciones antes de realizar acciones en tareas.

InvitacionPerfilNotification.php (/app/Notifications/) - Clase que formatea el email para el colaborador.

- **Políticas y Modelos (Laravel Policy & ORM)**

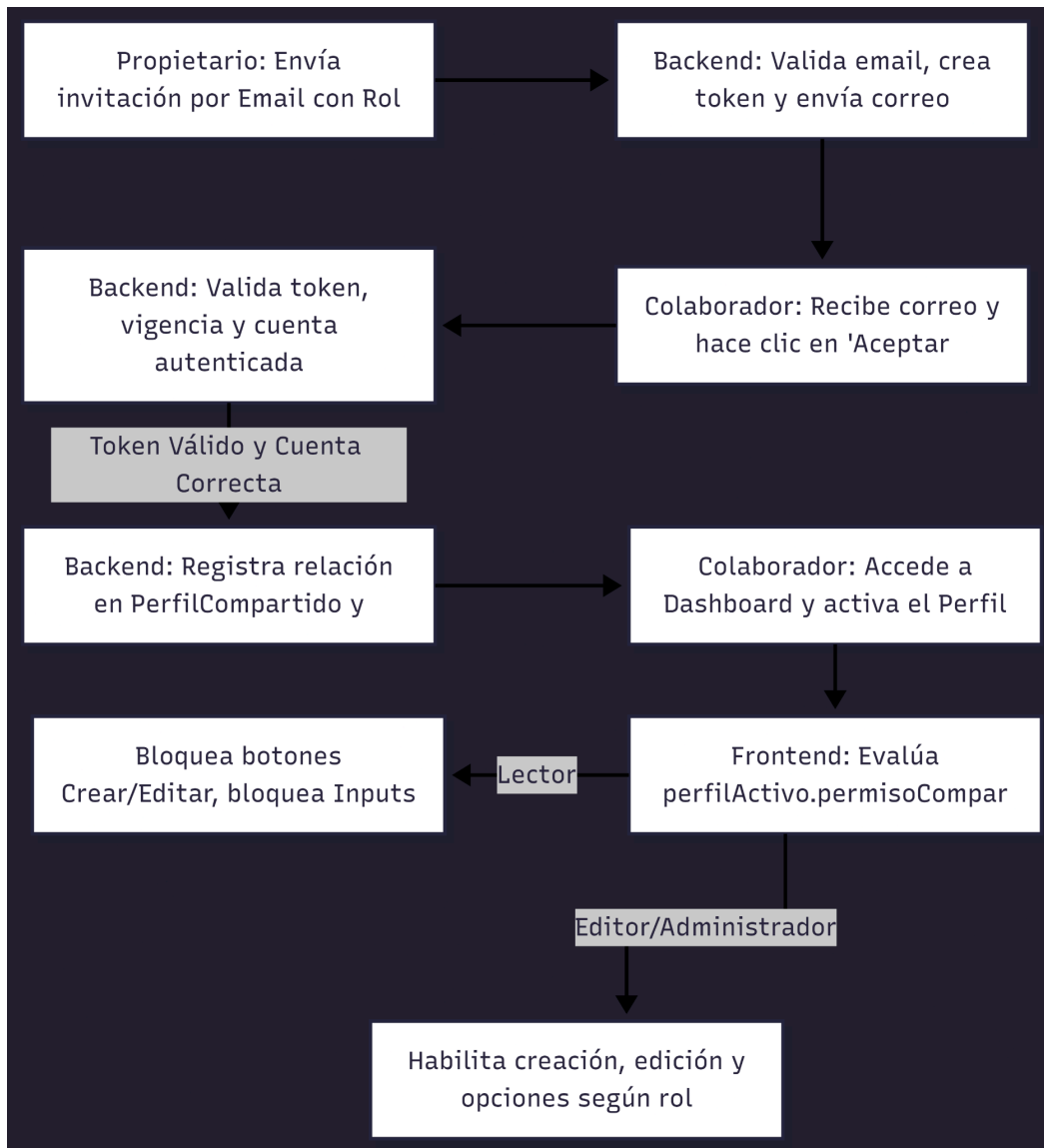
PerfilPolicy.php (/app/Policies/) - Centraliza la lógica jerárquica de permisos (`Lector` = 1, `Editor` = 2, `Administrador` = 3) y la resolución de la condición `before()`.

Perfil.php (/app/Models/) - Define la relación `usuariosCompartidos` muchos a muchos con la tabla pivote.

InvitacionPerfil.php (/app/Models/) - Manejo y persistencia de invitaciones y tokens.

PerfilCompartido.php (/app/Models/) - Representa el registro de acceso colaborativo.

Flujo de Datos y Control



1. Envío de Invitación:

- a. El propietario entra a "Gestión de Perfiles" y abre "Compartir". Escribe el email del colaborador, selecciona el rol (Lector/Editor/Administrador) y presiona "Invitar". El backend genera un token criptográfico único, establece la expiración en 7 días y envía el correo mediante `InvitacionPerfilNotification`.

2. Recepción y Validación:

- a. El invitado accede al enlace seguro `/invitacion/{token}`.

- b.** Si no está autenticado, el sistema le solicita ingresar/registrarse. Si su correo no coincide con el destinatario de la invitación, se deniega el acceso.
- c.** Al hacer clic en "Aceptar", el backend crea la entrada en `PerfilCompartido` con el rol seleccionado y marca la invitación como utilizada.

3. Control de Accesos Dinámicos:

- a.** Durante la sesión activa, el controlador de Tareas y Apuntes adjunta el atributo `permisoCompartido` al perfil.
- b.** El middleware y la clase `PerfilPolicy.php` verifican en cada request si el rol del usuario (`Lector` = 1, `Editor` = 2, `Administrador` = 3) alcanza el nivel mínimo requerido:
 - i.** `ver`: Requiere al menos `Lector` (1).
 - ii.** `crear` / `modificar`: Requiere al menos `Editor` (2).
 - iii.** `borrar` / `gestionarPermisos`: Requiere al menos `Administrador` (3).
 - iv.** El dueño del perfil omite las verificaciones gracias al método `before()` que siempre devuelve `true`.

Pruebas y Validación

- 1. Precondición:** Tener dos cuentas de usuario creadas (Usuario A: Propietario, Usuario B: Colaborador).
- 2. Paso 1 (Envío de Invitación):**
 - a.** Iniciar sesión como Usuario A. Ir a "Gestión de Perfiles".
 - b.** Abrir el modal "Compartir" en un perfil propio. Invitar al Usuario B ingresando su correo electrónico y seleccionando el rol Lector.
 - c. Resultado Esperado:** Se muestra la invitación en la lista como "Pendiente" y se despacha el email.

3. Paso 2 (Aceptación de Invitación):

- a. Iniciar sesión como Usuario B y entrar al link de invitación enviado al correo (o la URL ``/invitacion/{token}`` generada).
- b. Hacer clic en "Aceptar Invitación".
- c. **Resultado Esperado:** El sistema redirige al Dashboard con un mensaje de éxito. El perfil ahora está listado en sus perfiles disponibles.

4. Paso 3 (Validación de Rol Lector):

- a. Como Usuario B, activar el perfil compartido. Ir a "Mis apuntes".
- b. **Resultado Esperado:** Los botones de "Nuevo apunte" y "Eliminar" no están visibles. Se muestra un indicador de "Solo lectura". Al abrir un apunte existente, el título, el contenido (``NoteEditor``) y los paneles de control de audio/formato están totalmente bloqueados y en modo lectura.

5. Paso 4 (Modificación de Permiso):

- a. Iniciar sesión de nuevo como Usuario A. Abrir "Compartir" en el perfil correspondiente.
- b. En el menú desplegable del Usuario B, cambiar el rol de Lector a Editor.
- c. Volver a la cuenta del Usuario B, recargar la página.
- d. **Resultado Esperado:** Los controles de creación y edición de apuntes/tareas ahora se encuentran completamente activos para el Usuario B.

RF-M12: Cambios en el Sistema de Rachas

Descripción y Objetivo

Este requerimiento define una restricción en la acumulación y aumento de la racha de concentración activa del usuario:

- **Restricción de Racha:** La racha diaria de estudio/trabajo del usuario aumentará únicamente cuando complete de forma satisfactoria una sesión de concentración configurada para una duración exacta de 25 minutos (el estándar Pomodoro).
- **Objetivo:** Asegurar el valor e integridad del sistema de gamificación (rachas). Evita que los usuarios puedan abusar del sistema o inflar artificialmente sus estadísticas completando de forma reiterada micro-sesiones rápidas de corta duración (ej. sesiones configuradas de 1 a 5 minutos).

Tecnologías, Herramientas y Librerías

Este requerimiento utiliza la estructura relacional de estadísticas y configuraciones de Pomodoro en Laravel:

- **MySQL / Eloquent ORM:**
 - Tabla ``ConfiguracionPomodoro``: Almacena la columna ``duracionSesion`` (en minutos).
 - Tabla ``SesionPomodoro``: Registra el historial de sesiones asociadas a una configuración.
 - Tablas ``Estadistica`` y ``Racha``: Persisten los valores de racha activa e histórico del usuario.
- **Servicios de Negocio (Laravel Service Layer):**
 - Clase ``EstadisticaService.php``: Encapsula la lógica de evaluación e incremento de la racha, aplicando el filtro de duración.

Archivos Involucrados en el Requerimiento

- **Frontend**

No requiere modificaciones directas en el frontend, pero la racha se visualiza en:

PerfilUsuario.vue (/resources/js/Pages/) - Muestra al usuario su racha actual y su racha récord.

Dashboard.vue (/resources/js/Pages/) - Muestra el panel con resúmenes estadísticos que incluyen las rachas de productividad.

- **Backend & Controladores (Laravel)**

PomodoroController.php (/app/Http/Controllers/) - Recupera la duración configurada para la sesión activa y la envía como argumento al servicio al registrar y completar ciclos o finalizar la sesión.

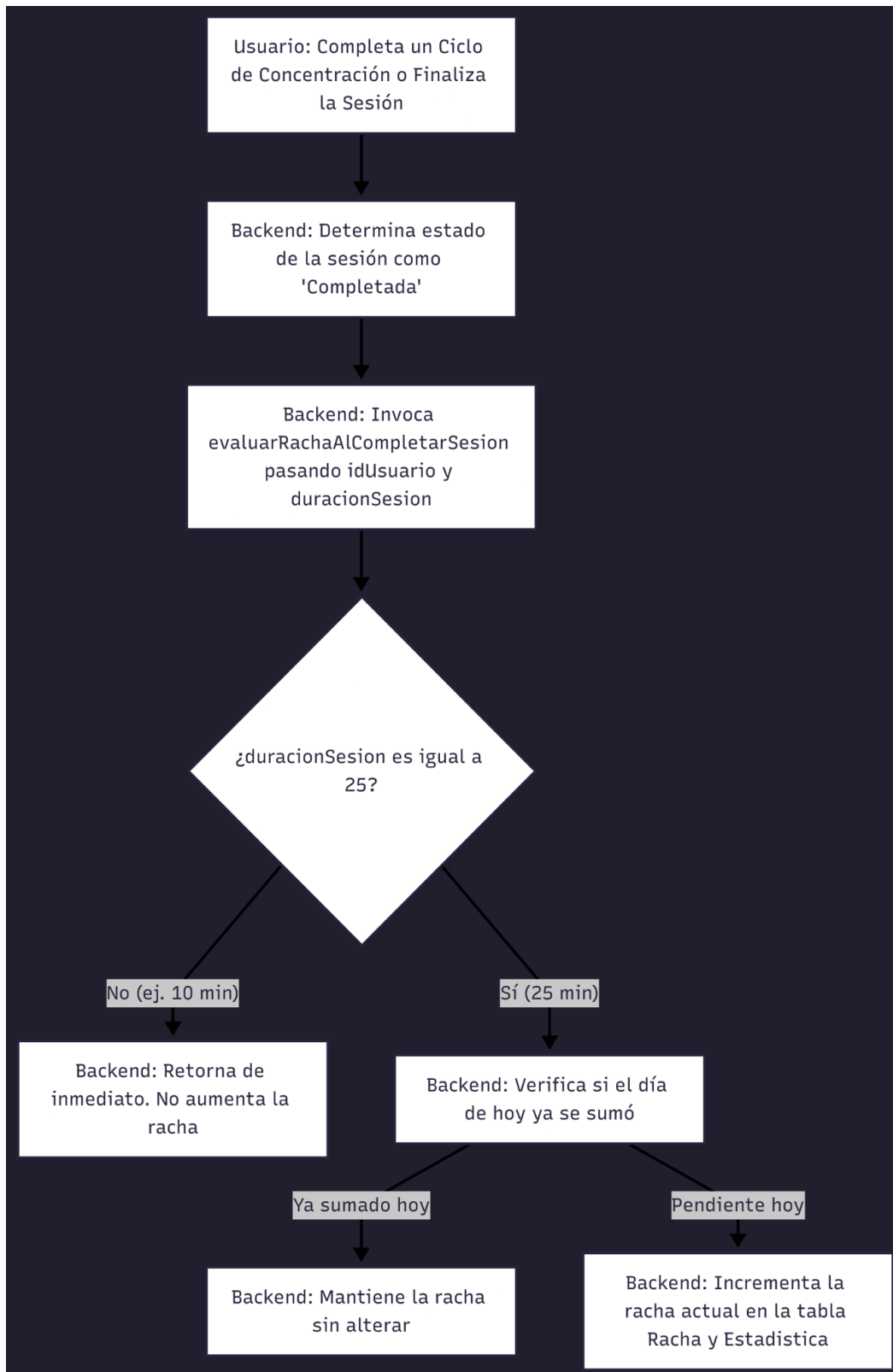
EstadisticaService.php (/app/Services/) - Filtra el incremento de racha en el método `evaluarRachaAlCompletarSesion`, finalizando el flujo de manera temprana si la sesión no es de exactamente 25 minutos.

- **Modelos y Datos (Eloquent ORM)**

SesionPomodoro.php (/app/Models/) - Representa la sesión de trabajo y provee la relación con su configuración.

ConfiguracionPomodoro.php (/app/Models/) - Almacena las preferencias de tiempo del temporizador.

Racha.php y Estadistica.php (/app/Models/) - Modelos de persistencia de las rachas de concentración.

Flujo de Datos y Control

1. **Frontend:** El temporizador llega a cero (o el usuario finaliza manualmente la sesión con éxito).
2. **Envío:** Se despacha la petición POST a ``pomodoro.registrar`` o ``pomodoro.finalizar``.
3. **Controlador (`PomodoroController``):**
 - a. Identifica que el estado final es ``Completada``.
 - b. Recupera el modelo de sesión y extrae la duración de la sesión configurada a través de la relación ``$sesion->configuracionPomodoro->duracionSesion``.
 - c. Invoca a ``$estadisticaService->evaluarRachaAlCompletarSesion($userId, $duracionSesion)``.
4. **Servicio (`EstadisticaService``):**
 - a. Evalúa si ``$duracionSesion`` es ``25``.
 - b. Si no es ``25``, detiene el procesamiento inmediatamente (``return;``).
 - c. Si es ``25``, realiza las comprobaciones cronológicas estándar (si ayer hubo racha para incrementarla, o si hoy ya fue incrementada) y escribe el resultado en MySQL.

Pruebas y Validación

1. **Precondición:** Iniciar sesión en el sistema y verificar que la racha actual en el perfil sea de ``0``.
2. **Paso 1 (Sesión no válida):**
 - a. Ir a la configuración de Pomodoro y definir una duración de sesión de 10 minutos y guardar.
 - b. Iniciar la sesión de Pomodoro y simular/completar el ciclo de trabajo de 10 minutos hasta finalizarla con éxito.
 - c. Ir a la sección "Perfil de Usuario" o ver las estadísticas del dashboard.

d. Resultado Esperado: La racha actual debe mantenerse en `0` (el tiempo de concentración total de 10 minutos se acumula en las horas de estudio, pero no afecta a la racha diaria).

3. Paso 2 (Sesión válida)

Iniciar una sesión rápida (Quick Start) o configurar la duración de sesión en exactamente 25 minutos.

Completar el ciclo de trabajo y finalizar la sesión de Pomodoro de forma correcta.

Ir a la sección de "Perfil de Usuario" o al dashboard.

Resultado Esperado: La racha actual del usuario debe haber incrementado a `1`.

RF-M13: Cambios en las Estadísticas

Descripción y Objetivo

Este requerimiento introduce dos nuevas métricas y estadísticas clave para potenciar la autoevaluación y productividad del usuario en el sistema:

- **Horas de Concentración Diaria:** Muestra el total de tiempo (en horas) acumulado de enfoque/concentración del día actual (hoy).
- **Cantidad de Sesiones por Perfil:** Muestra el volumen total (cantidad) de sesiones Pomodoro registradas en cada perfil del usuario.
- **Objetivo:** Proveer un panorama detallado de en qué invierte el usuario sus recursos temporales diarios, y medir cuantitativamente su nivel de apego a cada perfil de trabajo o asignatura (ej. cuántas sesiones le dedica a cada uno).

Tecnologías, Herramientas y Librerías

- **MySQL / Eloquent ORM:**

- Consulta asíncrona de suma de tiempos de la tabla `SesionPomodoro` filtrando por fecha (`Carbon::today()`).
- Agrupación e indexado de relaciones para contabilizar cantidad de sesiones por perfil (`SesionPomodoro::whereHas('tarea')` enlazando con la tabla `Perfil`).

- **Inertia.js & Vue 3:**

- Renderizado dinámico de tarjetas informativas en la fila superior con Bootstrap Icons.
- Actualización reactiva de la leyenda del gráfico de dona con el formato `Perfil: Xh (Y sesiones)`.

Archivos Involucrados en el Requerimiento

- **Frontend**

Estadisticas.vue (/resources/js/Pages/) - Muestra las tarjetas con la hora diaria de hoy con íconos vectoriales modernos y la cantidad de sesiones por perfil en la leyenda del gráfico de dona.

- **Backend & Controladores (Laravel)**

EstadisticaController.php (/app/Http/Controllers/) - Recalcula dinámicamente las horas de hoy e introduce el recuento de sesiones (`sesionesCount`) en el arreglo del gráfico de dona.

DashboardController.php (/app/Http/Controllers/) - Sincroniza dinámicamente el valor en la carga del dashboard del usuario.

EstadisticaService.php (/app/Services/) - Recalcula y guarda el valor en base de datos al registrar nuevos minutos de trabajo.

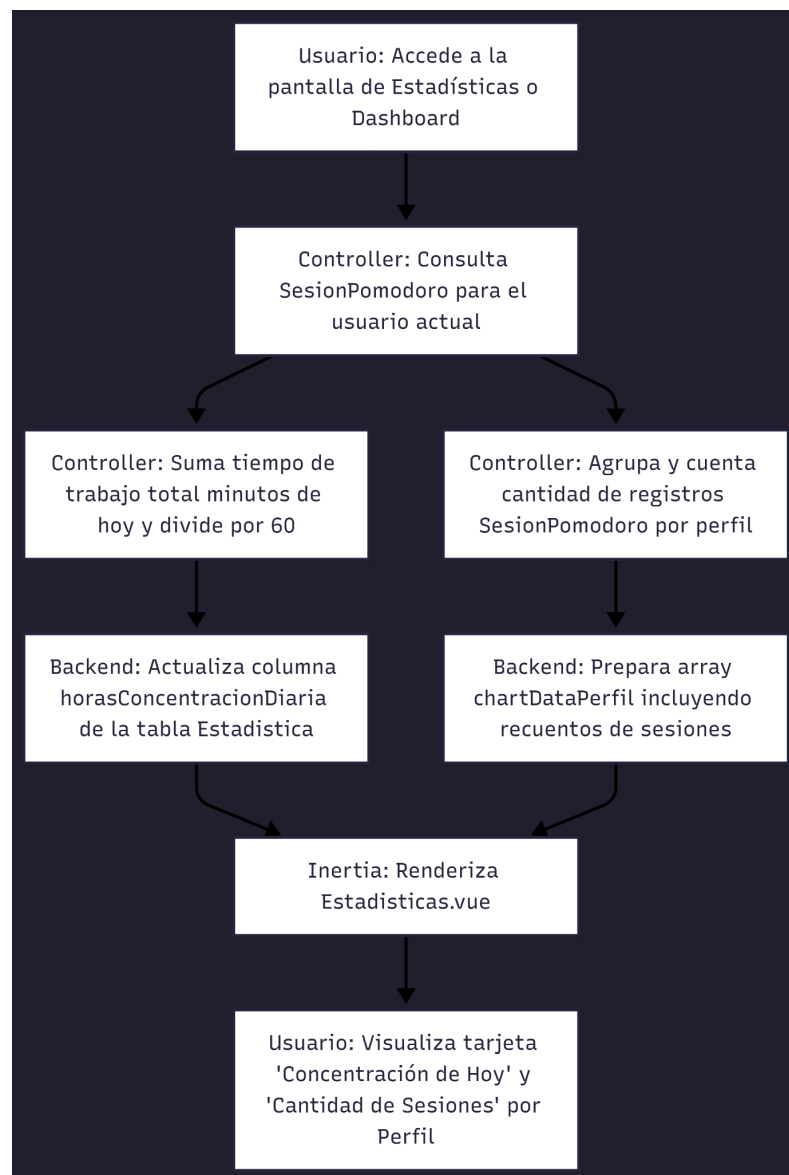
- **Modelos y Datos (Eloquent ORM)**

Estadistica.php (/app/Models/) - Modela la tabla que almacena

`horasConcentracionDiaria`.

SesionPomodoro.php (/app/Models/) - Utilizado para sumar el tiempo acumulado de concentración por fecha y perfil.

Flujo de Datos y Control



1. **Petición:** El usuario solicita la carga del dashboard o la pestaña de estadísticas.
2. **Cómputo en Backend:**
 - a. Para las horas diarias: Se suma la columna `tiempoTrabajoTotalMinutos` de todos los registros `SesionPomodoro` del usuario donde la fecha (`fechaCreacionSesion`) sea igual a `Carbon::today()`, y se divide el resultado por 60 para convertirlo a horas (con precisión decimal).
 - b. Para las sesiones por perfil: Para cada perfil, se cuenta el número de registros `SesionPomodoro` vinculados mediante sus tareas correspondientes (`SesionPomodoro::whereHas('tarea')` enlazada a la tabla `Perfil`).
3. **Persistencia:** Se actualiza la base de datos para que la columna `horasConcentracionDiaria` refleje el cálculo preciso de hoy.
4. **Presentación:** Inertia devuelve las variables calculadas al frontend, el cual dibuja la tarjeta de estadísticas de concentración diaria con un ícono de reloj de arena y añade las sesiones asociadas junto al nombre de cada perfil en la leyenda del gráfico de dona.

Pruebas y Validación

1. **Precondición:** Iniciar sesión en el sistema y tener al menos un perfil creado.
2. **Paso 1 (Verificación del estado inicial):**
 - a. Ir a la pestaña "Estadísticas".
 - b. **Resultado Esperado:** La tarjeta "Concentración de Hoy" debe reflejar `0 hrs` (o un valor decimal bajo si ya se ha trabajado hoy). El gráfico circular de dona de "Horas de pomodoro por perfil" no debe listar sesiones activas en la leyenda del perfil a menos que ya existieran de forma previa.
3. **Paso 2 (Registro de Enfoque):**
 - a. Iniciar una sesión de Pomodoro rápida de 25 minutos asociada a una tarea del perfil seleccionado y completarla.

b. Ir a la sección "Estadísticas".

c. Resultado Esperado:

- i.* La tarjeta "Concentración de Hoy" debe reflejar un aumento de `0.42 hrs` (representando 25/60 minutos).
- ii.* La leyenda del gráfico circular de dona para el perfil activo debe reflejar:
`NombrePerfil: 0.42h (1 sesiones)` demostrando que el contador de sesiones incrementó en `1`.

Calidad del Software

Para garantizar la calidad de Cronos-Note, se ha seleccionado el modelo ISO/IEC 25010 (parte de la familia ISO 25000 SQuaRE). Este estándar evalúa la calidad del producto de software bajo 8 características clave, las cuales se justifican en el proyecto de la siguiente manera:

- **Adecuación Funcional.** Permite auditar que las lógicas centrales del Pomodoro, apuntes de voz y control de accesos por perfil (PerfilPolicy) cumplan estrictamente los requisitos definidos.
- **Usabilidad.** Vital para una aplicación interactiva que tiene como objetivo el estudio focalizado (modo Zen), requiriendo un diseño estético y responsive en dispositivos móviles.
- **Fiabilidad.** Garantiza que no haya pérdidas en buffers de grabación de voz prolongados y que el cálculo horaria de las rachas diarias no se rompa ante interrupciones de red.
- **Seguridad.** Clave para el manejo de credenciales OAuth de terceros (Google, Spotify) y la protección de datos e identidades de sesión (CheckCustomSession).
- **Mantenibilidad.** Evalúa el grado de modularidad de la arquitectura basada en Laravel y Vue, facilitando futuras ampliaciones y correcciones en los servicios (ej. EstadisticaService).
- **Eficiencia de Desempeño.** Mide el consumo de recursos locales de los navegadores durante las grabaciones y peticiones de inicio/pausa de los temporizadores.
- **Compatibilidad.** Relevante para certificar la coexistencia con reproductores nativos del sistema y el flujo continuo con webhooks externos.
- **Portabilidad.** Asegura que el aplicativo funcione de manera idéntica en navegadores como Chrome, Safari, Firefox y Edge.

Cuestionario de Evaluación de Calidad (20 Preguntas)

El cuestionario estructurado bajo las 8 dimensiones del estándar ISO/IEC 25010 consta de las siguientes 20 preguntas específicas para auditar Cronos-Note:

Adecuación Funcional

- **Compleitud Funcional:** ¿El sistema implementa de forma completa todas las funciones clave planificadas (Pomodoro, gestión de tareas, registro de apuntes de texto/audio, invitaciones, rachas y estadísticas)?
- **Corrección Funcional:** ¿Las tareas del sistema de rachas se rompen de manera correcta ante la inactividad diaria, y las estimaciones de esfuerzo (Pomodoros) se calculan con exactitud matemática?
- **Pertinencia Funcional:** ¿Los perfiles de usuario compartidos y la asignación de permisos (Lector, Editor, Administrador) se adecúan de forma práctica a los casos de uso reales de colaboración entre alumnos?

Eficiencia de Desempeño

- **Tiempo de Respuesta:** ¿El tiempo de respuesta del servidor ante eventos interactivos críticos (ej. registrar la cancelación de una sesión, pausar el Pomodoro, o guardar un apunte) es inferior a 500 milisegundos?
- **Utilización de Recursos:** ¿El consumo de recursos de memoria (RAM) y procesamiento del navegador se mantiene estable durante grabaciones extensas de apuntes de audio (superiores a 45 minutos) sin congelar la pestaña?

Compatibilidad

- **Coexistencia:** ¿La aplicación (especialmente el reproductor y grabador de audio) funciona de manera armónica en pestañas en segundo plano sin interrumpir ni ser interrumpido por otras herramientas multimedia del dispositivo?
- **Interoperabilidad:** ¿El intercambio de datos y sincronización de autorizaciones OAuth con las APIs de Spotify y Google Calendar se realiza de forma limpia y sin generar bloqueos en el sistema?

Usabilidad

- **Estética de la Interfaz:** ¿El diseño visual (paleta de colores marrones, tipografías y el minimalismo de la pantalla de Sesión Zen) favorece el enfoque del usuario y evita distracciones?
- **Facilidad de Aprendizaje y Operabilidad:** ¿Los usuarios son capaces de entender intuitivamente cómo asociar tareas a perfiles y configurar una sesión Pomodoro personalizada en su primer uso sin necesidad de un manual?
- **Accesibilidad y Responsividad:** ¿La interfaz responsive permite realizar el 100% de las operaciones de gestión de tareas, edición de apuntes y consulta de estadísticas en pantallas móviles y táctiles sin desbordamientos visuales?

Fiabilidad

- **Tolerancia a Fallos:** ¿El sistema maneja las caídas temporales de conexión a internet durante el uso del Pomodoro sin perder el tiempo acumulado de la sesión en curso?
- **Recuperabilidad:** ¿Si una sesión de grabación de audio se interrumpe abruptamente (ej. por cierre accidental de pestaña o falta de energía), el sistema es capaz de

recuperar los fragmentos almacenados previamente en IndexedDB/almacenamiento local?

- **Disponibilidad:** ¿Las tareas programadas en segundo plano (ej. verificación de expiración de tokens de sesión) garantizan que el sistema esté disponible y active sesiones correctamente en cualquier momento?

Seguridad

- **Confidencialidad y Control de Acceso:** ¿La política de seguridad `PerfilPolicy` impide de manera estricta que usuarios con rol 'Lector' o usuarios externos modifiquen apuntes o eliminen tareas dentro de un perfil compartido?
- **Integridad y Autenticación:** ¿El token de sesión generado por `SesionUsuario` y la validación a través de la cookie `cronos_session_token` previenen el secuestro de sesiones y accesos no autorizados?
- **Responsabilidad:** ¿Las operaciones de compartir perfiles o modificar permisos quedan correctamente registradas en la base de datos (con marcas de tiempo y usuario responsable en la tabla `PerfilCompartido`)?

Mantenibilidad

- **Modularidad:** ¿La división lógica entre el Backend (modelos, servicios independientes como `EstadisticaService` y políticas) y el Frontend (componentes de Vue reutilizables como `ConfirmModal`) facilita cambios aislados sin afectar a otros componentes del sistema?
- **Analizabilidad y Modificabilidad:** ¿La nomenclatura de clases, variables, base de datos y la organización del código permiten localizar y corregir un error (como un ajuste en la lógica de rachas) en un tiempo razonable?

- **Capacidad de ser Probado:** ¿La arquitectura permite ejecutar pruebas unitarias y de integración sobre las reglas de negocio (Por ejemplo: verificar la racha pérdida o validar tokens) de forma automatizada?

Portabilidad

- **Adaptabilidad e Instalabilidad:** ¿La aplicación web se visualiza y ejecuta sin errores funcionales o diferencias en los motores de renderizado de los navegadores web líderes (Chrome, Safari, Firefox y Edge)?

Análisis de riesgos

1. Desincronización y Ruptura Falsa de Rachas por Zonas Horarias (Streak Inconsistency)

Descripción

Al implementar un sistema de rachas diario al estilo Duolingo (donde la racha se rompe a las 00:00 si no se ha completado ninguna tarea), existe un riesgo alto de desincronización si el cálculo se realiza estrictamente en base a la zona horaria del servidor (por ejemplo, UTC) en lugar de la zona horaria local del navegador del usuario.

Impacto

Usuarios que viajen o vivan en zonas horarias distintas a la del servidor verán rotas sus rachas de forma errónea, lo que produce una alta frustración y pérdida de confianza en la plataforma.

2. Conflictos de Concurrencia y Pérdida de Datos en Perfiles Compartidos (Race Conditions)

Descripción

Al habilitar la edición de apuntes, notas y tareas dentro de perfiles compartidos entre múltiples usuarios con permisos de escritura (Editor/Administrador), existe el riesgo de que dos personas modifiquen simultáneamente el mismo recurso.

Impacto

Si no se implementa un control de versiones o bloqueo optimista (como websockets para edición en tiempo real o tokens de concurrencia), el último guardado sobrescribirá silenciosamente al anterior, provocando pérdida de datos de estudio críticos de los alumnos.

3. Pérdida de Datos en Grabaciones de Audio por Caídas de Conexión o Cierres de Pestaña

Descripción

La funcionalidad de grabar audio de clases directamente en el navegador almacena buffers en la memoria local. Si el navegador se queda sin memoria, la pestaña se congela o la conexión a internet del usuario se corta antes de completar la subida del archivo al servidor, el archivo de audio se perderá.

Impacto

Pérdida irrecuperable de explicaciones de clases de hasta 2 horas, generando descontento severo en el usuario y una mala reputación de confiabilidad para la app.

4. Vulnerabilidad y Dependencia Extrema de APIs de Terceros (Google)***Descripción***

El software depende de la APIs externas de Google (OAuth)

Impacto

Cambios inesperados en los términos de servicio, deprecación de endpoints por parte de Google, revocaciones de tokens de acceso por inactividad o fallos en las pasarelas externas pueden deshabilitar repentinamente flujos clave como el inicio de sesión, la sincronización de tareas con el calendario de estudio o la ambientación musical de la Sesión Zen.

5. Cuello de Botella y Agotamiento de Almacenamiento en Servidor por Archivos de Audio***Descripción***

El almacenamiento de archivos de audio de clases grabadas directamente en el disco duro del servidor sin una política de compresión agresiva, límites de duración de audio, CDN o almacenamiento en la nube (como Amazon S3/Google Cloud Storage) llenará rápidamente el espacio en disco.

Impacto

Una vez que el disco del servidor se llene, toda la aplicación dejará de funcionar (fallos en base de datos, sesiones inaccesibles, bloqueos del servidor), provocando una caída completa del servicio.

Prioridad	ID de Riesgo	Título del Riesgo	Probabilidad	Impacto
1	R-01	Agotamiento de almacenamiento del servidor por grabaciones de audio	70%	Alto
2	R-02	Dependencia y fallos en APIs externas de Terceros (Google / Spotify)	60%	Alto
3	R-03	Conflictos de concurrencia y pérdida de datos en Perfiles Compartidos	50%	Alto
4	R-04	Pérdida de audio local durante la grabación por fallos de red/navegador	65%	Medio
5	R-05	Desincronización horaria y ruptura falsa del sistema de rachas	80%	Medio

Hoja de Información de Riesgo y Plan de Continencia

Hoja de información de riesgo			
Riesgo ID: R-01	Fecha: 14/06/26	Prob: 70%	Impacto: Alto
Descripción: El crecimiento desmedido en el almacenamiento de audios grabados por los usuarios para sus apuntes llenará el espacio en disco del servidor local de Laravel. Esto impedirá la escritura en la base de datos MySQL y provocará la caída total del sistema Cronos-Note.			
Refinamiento/Contexto: Subcondición 1: El sistema almacena los archivos de audio en formato nativo sin compresión de bitrate o conversión a formatos ligeros (como mono o WebM de baja tasa). Subcondición 2: No existe un límite de tiempo por grabación de apuntes ni un sistema de cuotas por usuario. Subcondición 3: La infraestructura actual carece de integraciones con almacenamiento elástico en la nube (como AWS S3 o Google Cloud Storage).			
Mitigación/monitoreo: 1. Implementar compresión automática de audio en el frontend antes de enviar el archivo a Laravel (utilizando librerías de conversión a OGG/WebM optimizadas para voz). 2. Definir alertas automáticas en el sistema de monitorización del servidor que se disparen cuando el almacenamiento en disco supere el 80% y 90%. 3. Establecer un límite estricto de duración máxima por sesión de apuntes de audio (por ejemplo, 120 minutos por apunte).			
Manejo/plan de contingencia/disparador: Plan de contingencia: Implementar inmediatamente una rutina de almacenamiento secundario en			

servicios externos (ej. AWS S3 o compatible) moviendo archivos de audio antiguos (inactivos por más de 30 días) para liberar espacio físico del disco.

- Habilitar un script de purga automatizada de archivos temporales de audio que no hayan sido completados o confirmados.

Disparador: El almacenamiento en disco del servidor supera el 85% de capacidad.

Estado actual:

14/06/2026: Pasos de mitigación y estimación de costos para migración a AWS S3 iniciados.

Originador: Tester

Asignado: Dev Backend

Hoja de información de riesgo

Riesgo ID: R-02

Fecha: 14/06/26

Prob: 60%

Impacto: Alto

Descripción:

Los servicios integrados de Google (para autenticación y sincronización del Calendario) pueden sufrir caídas de servicio, cambios de API no anunciados, o bloqueos de credenciales OAuth de nuestro cliente.

Refinamiento/Contexto:

Subcondición 1: El flujo de registro e inicio de sesión de Cronos-Note depende fuertemente de Google Auth; si este falla, los usuarios federados no podrán entrar a sus cuentas.

Mitigación/monitoreo:

1. Diseñar flujos alternativos donde el fallo de una API externa no bloquee el sistema (por ejemplo, permitir siempre login local clásico con contraseña).
2. Monitorear los tiempos de respuesta y errores de red de las peticiones a Google y Spotify con logs detallados.
3. Suscribirse a los portales de desarrollo de Google Cloud Console y Spotify Developer para recibir alertas de cambios en políticas y APIs.

Manejo/plan de contingencia/disparador:

Plan de contingencia:

- Si falla el login de Google, ofrecer restablecimiento rápido de contraseña para crear una credencial local.
- Si la API de Spotify falla, activar un reproductor alternativo de audio local offline integrado en la Sesión Zen que contenga 5 pistas de sonido ambiente precargadas (ruido blanco, lluvia, bosque, lo-fi local).

Disparador: Tasa de error en peticiones a APIs externas supera el 15% en un lapso de 10 minutos.

Estado actual:

14/06/2026: Pistas de sonido local predeterminadas añadidas como fallback en el reproductor Zen.

Originador: Tester

Asignado: Dev Backend

Hoja de información de riesgo			
Riesgo ID: R-03	Fecha: 14/06/26	Prob: 50%	Impacto: Alto
Descripción: Cuando múltiples usuarios con permisos de edición acceden y modifican tareas, apuntes o calendarios simultáneamente en un perfil compartido, las modificaciones concurrentes de uno pueden sobrescribir silenciosamente las del otro sin previo aviso.			
Refinamiento/Contexto: Subcondición 1: El sistema actualiza los apuntes y notas enviando el contenido completo mediante peticiones PUT HTTP clásicas sin validar marcas de tiempo de última modificación. Subcondición 2: No existe un canal bidireccional en tiempo real (como WebSockets) que notifique a un usuario si otra persona está editando el mismo archivo en ese momento.			
Mitigación/monitoreo: 1. Implementar "Bloqueo Optimista" en la base de datos añadiendo una columna versión o updated_at que se valide al momento de guardar. Si el registro ya fue modificado en el backend por otro cliente, se rechaza la edición del segundo usuario advirtiéndole del conflicto. 2. Habilitar un historial básico de cambios o papelera de reciclaje para poder recuperar versiones previas de los apuntes borrados o modificados.			
Manejo/plan de contingencia/disparador: Plan de contingencia: Mostrar una ventana de resolución de conflictos visual en Vue donde el usuario pueda comparar el texto actual guardado en la base de datos frente al texto local que él intentó enviar, dándole la opción de combinar o elegir cuál conservar. Disparador: Ocurrencia de un conflicto de guardado concurrente (error de actualización en base de datos por colisión de marcas de tiempo).			
Estado actual: 14/06/2026: Planificación de la columna updated_at en base de datos para la tabla Apuntes iniciada.			
Originador: Tester		Asignado: Dev Backend	

Hoja de información de riesgo			
Riesgo ID: R-04	Fecha: 14/06/26	Prob: 65%	Impacto: Medio
Descripción: El navegador del usuario puede sufrir un bloqueo, desconexión del micrófono, cierre de la pestaña o interrupción de la red eléctrica a mitad de una clase larga, lo que impediría la subida final del archivo de audio grabado.			
Refinamiento/Contexto: Subcondición 1: La grabación de apuntes de voz no se almacena de manera progresiva en el dispositivo local, sino que se mantiene en la memoria volátil del objeto MediaRecorder . Subcondición 2: Una grabación promedio dura entre 45 y 90 minutos, lo que aumenta la probabilidad de que ocurra una interrupción en ese intervalo.			
Mitigación/monitoreo: 1. Utilizar almacenamiento local del navegador (IndexedDB o la API de archivos locales de HTML5) para guardar fragmentos pequeños (chunks) de audio cada 30 segundos durante la grabación. 2. Añadir un detector en Vue que advierta al usuario si intenta cerrar la pestaña mientras una grabación de apuntes sigue activa.			
Manejo/plan de contingencia/disparador: Plan de contingencia: • Si la pestaña se cierra o el navegador se cae, al reiniciar la aplicación se detectará el archivo temporal guardado en IndexedDB y se le ofrecerá al usuario subirlo manualmente o reanudar su edición. Disparador: Pérdida abrupta del estado activo de la grabación o desconexión en el frontend sin envío exitoso al endpoint de Laravel.			
Estado actual: 14/06/2026: Integración inicial de advertencia beforeunload en la pantalla de grabación de apuntes.			
Originador: Tester		Asignado: Desarrollador frontend	

Hoja de información de riesgo			
Riesgo ID: R-05	Fecha: 14/06/26	Prob: 80%	Impacto: Medio
Descripción: El reinicio del estado diario de tareas y rachas de estudio a las 00:00 del servidor causa que usuarios de distintas regiones horarias sufran la ruptura repentina e involuntaria de sus rachas acumuladas debido a la falta de adaptabilidad del horario del backend al huso local del cliente.			
Refinamiento/Contexto: Subcondición 1: El sistema almacena la fecha de la última sesión en formato de fecha y hora local del servidor (ej. UTC). Subcondición 2: El validador de racha utiliza <code>Carbon::today()</code> del backend para calcular la diferencia de días transcurridos. Subcondición 3: La diferencia horaria entre el servidor y el cliente puede ser de más de 12 horas dependiendo de la región geográfica.			
Mitigación/monitoreo: 1. Enviar siempre el desfase horario (timezone) del cliente o su marca de tiempo Unix local en cada inicio de sesión y guardado de tareas. 2. Calcular la validez del día en base al huso horario registrado del usuario en su cuenta, adaptando <code>Carbon</code> al huso horario correspondiente del usuario (ej. <code>America/Argentina/Buenos_Aires</code>)			
Manejo/plan de contingencia/disparador: Plan de contingencia: • Ofrecer una opción de "Protector de Racha" o reintegro de racha automático si el sistema detecta que el usuario completó una tarea en su fecha local del día anterior y la discrepancia fue debida a desfase horaria de procesamiento. Disparador: Diferencia horaria mayor a 3 horas entre la fecha de guardado registrada en base de datos y la hora local del navegador del cliente al procesar el estado de rachas.			
Estado actual: 14/06/2026: Modificación del controlador de Rachas para guardar la fecha en UTC y convertirla a la zona del usuario localmente en Vue.			
Originador: Tester		Asignado: Desarrollador backend	

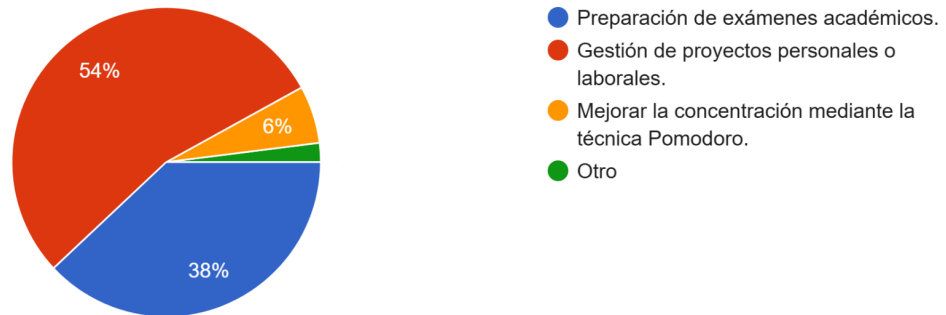
Conclusión

Anexos

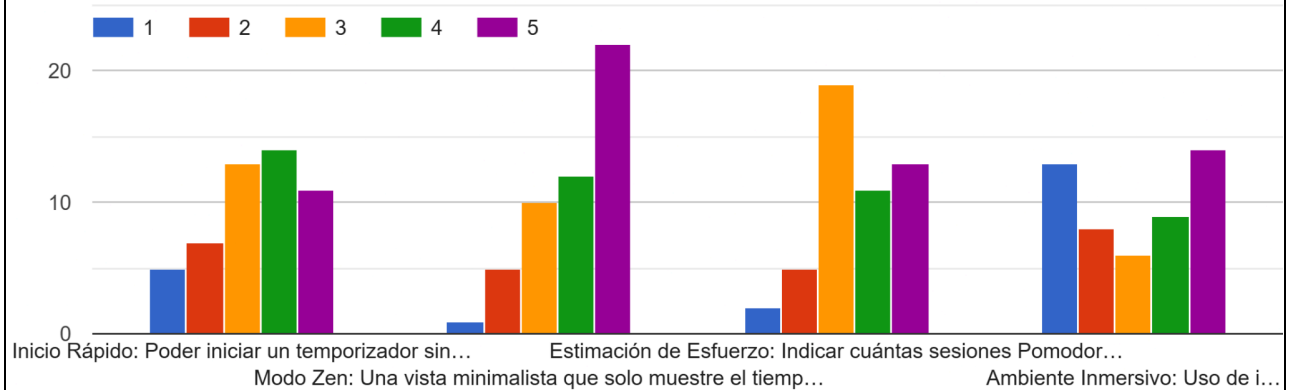
Respuestas del Cuestionario

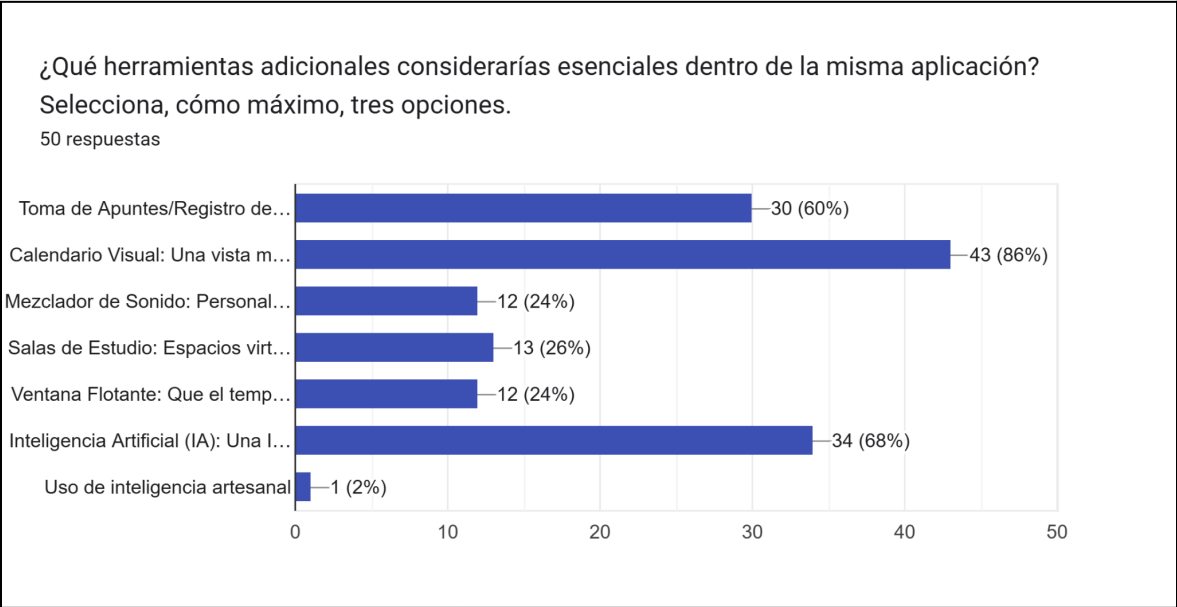
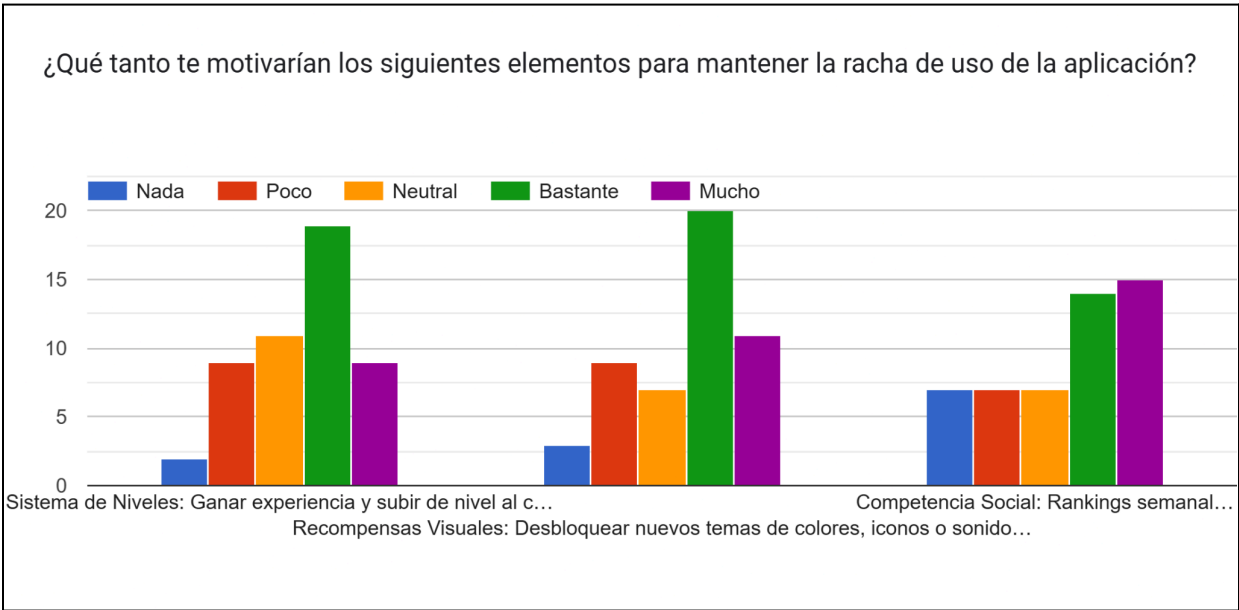
¿Cuál es tu principal motivo para usar un gestor de tareas?

50 respuestas



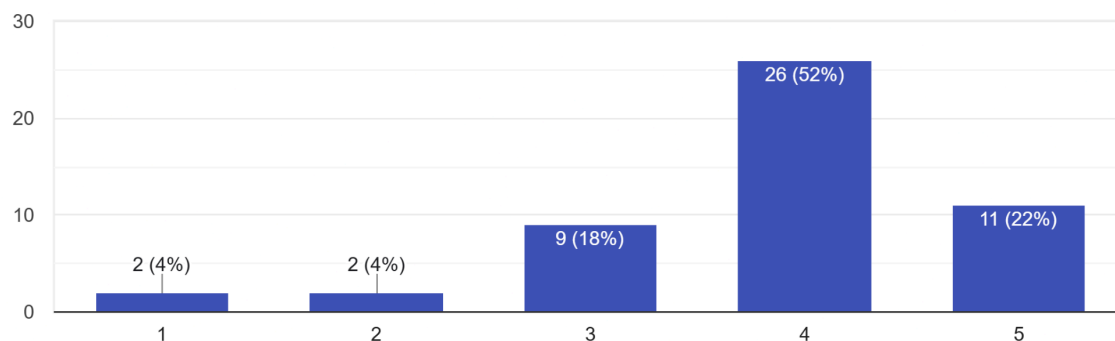
Experiencia de Uso: Califica del 1 al 5 (donde 1 es 'Nada importante' y 5 es 'Muy importante') la importancia de estas propuestas:





¿Qué tan relevante es poder integrar Cronos con otras herramientas que ya utilizas (ej. Google Calendar, Spotify)?

50 respuestas



¿Cómo preferirías que el sistema maneje la procrastinación?

50 respuestas

